

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



NGUYỄN TRỌNG NHÂN

**MẠNG ĐỐI NGHỊCH TẠO SINH TRONG CHUYỂN ĐỔI
ẢNH CHÂN DUNG SANG PHONG CÁCH ANIME**

LUẬN VĂN THẠC SĨ KHOA HỌC MÁY TÍNH

HÀ NỘI - 2026

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



NGUYỄN TRỌNG NHÂN

**MẠNG ĐỐI NGHỊCH TẠO SINH TRONG CHUYỂN ĐỔI
ẢNH CHÂN DUNG SANG PHONG CÁCH ANIME**

Ngành: Khoa học máy tính
Chuyên ngành: Khoa học máy tính
Mã số: 7480101

LUẬN VĂN THẠC SĨ KHOA HỌC MÁY TÍNH

NGƯỜI HƯỚNG DẪN KHOA HỌC: TS. MA THỊ CHÂU

HÀ NỘI - 2026

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



NGUYEN TRONG NHAN

**GENERATIVE ADVERSARIAL NETWORKS FOR
CONVERTING PORTRAITS TO ANIME STYLE**

Major: Computer Science
Specialization: Computer Science
Code: 7480101

MASTER'S THESIS IN COMPUTER SCIENCE

SUPERVISOR: DR. MA THI CHAU

HANOI - 2026

LỜI CẢM ƠN

Trước hết, tôi xin chân thành cảm ơn TS. Ma Thị Châu, người hướng dẫn đề tài nghiên cứu lần này, đã tận tình hỗ trợ, chỉ bảo và động viên tôi trong suốt quá trình nghiên cứu và hoàn thành. Nhờ có sự hướng dẫn của cô, tôi đã có được những kiến thức quý báu và kinh nghiệm thực tiễn trong lĩnh vực nghiên cứu của mình.

Tôi cũng xin gửi lời cảm ơn đến các thầy cô giáo của Trường Đại học Công Nghệ, đã truyền đạt cho tôi những kiến thức cơ bản và nâng cao về ngành Khoa học Máy Tính. Tôi xin chúc các thầy cô luôn mạnh khỏe và thành công trong công tác giảng dạy và nghiên cứu khoa học.

Tôi xin gửi lời cảm ơn đặc biệt đến tác giả của bài báo DTGAN (AnimeGANv3) – anh Asher Chan (asher_chan@foxmail.com) – đã nhiệt tình hỗ trợ tôi trong việc lý giải mã nguồn và kiến trúc mô hình trong suốt quá trình nghiên cứu. Sự giúp đỡ của anh đã giúp tôi hiểu sâu hơn về các chi tiết kỹ thuật và hoàn thành luận văn một cách trọn vẹn.

Sau cùng, tôi muốn bày tỏ lòng biết ơn sâu sắc đến gia đình, người thân và bạn bè, những người luôn ở bên cạnh tôi trong những thời điểm khó khăn nhất, luôn động viên và khích lệ tôi trong cuộc sống và công việc. Thành tựu này của tôi không thể có được nếu thiếu sự giúp đỡ của họ.

Mặc dù tôi đã cố gắng hoàn thành báo cáo nhưng không thể tránh khỏi những sai sót, tôi rất mong nhận được nhận xét và sự hướng dẫn từ phía giáo viên và hội đồng.

Hà Nội, ngày ... tháng ... năm 2026

Học viên

Nguyễn Trọng Nhân

LỜI CAM ĐOAN

Tôi xin cam đoan rằng tất cả các nội dung trong tài liệu này đều là kết quả của công trình nghiên cứu cá nhân của tôi dưới sự hướng dẫn của TS.Ma Thị Châu. Tôi không sao chép bất kỳ tài liệu hay công trình nghiên cứu nào của người khác mà không chỉ rõ nguồn gốc trong phần tài liệu tham khảo. Tôi hiểu rằng việc sao chép trái phép là một hành vi vi phạm đạo đức học thuật và tôi sẽ chịu trách nhiệm về những hành vi này.

Tôi cam kết rằng, bản báo cáo của tôi không vi phạm bản quyền của bất kỳ ai và không vi phạm bất kỳ quyền sở hữu nào, cũng như bất kỳ ý tưởng, kỹ thuật, trích dẫn hoặc bất kỳ tài liệu nào từ công trình nghiên cứu của người khác. Tôi xin nhận đầy đủ trách nhiệm và sẵn sàng chấp nhận mọi biện pháp kỷ luật nếu vi phạm cam kết.

Hà Nội, ngày ... tháng ... năm 2026

Học viên

Nguyễn Trọng Nhân

TÓM TẮT

Tóm tắt: Sự phát triển mạnh mẽ của thị trường Anime/Manga toàn cầu cùng với nhu cầu ngày càng lớn về các công cụ sáng tạo nội dung dựa trên trí tuệ nhân tạo đã thúc đẩy nghiên cứu bài toán chuyển đổi ảnh chân dung người thật sang phong cách anime 2D (photo-to-anime). Đây là bài toán giàu tiềm năng ứng dụng nhưng đặt ra nhiều thách thức về chất lượng hình ảnh, khả năng bảo toàn đặc điểm nhận dạng khuôn mặt và tính ổn định khi huấn luyện. Các phương pháp trước đó như Neural Style Transfer hay CycleGAN thường gặp hạn chế về nhiễu, tạo tác (artifacts) và tốc độ xử lý.

Luận văn này nghiên cứu, tái hiện và mở rộng mô hình AnimeGANv3 dựa trên kiến trúc Double-Tail GAN (DTGAN) – một kiến trúc Mạng đối nghịch tạo sinh (GAN) thế hệ mới sử dụng Generator hai nhánh (Support Tail và Main Tail), kỹ thuật chuẩn hóa Linearly Adaptive Denormalization (LADE), cơ chế External Attention v3 và hệ thống hàm mất mát chuyên biệt kết hợp mất mát đối nghịch, mất mát nội dung (dựa trên VGG19), mất mát phong cách, mất mát màu sắc và mất mát làm mượt vùng. Đóng góp chính của luận văn là xây dựng và tích hợp module Face Mode sử dụng RetinaFace MobileNet 0.25 kết hợp thuật toán Margin Expansion, tạo thành pipeline hoàn chỉnh: phát hiện khuôn mặt → cắt và mở rộng vùng mặt → chuyển đổi anime chất lượng cao.

Mô hình được huấn luyện trên nền tảng điện toán đám mây vast.ai với hai tập dữ liệu không ghép cặp gồm ảnh chân dung người thật và ảnh anime phong cách Hayao Miyazaki, Makoto Shinkai. Quá trình huấn luyện qua 2 giai đoạn (5 epoch pre-training và 69 epoch adversarial) cho thấy hội tụ ổn định. Kết quả thực nghiệm đạt FID = 58,6 và LPIPS = 0,384 ở chế độ Face Mode – vượt trội so với CycleGAN, CartoonGAN và AnimeGANv2. Module Face Mode cải thiện khoảng 8,7% FID so với chế độ Landscape Mode. Model ONNX chỉ 9,1 MB đạt tốc độ 42 FPS tại 256×256 trên GPU. Hệ thống được triển khai thành ứng dụng web hoàn chỉnh (React frontend và FastAPI backend), hỗ trợ chuyển đổi cả ảnh và video. Luận văn cũng đề xuất các hướng phát triển tương lai bao gồm face alignment, face blending, temporal consistency cho video và mở rộng đa phong cách.

Từ khóa: GAN, AnimeGANv3, DTGAN, chuyển phong cách ảnh, ảnh chân dung, phong cách anime, LADE, RetinaFace, Face Mode, ONNX.

ABSTRACT

Abstract: The rapid growth of the global Anime/Manga market, coupled with the increasing demand for AI-powered content creation tools, has driven research into the problem of converting real-person portraits to 2D anime style (photo-to-anime). This task offers significant application potential but poses many challenges regarding image quality, facial identity preservation, and training stability. Previous approaches such as Neural Style Transfer and CycleGAN often suffer from noise, artifacts, and slow processing speed.

This thesis investigates, reproduces, and extends the AnimeGANv3 model based on the Double-Tail GAN (DTGAN) architecture – a next-generation Generative Adversarial Network (GAN) employing a dual-branch Generator (Support Tail and Main Tail), Linearly Adaptive Denormalization (LADE), External Attention v3, and a specialized loss system combining adversarial loss, content loss (based on VGG19), style loss, color loss, and region smoothing loss. The main contribution of this thesis is the design and integration of a Face Mode module using RetinaFace MobileNet 0.25 combined with a Margin Expansion algorithm, forming a complete pipeline: face detection → face region cropping and expansion → high-quality anime conversion.

The model was trained on the vast.ai cloud computing platform using two unpaired datasets consisting of real-person portraits and anime images in the styles of Hayao Miyazaki and Makoto Shinkai. The two-phase training process (5 epochs of pre-training and 69 epochs of adversarial training) demonstrated stable convergence. Experimental results achieved $FID = 58.6$ and $LPIPS = 0.384$ in Face Mode – outperforming CycleGAN, CartoonGAN, and AnimeGANv2. The Face Mode module improved FID by approximately 8.7% compared to Landscape Mode. The ONNX model is only 9.1 MB and achieves 42 FPS at 256×256 on GPU. The system was deployed as a complete web application (React frontend and FastAPI backend), supporting both image and video conversion. The thesis also proposes future development directions including face alignment, face blending, temporal consistency for video, and multi-style expansion.

Keywords: GAN, AnimeGANv3, DTGAN, image style transfer, portrait, anime style, LADE, RetinaFace, Face Mode, ONNX.

MỤC LỤC

Lời cảm ơn	
Lời cam đoan	
Tóm tắt	
Abstract	
Mục lục	i
Danh mục hình vẽ	v
Danh mục bảng biểu	vii
MỞ ĐẦU	1
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN NGHIÊN CỨU	4
1.1. Bài toán Chuyển ảnh sang Phong cách Anime	4
1.1.1. Định nghĩa bài toán	4
1.1.2. Đặc trưng thị giác của phong cách Anime	4
1.1.3. Thách thức kỹ thuật	4
1.2. Nền tảng Mạng Nơ-ron Tích chập	5
1.2.1. Lớp Tích chập, Pooling và Hàm kích hoạt	5
1.2.2. Các Kỹ thuật Chuẩn hóa	6
1.2.3. Kiến trúc Encoder-Decoder	7
1.3. Kiến trúc Mạng Đối nghịch Tạo sinh	8
1.3.1. Nguyên lý Hoạt động và Hàm Mục tiêu	8
1.3.2. Thách thức trong Huấn luyện GAN	9
1.3.3. Các Giải pháp về Hàm Mục tiêu	11
1.3.4. Đánh giá Chất lượng GAN	13
1.4. Các mô hình GAN tiêu biểu cho bài toán Anime	13
1.4.1. CycleGAN và dịch ảnh không ghép cặp	14
1.4.2. StyleGAN và GAN Inversion	14
1.4.3. AnimeGANv3 (DTGAN) – Sơ bộ kiến trúc	14
1.4.4. So sánh tổng quan và Định hướng	15
1.5. Phát hiện Khuôn mặt – Sơ bộ	16
1.5.1. Tổng quan các Phương pháp	16
1.5.2. RetinaFace – Sơ bộ Kiến trúc	17
CHƯƠNG 2. KIẾN TRÚC MÔ HÌNH AnimeGANv3 (DTGAN)	20
2.1. Tổng quan kiến trúc DTGAN	20
2.2. Mạng Generator (G_net)	22
2.2.1. Base Encoder (Chia sẻ)	22

2.2.2.	Support Tail (Decoder nhánh hỗ trợ)	22
2.2.3.	Main Tail (Decoder nhánh chính)	23
2.3.	Kỹ thuật chuẩn hóa LADE	23
2.3.1.	Động lực và ý tưởng	23
2.3.2.	Công thức toán học của LADE	23
2.3.3.	So sánh LADE với các kỹ thuật chuẩn hóa khác	25
2.3.4.	Biến thể LADE_D trong Discriminator	25
2.4.	Cơ chế External Attention v3	25
2.4.1.	Hạn chế của Self-Attention và Giải pháp	25
2.4.2.	Kiến trúc External Attention v3	26
2.5.	Discriminator (D_net)	27
2.6.	Hệ thống Hàm Mất mát	27
2.6.1.	Hàm mất mát Giai đoạn Pre-training	28
2.6.2.	Content Loss (Perceptual Loss)	29
2.6.3.	Style Loss (Decentralized Gram Matrix)	29
2.6.4.	Region Smoothing Loss	29
2.6.5.	Lab Color Loss	29
2.6.6.	Total Variation Loss	30
2.6.7.	Adversarial Loss (LSGAN)	30
2.6.8.	Fine-grained Revision Loss (Main Tail)	30
2.6.9.	Tổng hợp Loss	30
2.7.	Xử lý Hậu kỳ: Guided Filter	31
2.8.	Quy trình Huấn luyện	31
2.8.1.	Hai giai đoạn huấn luyện	31
2.8.2.	Siêu tham số huấn luyện	32
2.8.3.	Tiền xử lý Dữ liệu Huấn luyện	32
2.9.	Phân tích và Điểm nổi bật của DTGAN	32
2.9.1.	Ưu điểm so với AnimeGAN và AnimeGANv2	32
CHƯƠNG 3. LUỒNG ANIME TÍCH HỢP PHÁT HIỆN KHUÔN MẶT		35
3.1.	Tổng quan Pipeline Tích hợp	35
3.2.	Phát hiện Khuôn mặt với RetinaFace	36
3.2.1.	Tổng quan RetinaFace	36
3.2.2.	Kiến trúc Mạng Phát hiện	37
3.2.3.	Tiền xử lý Ảnh Đầu vào	38
3.2.4.	Hậu xử lý và Prior Box	38
3.3.	Kỹ thuật Mở rộng Vùng Khuôn mặt (Margin Expansion)	39
3.3.1.	Động lực và Mục tiêu	39
3.3.2.	Thuật toán Margin Expansion	40

3.3.3.	Xử lý Trường hợp Đặc biệt (Edge Cases)	41
3.3.4.	Ý nghĩa của Tham số margin = 0.5	42
3.4.	Luồng xử lý Ảnh trong Pipeline	42
3.4.1.	Đọc và Giới hạn Kích thước Ảnh	42
3.4.2.	Xử lý Phân nhánh: Face / Landscape / Hybrid	42
3.4.3.	Chuẩn hóa và Suy luận ONNX	43
3.4.4.	Tóm tắt Luồng Dữ liệu	43
3.5.	Chế độ Hybrid Mode	44
3.5.1.	Nguyên lý Hoạt động	44
3.5.2.	Ánh xạ Tọa độ giữa Ảnh Giới hạn và Ảnh Gốc	45
3.5.3.	Kỹ thuật Feathered Blending	46
3.5.4.	Phân tích Độ phức tạp	47
3.6.	Phân tích Kỹ thuật và Đánh giá Thiết kế	48
3.6.1.	Lý do Chọn RetinaFace MobileNet 0.25	48
3.6.2.	Đánh giá Pipeline Tổng thể	49
CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ		50
4.1.	Môi trường Thực nghiệm	50
4.1.1.	Cấu hình Phần cứng	50
4.1.2.	Cấu hình Phần mềm	51
4.2.	Bộ Dữ liệu	51
4.2.1.	Dữ liệu Ảnh Thực (Photo)	51
4.2.2.	Dữ liệu Ảnh Anime (Style)	51
4.3.	Kết quả Huấn luyện	52
4.3.1.	Giai đoạn Pre-training Generator	52
4.3.2.	Giai đoạn Huấn luyện Đầy đủ	53
4.4.	Đánh giá Định lượng	55
4.4.1.	Các Chỉ số Đánh giá	55
4.4.2.	Kết quả So sánh Định lượng	56
4.4.3.	Tốc độ Suy luận theo Độ Phân giải	56
4.5.	Đánh giá Định tính	56
4.5.1.	So sánh ba Chế độ: Hybrid, Landscape và Face Mode	56
4.5.2.	Phân tích Ảnh hưởng của Module Face Mode	58
4.6.	Đánh giá Module Face Mode	58
4.6.1.	Độ chính xác Phát hiện Khuôn mặt	58
4.6.2.	Phân tích Tham số margin	59
4.6.3.	Xử lý Trường hợp Đặc biệt	60
4.7.	Thảo luận và Phân tích	60
4.7.1.	Điểm mạnh của Hệ thống	60

4.7.2. Hạn chế và Định hướng Cải thiện	60
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	62
5.1. Kết luận	62
5.1.1. Mức độ hoàn thành mục tiêu	63
5.2. Hạn chế	63
5.3. Hướng phát triển	64
5.3.1. Cải thiện chất lượng xử lý khuôn mặt	64
5.3.2. Xử lý video với temporal consistency	64
5.3.3. Mở rộng đa phong cách và triển khai mobile	64
5.4. Lời kết	65
TÀI LIỆU THAM KHẢO	66

DANH MỤC HÌNH VẼ

Hình 1.1	Kiến trúc cơ bản của Mạng Nơ-ron Tích chập	5
Hình 1.2	So sánh các hàm kích hoạt: Sigmoid, Tanh, ReLU và Leaky ReLU	6
Hình 1.3	So sánh bốn kỹ thuật chuẩn hóa trên tensor ($N \times C \times H \times W$)	7
Hình 1.4	Kiến trúc U-Net với skip connections	8
Hình 1.5	Kiến trúc tổng quát của Mạng Đối nghịch Tạo sinh (GAN)	9
Hình 1.6	Minh họa hiện tượng Mode Collapse trong GAN	10
Hình 1.7	Động lực học huấn luyện GAN và vấn đề Không Hội tụ	11
Hình 1.8	So sánh gradient giữa GAN gốc (JS) và WGAN (Wasserstein)	12
Hình 1.9	Quy trình tính FID và IS qua mạng Inception-v3	13
Hình 1.10	Kiến trúc CycleGAN với hai Generator và ràng buộc cycle consistency	14
Hình 1.11	Sơ đồ tổng quát của Double-Tail GAN (DTGAN)	15
Hình 1.12	Biểu đồ radar so sánh đa chiều ba hướng tiếp cận GAN cho bài toán anime	16
Hình 1.13	Tổng quan kiến trúc RetinaFace và pipeline suy luận	18
Hình 2.1	Tổng quan kiến trúc DTGAN (AnimeGANv3)	21
Hình 2.2	Cơ chế hoạt động của LADE	24
Hình 2.3	Kiến trúc External Attention v3 sử dụng trong DTGAN tại vị trí bottleneck 32×32	26
Hình 2.4	Pipeline huấn luyện hai giai đoạn và hệ thống hàm mất mát của DTGAN	28
Hình 3.1	Pipeline tích hợp Face Extraction vào AnimeGANv3	36
Hình 3.2	Kiến trúc RetinaFace với backbone MobileNet 0.25 và Feature Pyramid Network (FPN)	37
Hình 3.3	Minh họa kỹ thuật Margin Expansion	40
Hình 3.4	Pipeline xử lý Hybrid Mode: kết hợp Landscape Inference cho nền và Face Inference cho từng khuôn mặt	45
Hình 3.5	Kỹ thuật Feathered Blending: mặt nạ alpha với viền mờ Gaussian tạo vùng chuyển tiếp liền mạch	46
Hình 4.1	Đường cong hội tụ Pre-train G_loss trong giai đoạn pre-training (Epoch 0–4)	52
Hình 4.2	Đường cong G_loss (xanh lam) và D_loss (cam) qua 28.632 steps trong giai đoạn huấn luyện đầy đủ (Epoch 5–73)	53

Hình 4.3	Đường cong chi tiết bốn thành phần loss phụ trong giai đoạn adversarial training	54
Hình 4.4	Giá trị loss trung bình theo epoch: (trái) G_loss và D_loss, (phải) các thành phần loss phụ	55
Hình 4.5	So sánh kết quả chuyển đổi anime giữa Hybrid, Landscape và Face Mode trên ảnh chụp thẳng mặt	57
Hình 4.6	So sánh ba chế độ trên ảnh nhóm người	57
Hình 4.7	So sánh ba chế độ trên ảnh chân dung góc nghiêng	58

DANH MỤC BẢNG BIỂU

Bảng 1.1	Tóm tắt các chế độ thất bại của GAN	11
Bảng 1.2	So sánh định lượng các mô hình chuyển ảnh sang anime trên tập Selfie2Anime [55]	15
Bảng 1.3	So sánh các phương pháp phát hiện khuôn mặt	17
Bảng 2.1	Cấu trúc chi tiết Base Encoder của DTGAN	22
Bảng 2.2	So sánh Support Tail và Main Tail trong DTGAN	23
Bảng 2.3	So sánh các kỹ thuật chuẩn hóa trong mạng chuyển đổi phong cách ảnh	25
Bảng 2.4	Siêu tham số huấn luyện mặc định của DTGAN	32
Bảng 2.5	So sánh DTGAN với các mô hình chuyển đổi ảnh anime tiền nhiệm	33
Bảng 3.1	Cấu hình Prior Box của RetinaFace với backbone MobileNet 0.25	38
Bảng 3.2	So sánh ba chế độ xử lý trong pipeline	42
Bảng 3.3	Phân tích độ phức tạp thời gian giữa ba chế độ	48
Bảng 3.4	So sánh các phương pháp phát hiện khuôn mặt cho ứng dụng thực tế	48
Bảng 4.1	Cấu hình phần cứng sử dụng trong thực nghiệm	50
Bảng 4.2	Cấu hình phần mềm và thư viện sử dụng	51
Bảng 4.3	Tóm tắt các chỉ số đánh giá định lượng sử dụng trong thực nghiệm	55
Bảng 4.4	Kết quả đánh giá định lượng trên tập ảnh chân dung (FID đo trên 1.000 ảnh, ONNX inference trên GPU RTX A5000)	56
Bảng 4.5	Tốc độ suy luận ONNX theo độ phân giải ảnh đầu vào	56
Bảng 4.6	So sánh chất lượng chuyển đổi có và không sử dụng module Face Mode	58
Bảng 4.7	Kết quả phát hiện khuôn mặt theo kịch bản khác nhau (ngưỡng confidence = 0.8)	59
Bảng 4.8	Ảnh hưởng của tham số margin đến chất lượng ảnh đầu ra và mức độ bao phủ ngữ cảnh	59
Bảng 4.9	Kết quả xử lý các trường hợp đặc biệt trong module Face Mode	60
Bảng 5.1	Đánh giá mức độ hoàn thành các mục tiêu nghiên cứu	63

MỞ ĐẦU

Tính cấp thiết và ý nghĩa của đề tài

Sự phát triển mạnh mẽ của nghệ thuật kỹ thuật số cùng với thị trường Anime/Manga toàn cầu đã tạo động lực lớn cho các nghiên cứu về chuyển đổi phong cách hình ảnh. Theo thống kê năm 2024, quy mô thị trường anime toàn cầu đạt khoảng 81,96 tỷ USD và dự kiến vượt 200 tỷ USD vào năm 2034. Công nghệ AI hiện nay cũng đang dần xâm nhập vào quy trình sản xuất anime – ví dụ, các công cụ Generative AI đã có thể tự động hoá việc vẽ phông nền và tô màu, giúp giảm bớt công việc lặp lại cho các họa sĩ. Trong bối cảnh đó, nhu cầu tự động hoá chuyển đổi ảnh thực sang phong cách anime trở nên cấp thiết, nhằm phục vụ cộng đồng người hâm mộ khổng lồ và ngành công nghiệp sáng tạo nội dung đang bùng nổ.

Tuy nhiên, việc chuyển một ảnh chân dung người thật sang tranh anime là thách thức không nhỏ. Phong cách anime có đặc trưng rất khác biệt (đường nét đơn giản, mắt to, màu sắc phẳng, v.v.), trong khi ảnh chụp chứa nhiều chi tiết thực tế phức tạp. Các phương pháp truyền thống dựa trên Neural Style Transfer thường gặp khó khăn trong việc giữ được nội dung gốc và dễ sinh ra nhiễu, tạo tác (artifacts) khi khác biệt phong cách quá lớn. Thêm vào đó, các phương pháp như CycleGAN tuy có khả năng học dạng chuyển đổi không ghép cặp nhưng còn chậm và chất lượng đầu ra chưa ổn định. Vẽ tay thủ công bởi các họa sĩ tuy chất lượng cao nhưng tốn kém thời gian và công sức. Do đó, bài toán đặt ra là làm thế nào để tự động hoá quá trình này mà vẫn đảm bảo chất lượng cao và bảo toàn được đặc điểm nhận dạng của đối tượng trong ảnh gốc.

Mạng Đối nghịch Tạo sinh (GAN) nổi lên như một công nghệ đột phá có thể giải quyết các nhiệm vụ tổng hợp hình ảnh phức tạp. Được Ian Goodfellow giới thiệu năm 2014 [11] và được Yann LeCun ca ngợi là “ý tưởng thú vị nhất của ML trong 10 năm”, GAN đã chứng tỏ hiệu quả vượt trội trong việc sinh ảnh chân thực từ dữ liệu huấn luyện. Đặc biệt trong bài toán dịch chuyển phong cách (style transfer), GANs cung cấp một khuôn khổ linh hoạt để huấn luyện mô hình tạo ảnh anime từ ảnh thật mà không đòi hỏi dữ liệu ảnh cặp một-một. Trong số các mô hình GAN ứng dụng cho anime hóa ảnh, AnimeGANv3 (DTGAN) [53] nổi bật với kiến trúc Double-Tail Generator sáng tạo, kỹ thuật chuẩn hóa LADE và bộ hàm mất mát chuyên biệt – đạt được đồng thời tốc độ nhanh và chất lượng ảnh cao. Luận văn này nghiên cứu, tái hiện và mở rộng hệ thống đó bằng cách tích hợp thêm module **trích xuất khuôn mặt (Face Extraction)** sử dụng RetinaFace, tạo thành một pipeline hoàn chỉnh từ đầu đến cuối: từ ảnh chân dung thô đến ảnh anime chất lượng cao.

Mục tiêu nghiên cứu

Mục tiêu tổng quát của luận văn là nghiên cứu, tái hiện và mở rộng mô hình AnimeGANv3 (DTGAN) để xây dựng một hệ thống chuyển đổi ảnh chân dung người thật sang phong cách anime chất lượng cao, có tích hợp module trích xuất khuôn mặt tự động. Cụ thể, luận văn hướng đến các mục tiêu sau:

1. Nghiên cứu và trình bày nền tảng lý thuyết về mạng nơ-ron tích chập (CNN), các biến thể GAN ứng dụng trong chuyển đổi phong cách ảnh, cùng với các kỹ thuật phát hiện khuôn mặt và tiền xử lý ảnh liên quan.
2. Phân tích chi tiết kiến trúc AnimeGANv3 (DTGAN): cấu trúc Generator hai nhánh (Double-Tail), kỹ thuật chuẩn hóa LADE, cơ chế External Attention v3 và hệ thống hàm mất mát đặc thù.
3. Xây dựng và tích hợp module trích xuất khuôn mặt (Face Extraction) sử dụng RetinaFace kết hợp kỹ thuật mở rộng vùng mặt (Margin Expansion), tạo thành pipeline hoàn chỉnh: Ảnh đầu vào → Phát hiện khuôn mặt → Cắt & mở rộng vùng mặt → Chuyển đổi anime → Kết quả.
4. Huấn luyện và đánh giá toàn diện mô hình trên bộ dữ liệu phong cách Hayao Miyazaki và Makoto Shinkai bằng các chỉ số định lượng (FID, LPIPS, FPS) và đánh giá định tính.
5. Xây dựng ứng dụng demo (GUI sử dụng PyQt5 hoặc Web) triển khai suy luận qua ONNX Runtime để minh chứng khả năng ứng dụng thực tiễn của hệ thống.

Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Kiến trúc mạng Double-Tail Generative Adversarial Network (DTGAN) áp dụng cho bài toán chuyển đổi phong cách ảnh chân dung sang anime. Cụ thể, đề tài tập trung vào: (1) các thành phần cấu trúc của Generator và Discriminator trong AnimeGANv3, (2) kỹ thuật chuẩn hóa LADE và cơ chế External Attention v3, (3) hệ thống hàm mất mát đặc thù cho bài toán anime hóa, và (4) module phát hiện – trích xuất khuôn mặt sử dụng RetinaFace.

Phạm vi nghiên cứu:

- *Dữ liệu:* Ảnh chân dung người thật (portrait photos) làm đầu vào; ảnh anime phong cách Hayao Miyazaki và Makoto Shinkai làm tập dữ liệu phong cách. Tập dữ liệu ở dạng không ghép cặp (unpaired).
- *Mô hình:* Sử dụng mô hình AnimeGANv3 (DTGAN) [53] làm nền tảng, có mở rộng thêm module Face Extraction dựa trên RetinaFace [44] với backbone MobileNet 0.25.

- *Triển khai*: Mô hình được chuyển đổi và triển khai suy luận qua ONNX Runtime, hỗ trợ cả ảnh đơn và xử lý theo lô (batch processing).
- *Ngoài phạm vi*: Chuyển đổi phong cách cho video thời gian thực, các phong cách hoạt hình khác (cartoon phương Tây, 3D), và các kiến trúc mới hơn như Diffusion Models hay Vision Transformers (chỉ tập trung vào GAN giai đoạn 2018–2024).

Cấu trúc luận văn

Luận văn được tổ chức theo cấu trúc như sau:

- **Mở đầu**: Trình bày lý do chọn đề tài, sự cần thiết, mục tiêu, đối tượng, phạm vi nghiên cứu và giới thiệu cấu trúc của luận văn.
- **Chương 1: Cơ sở lý thuyết và tổng quan nghiên cứu**. Giới thiệu bài toán chuyển ảnh sang phong cách anime, nền tảng mạng nơ-ron tích chập (CNN), lý thuyết mạng đối nghịch tạo sinh (GAN) và các biến thể (WGAN, LSGAN, CycleGAN), các mô hình GAN tiêu biểu cho bài toán anime, và sơ bộ về phương pháp phát hiện khuôn mặt RetinaFace.
- **Chương 2: Kiến trúc mô hình AnimeGANv3 (DTGAN)**. Phân tích chi tiết kiến trúc mạng Double-Tail GAN: Base Encoder, Support Tail và Main Tail của Generator; hai Discriminator; kỹ thuật chuẩn hóa LADE; cơ chế External Attention v3; hệ thống hàm mất mát; pipeline tiền xử lý dữ liệu huấn luyện và quy trình huấn luyện.
- **Chương 3: Luồng anime tích hợp phát hiện khuôn mặt**. Trình bày pipeline end-to-end tích hợp phát hiện khuôn mặt (RetinaFace) và chuyển đổi phong cách anime (AnimeGANv3), kỹ thuật mở rộng vùng mặt (Margin Expansion), và phân tích hai chế độ xử lý Face Mode/Landscape Mode.
- **Chương 4: Thực nghiệm và đánh giá**. Trình bày môi trường thực nghiệm, bộ dữ liệu, kết quả huấn luyện, đánh giá định lượng (FID, LPIPS, FPS) và định tính, triển khai ứng dụng web (React + FastAPI) và demo hệ thống.
- **Kết luận**: Tóm tắt những kết quả nổi bật, đóng góp của đề tài, thảo luận các hạn chế còn tồn tại và đề xuất phương hướng mở rộng trong tương lai.

CHƯƠNG 1

CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN NGHIÊN CỨU

1.1. Bài toán Chuyển ảnh sang Phong cách Anime

1.1.1. Định nghĩa bài toán

Bài toán chuyển ảnh sang phong cách anime thuộc lớp bài toán *dịch ảnh giữa các miền* (image-to-image translation): cho một ảnh chụp thực tế $x \in \mathcal{X}$ thuộc miền ảnh thật, tìm ánh xạ $G : \mathcal{X} \rightarrow \mathcal{Y}$ sao cho ảnh đầu ra $G(x) \in \mathcal{Y}$ mang phong cách anime nhưng vẫn bảo toàn nội dung ngữ nghĩa của ảnh gốc [26, 30]. Đặc thù của bài toán này là dữ liệu huấn luyện ở dạng **không ghép cặp** (unpaired): ta chỉ có tập ảnh thật $\{x_i\}$ và tập ảnh anime $\{y_j\}$ độc lập, không có sự tương ứng một-một giữa hai tập [30].

1.1.2. Đặc trưng thị giác của phong cách Anime

Phong cách anime có các đặc trưng thị giác khác biệt rõ ràng so với ảnh chụp thực tế, bao gồm [42, 53]:

- **Đường nét viền rõ ràng:** Các đối tượng được phân tách bởi các đường viền đen hoặc tối, tạo hiệu ứng “ink outline” đặc trưng.
- **Vùng màu phẳng (cel-shading):** Thay vì gradient mịn như ảnh thật, anime sử dụng các vùng màu đồng nhất với ranh giới rõ ràng giữa vùng sáng và vùng tối.
- **Đơn giản hóa chi tiết:** Kết cấu bề mặt (texture) như da, tóc, vải được tối giản hoá, loại bỏ phần lớn chi tiết thực tế.
- **Phân phối màu sắc đặc thù:** Màu sắc anime thường tươi sáng hơn, bão hoà hơn và có phân phối brightness khác biệt đáng kể so với ảnh chụp.

1.1.3. Thách thức kỹ thuật

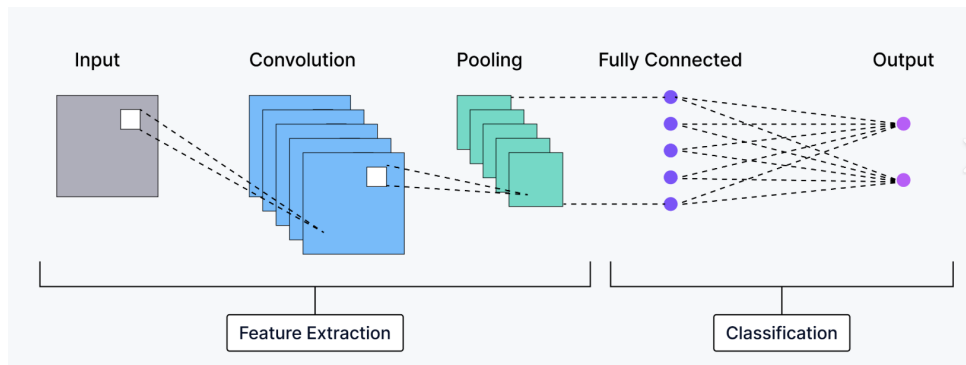
Việc chuyển đổi tự động ảnh thật sang anime đặt ra ba thách thức chính:

1. **Cân bằng nội dung và phong cách:** Mô hình phải đồng thời bảo toàn cấu trúc ngữ nghĩa của ảnh gốc (bố cục, hình dáng đối tượng, danh tính khuôn mặt) và chuyển đổi phong cách thị giác sang anime. Hai mục tiêu này thường mâu thuẫn: biến đổi phong cách quá mạnh sẽ phá hủy nội dung, trong khi bảo toàn quá chặt sẽ không đạt được phong cách anime đích [15, 30].
2. **Dữ liệu không ghép cặp:** Không thể thu thập bộ dữ liệu gồm các cặp (ảnh thật, ảnh anime tương ứng) vì mỗi ảnh anime được vẽ theo phong cách chủ quan của họa sĩ. Do đó, mô hình phải học ánh xạ giữa hai miền từ dữ liệu unpaired [30, 28].

3. **Bảo toàn danh tính khuôn mặt:** Khi xử lý ảnh chân dung, các đặc điểm nhận dạng (ánh mắt, nụ cười, tỉ lệ khuôn mặt) cần được giữ nguyên trong phiên bản anime. Nhiều mô hình hiện tại gặp khó khăn với khuôn mặt có tỉ lệ nhỏ trong ảnh hoặc bị biến dạng chi tiết khuôn mặt [47, 53].

1.2. Nền tảng Mạng Nơ-ron Tích chập

Mạng Nơ-ron Tích chập (Convolutional Neural Networks – CNNs) là kiến trúc nền tảng trong thị giác máy tính, có khả năng tự động học các biểu diễn đặc trưng phân cấp từ dữ liệu ảnh thô [41]. Mục này tóm tắt các thành phần CNN thiết yếu cho việc hiểu kiến trúc Generator và Discriminator trong các mô hình GAN ở các chương sau.



Hình 1.1. Kiến trúc cơ bản của Mạng Nơ-ron Tích chập

1.2.1. Lớp Tích chập, Pooling và Hàm kích hoạt

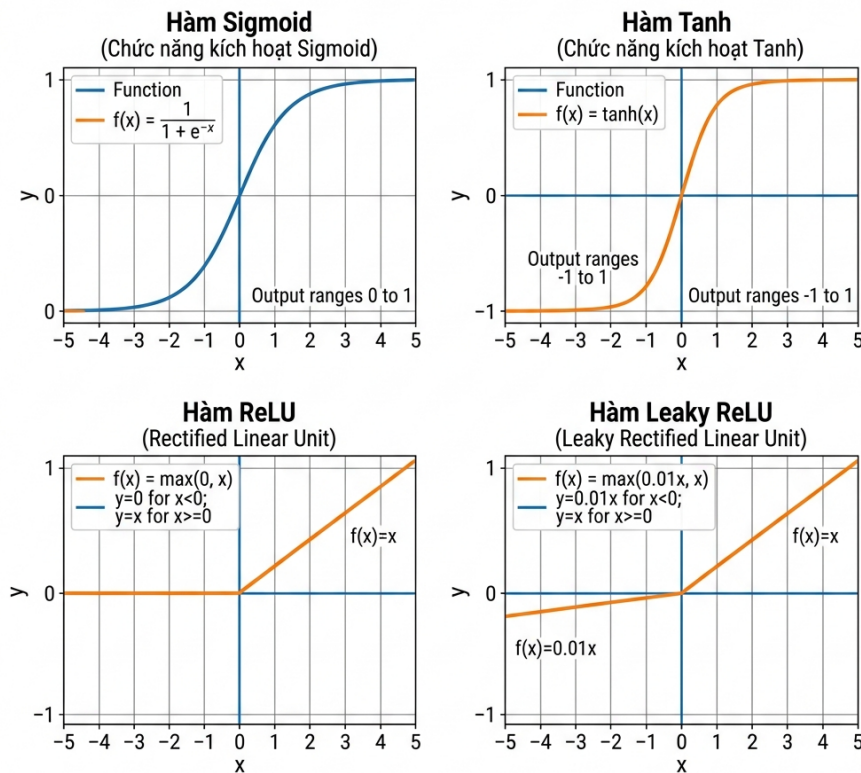
Lớp tích chập sử dụng các kernel nhỏ (thường 3×3) trượt trên ảnh đầu vào, thực hiện phép nhân tích chập cục bộ để trích xuất các đặc trưng hình học như cạnh, đường cong và kết cấu. Mỗi kernel được học tự động trong quá trình huấn luyện, thay thế hoàn toàn các đặc trưng thủ công (SIFT, HOG) của thị giác máy tính truyền thống [41, 10].

Lớp Pooling (thường là Max Pooling) giảm độ phân giải không gian của bản đồ đặc trưng, giúp giảm chi phí tính toán và tạo tính bất biến dịch chuyển cục bộ. Tuy nhiên, pooling gây mất thông tin vị trí chính xác – một hạn chế cần được xử lý trong các tác vụ yêu cầu độ chính xác pixel như sinh ảnh [41].

Hàm kích hoạt đưa tính phi tuyến vào mạng. Trong các kiến trúc GAN hiện đại, hai hàm được sử dụng phổ biến nhất là [6, 9]:

- **ReLU** ($\max(0, x)$): Tiêu chuẩn cho hầu hết CNN, giảm vanishing gradient nhưng có thể gây “Dying ReLU” khi nơ-ron nhận giá trị âm liên tục.
- **Leaky ReLU** ($\max(\alpha x, x)$ với $\alpha = 0.01$): Khắc phục Dying ReLU bằng cách giữ gradient nhỏ cho giá trị âm. Được sử dụng rộng rãi trong Generator và Discriminator của AnimeGANv3 [53].

CÁC CHỨC NĂNG KÍCH HOẠT MẠNG NEURAL PHỔ BIẾN

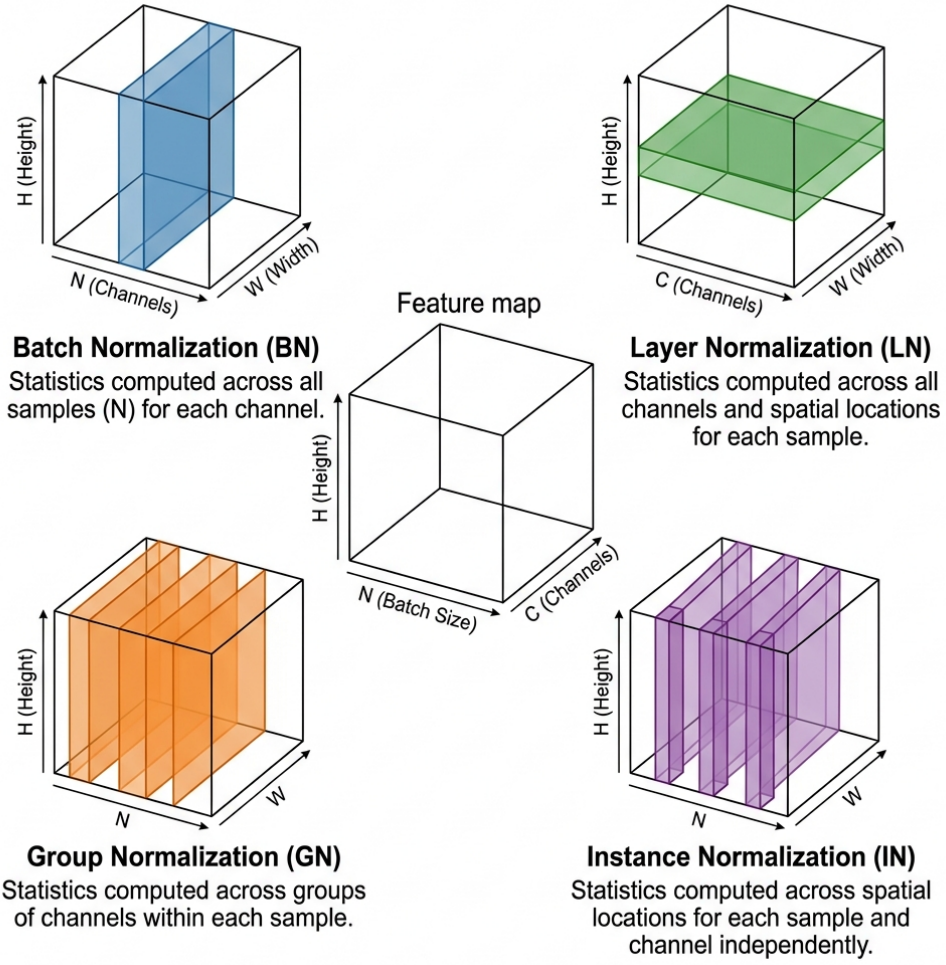


Hình 1.2. So sánh các hàm kích hoạt: Sigmoid, Tanh, ReLU và Leaky ReLU

1.2.2. Các Kỹ thuật Chuẩn hóa

Chuẩn hóa (Normalization) là cơ chế thiết yếu để ổn định và tăng tốc quá trình huấn luyện mạng sâu. Bốn kỹ thuật chính được phân biệt theo chiều tính toán thống kê trên tensor đặc trưng ($N \times C \times H \times W$):

- **Batch Normalization (BN)** [12]: Chuẩn hóa theo chiều batch N , giảm Internal Covariate Shift. Nhạy cảm với kích thước batch nhỏ.
- **Layer Normalization (LN)** [14]: Chuẩn hóa toàn bộ kênh C của một mẫu. Phổ biến trong Transformers và RNNs.
- **Group Normalization (GN)** [35]: Chia kênh thành nhóm nhỏ, cân bằng giữa BN và LN. Không phụ thuộc batch size.
- **Instance Normalization (IN)** [19]: Chuẩn hóa từng kênh của từng mẫu độc lập. **Đặc biệt quan trọng** cho bài toán chuyển phong cách vì IN loại bỏ thông tin phong cách toàn cục, cho phép mô hình học phong cách mới. IN là nền tảng trực tiếp của kỹ thuật LADE trong AnimeGANv3 (trình bày ở Chương 2).



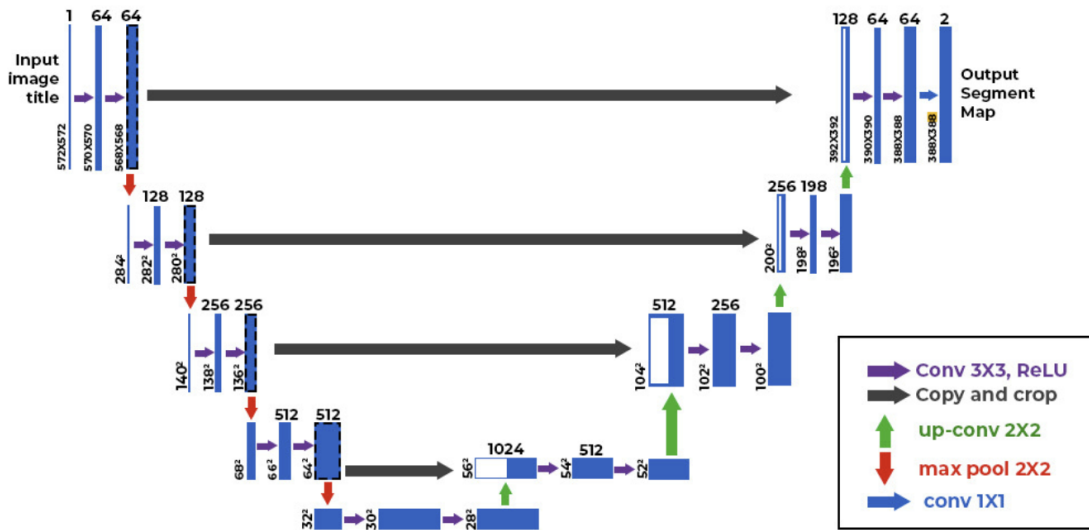
Hình 1.3. So sánh bốn kỹ thuật chuẩn hóa trên tensor ($N \times C \times H \times W$)

Chú thích: BN chuẩn hóa theo chiều batch N ; LN theo toàn bộ kênh C của một mẫu; GN theo nhóm kênh; IN theo từng kênh của từng mẫu độc lập [35, 19]

Ngoài ra, **Spectral Normalization (SN)** [34] chuẩn hóa ma trận trọng số W bằng cách chia cho giá trị kỳ dị lớn nhất $\sigma(W)$, đảm bảo ràng buộc Lipschitz cho Discriminator. SN đã trở thành tiêu chuẩn trong hầu hết kiến trúc GAN hiện đại [40, 32].

1.2.3. Kiến trúc Encoder-Decoder

Kiến trúc Encoder-Decoder là nền tảng cho các bài toán dịch ảnh (image-to-image translation). Phần **Encoder** sử dụng tích chập và downsampling để nén ảnh gốc thành vector đặc trưng ẩn, trong khi phần **Decoder** tái tạo vector ẩn thành ảnh đầu ra mong muốn [37].



Hình 1.4. Kiến trúc U-Net với skip connections

Biến thể U-Net bổ sung **skip connections** kết nối trực tiếp giữa các tầng encoder và decoder cùng độ phân giải. Cơ chế này cho phép decoder truy cập đặc trưng chi tiết cục bộ từ encoder, khắc phục hiện tượng mất thông tin không gian qua bottleneck [37]. Generator trong AnimeGANv3 sử dụng kiến trúc U-Net cho cả hai nhánh decoder (Chương 2).

Trong phần decoder, việc tăng độ phân giải (upsampling) có hai lựa chọn chính:

- **Transposed Convolution:** Lớp học được, nhưng dễ gây tạo phẩm checkerboard do chồng lớp không đều [37].
- **Bilinear/Nearest-Neighbor Interpolation + Conv:** Không học được phần upsampling, nhưng loại bỏ tạo phẩm checkerboard. AnimeGANv3 chọn phương pháp này để đảm bảo ảnh đầu ra mịn [53].

1.3. Kiến trúc Mạng Đối nghịch Tạo sinh

Mạng Đối nghịch Tạo sinh (Generative Adversarial Networks – GANs) do Ian Goodfellow và cộng sự đề xuất năm 2014 [11] là nền tảng cho hầu hết các phương pháp chuyển đổi phong cách ảnh hiện đại. Khác với các mô hình tạo sinh trước đây dựa trên phương pháp xác suất phức tạp, GAN sử dụng trò chơi đối kháng giữa hai mạng nơ-ron để tạo ra dữ liệu có độ trung thực cao.

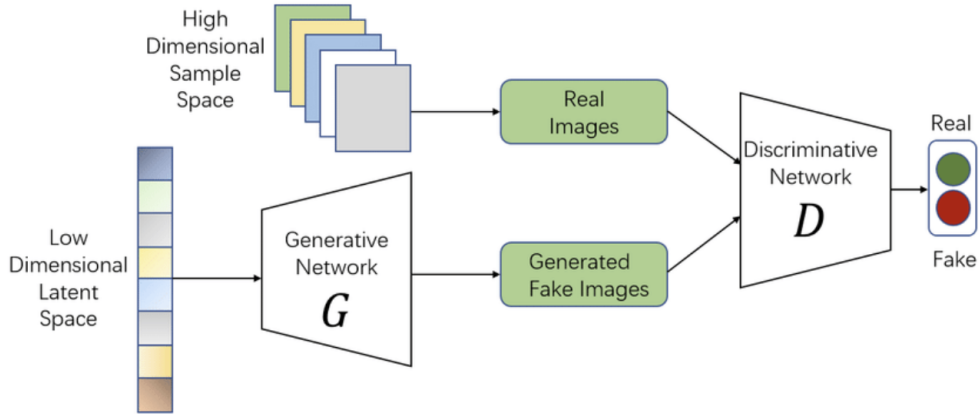
1.3.1. Nguyên lý Hoạt động và Hàm Mục tiêu

1.3.1.1. Khung Làm việc Đối kháng

GAN gồm hai mạng nơ-ron được huấn luyện đồng thời qua trò chơi có tổng bằng không [11]:

- **Generator (G):** Nhận vector nhiễu ngẫu nhiên $z \sim p_z(z)$ từ không gian tiềm ẩn và tạo ra mẫu giả $G(z)$. Mục tiêu: đánh lừa Discriminator.

- **Discriminator (D):** Bộ phân loại nhị phân, nhận dữ liệu x và xuất xác suất $D(x)$ dữ liệu đó là thật. Mục tiêu: phân biệt dữ liệu thật và giả.



Hình 1.5. Kiến trúc tổng quát của Mạng Đối nghịch Tạo sinh (GAN)

Chú thích: Generator G nhận vector nhiễu $z \sim p_z(z)$ để tạo mẫu giả $G(z)$, Discriminator D phân biệt dữ liệu thật x với dữ liệu giả. Hai mạng huấn luyện đồng thời theo trò chơi minimax [11]

1.3.1.2. Trò chơi Minimax

Quá trình huấn luyện được mô hình hóa dưới dạng bài toán tối ưu Minimax [11]:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad (1.1)$$

Số hạng đầu khuyến khích D gán xác suất cao cho dữ liệu thật, số hạng thứ hai khuyến khích D gán xác suất thấp cho dữ liệu giả $G(z)$.

1.3.1.3. Điểm Cân bằng Nash và Discriminator Tối ưu

Mục tiêu cuối cùng là đạt Điểm cân bằng Nash, nơi D không thể phân biệt ảnh thật và giả ($D(x) = 0.5$ cho mọi x), và phân phối sinh p_g trùng khớp với p_{data} [49]. Goodfellow et al. [11] chứng minh Discriminator tối ưu có dạng:

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} \quad (1.2)$$

Khi thay D^* vào hàm mục tiêu, bài toán tối thiểu hóa của Generator trở thành tối thiểu hóa khoảng cách Jensen-Shannon giữa hai phân phối:

$$C(G) = -\log(4) + 2 \cdot D_{\text{JS}}(p_{\text{data}} \parallel p_g) \quad (1.3)$$

1.3.2. Thách thức trong Huấn luyện GAN

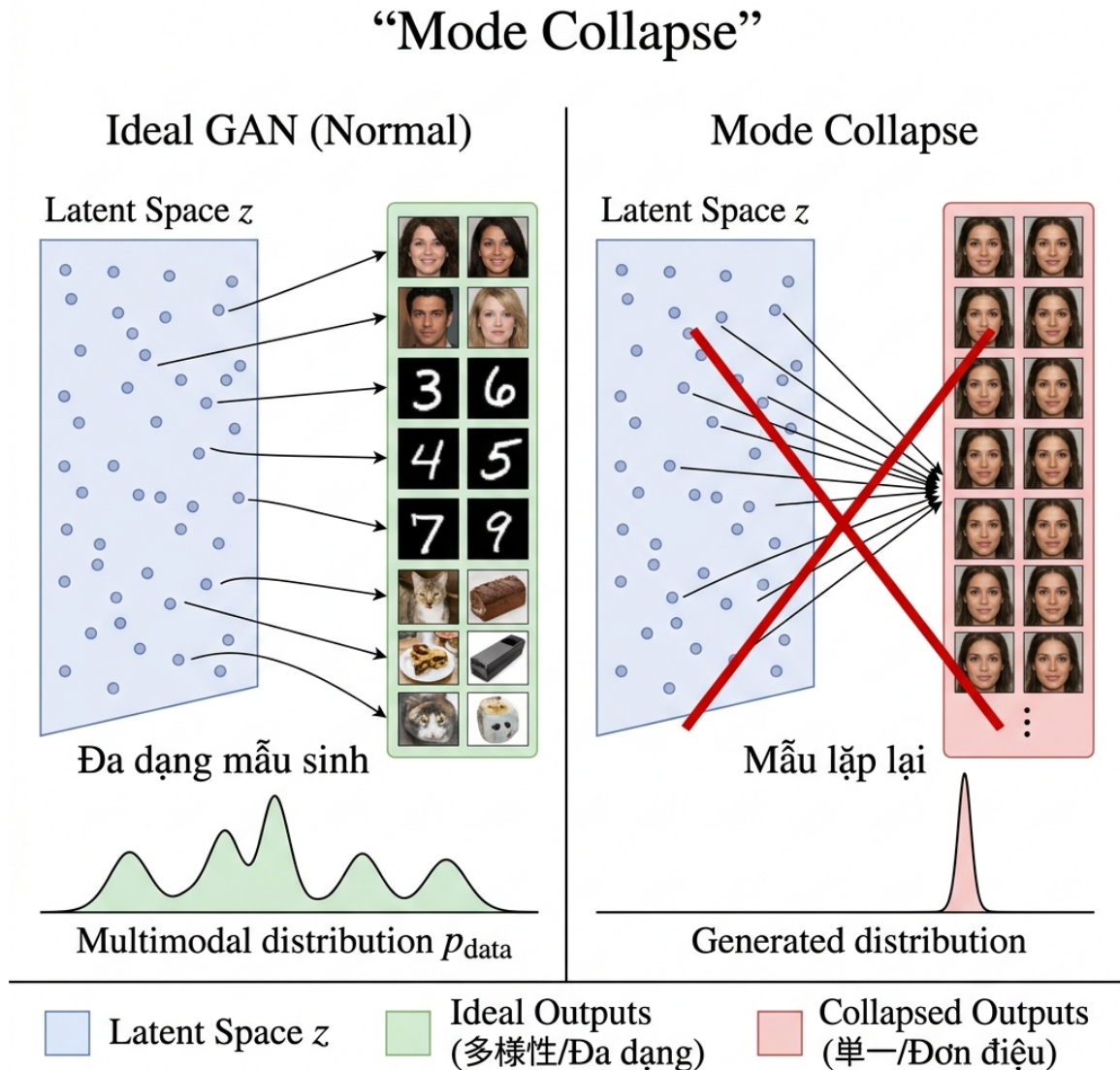
Mặc dù lý thuyết chặt chẽ, GAN trong thực tế gặp ba vấn đề nghiêm trọng [18, 23]:

1.3.2.1. Biến mất Gradient (Vanishing Gradients)

Khi p_{data} và p_g nằm trong các không gian con không giao nhau (rất phổ biến với dữ liệu ảnh nhiều chiều), khoảng cách JS là hằng số $\log 2$, dẫn đến gradient bằng 0. Khi Discriminator quá mạnh, Generator “không biết” hướng cải thiện [11].

1.3.2.2. Suy đổ Mode (Mode Collapse)

Generator học cách tạo ra một hoặc vài mẫu “an toàn” thay vì phân phối đa dạng. Hai mạng đuổi bắt nhau quanh các mode mà không bao giờ hội tụ [18].



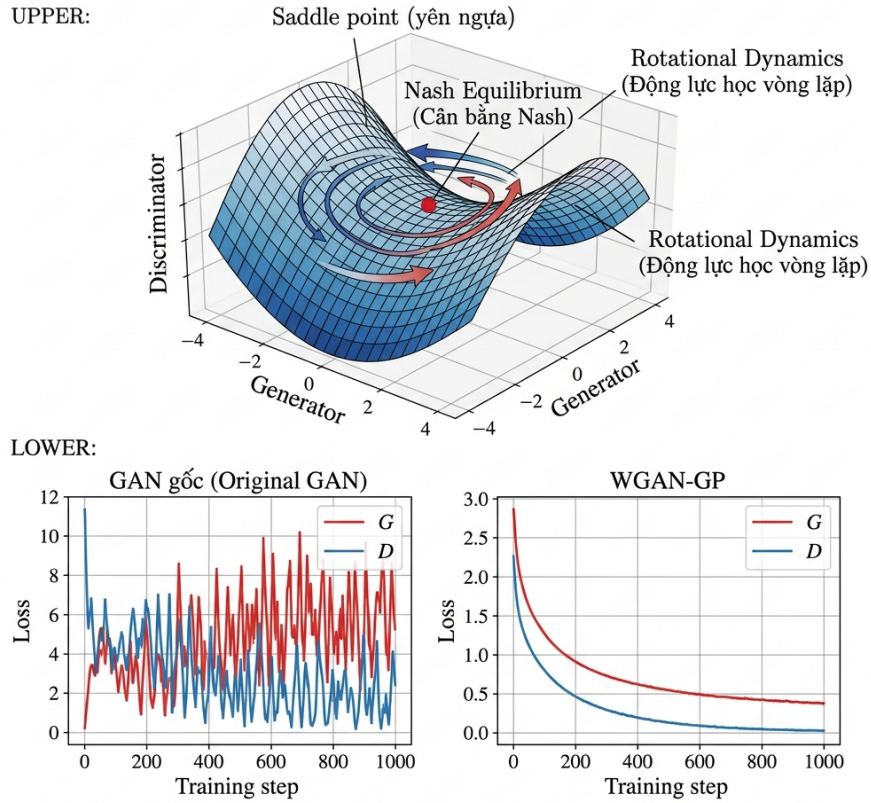
Hình 1.6. Minh họa hiện tượng Mode Collapse trong GAN

Chú thích: (Trái) GAN lý tưởng: không gian z đa dạng ánh xạ sang nhiều mode. (Phải) Mode Collapse: mọi z bị ánh xạ vào một mode duy nhất [18]

1.3.2.3. Không Hội tụ (Non-Convergence)

Gradient descent được thiết kế để tìm cực tiểu cục bộ, không phải điểm yên ngựa cần thiết cho GAN. Động lực học thường dẫn đến quỹ đạo xoay tròn thay vì hội tụ [23].

GAN Training Challenges — Nash Equilibrium and Non-Convergence



Hình 1.7. Động lực học huấn luyện GAN và vấn đề Không Hội tụ

Bảng 1.1. Tóm tắt các chế độ thất bại của GAN

Vấn đề	Biểu hiện	Nguyên nhân Toán học
Vanishing Gradients	G ngừng học, loss không đổi	D quá mạnh, JS divergence bảo toàn
Mode Collapse	Ảnh sinh ra giống hệt nhau	G tối ưu hóa cục bộ, thiếu ràng buộc đa dạng
Non-Convergence	Loss dao động mạnh	Quỹ đạo xoay quanh điểm cân bằng Nash

1.3.3. Các Giải pháp về Hàm Mục tiêu

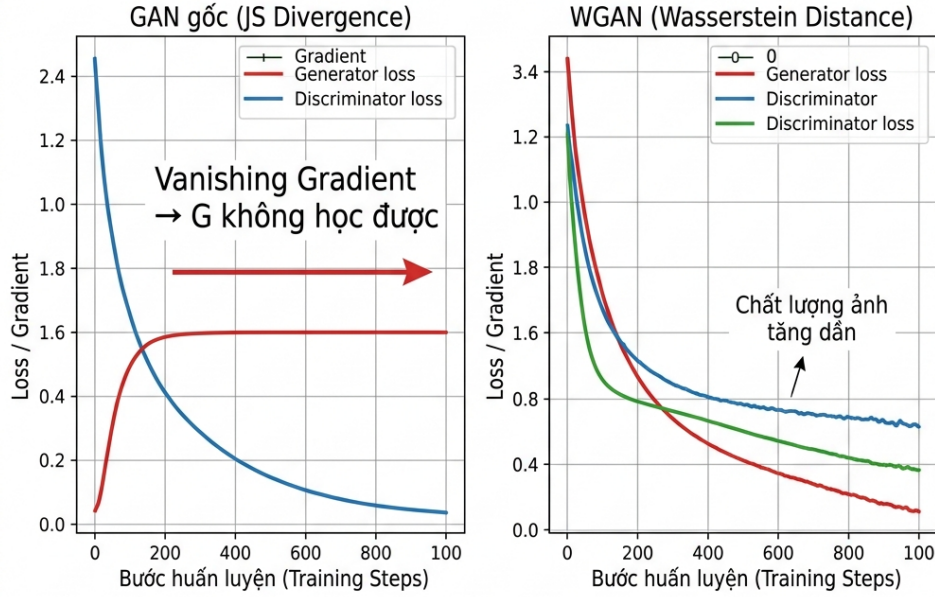
1.3.3.1. Wasserstein GAN (WGAN)

Arjovsky et al. [21] thay thế khoảng cách JS bằng khoảng cách Wasserstein (Earth-Mover distance):

$$W(p_{\text{data}}, p_g) = \inf_{\gamma \in \Pi(p_{\text{data}}, p_g)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|] \quad (1.4)$$

Khoảng cách Wasserstein liên tục và khả vi hầu khắp mọi nơi, cung cấp gradient

có ý nghĩa cho Generator ngay cả khi hai phân phối không giao nhau. Giá trị loss tương quan tốt với chất lượng ảnh [21].



Hình 1.8. So sánh gradient giữa GAN gốc (JS) và WGAN (Wasserstein)

1.3.3.2. WGAN-GP: Gradient Penalty

Gulrajani et al. [22] cải tiến WGAN bằng Gradient Penalty thay cho Weight Clipping:

$$L_{\text{WGAN-GP}} = \underbrace{\mathbb{E}_{\tilde{x} \sim p_g}[D(\tilde{x})] - \mathbb{E}_{x \sim p_{\text{data}}}[D(x)]}_{\text{WGAN Loss}} + \underbrace{\lambda \mathbb{E}_{\hat{x} \sim p_{\hat{x}}} \left[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2 \right]}_{\text{Gradient Penalty}} \quad (1.5)$$

Kỹ thuật này ổn định đáng kể quá trình huấn luyện các kiến trúc phức tạp như ResNet [22].

1.3.3.3. Spectral Normalization (SN-GAN)

Miyato et al. [34] chuẩn hóa trọng số W bằng spectral norm: $W_{\text{SN}} = W/\sigma(W)$, đảm bảo Lipschitz toàn cục với chi phí tính toán thấp. SN được sử dụng trong Discriminator của AnimeGANv3 (Mục 2.4).

1.3.3.4. Least Squares GAN (LSGAN)

Mao et al. [29] thay cross-entropy bằng bình phương sai số, tránh vanishing gradient khi D hội tụ:

$$\mathcal{L}_{adv}^G = \mathbb{E}[(D(\hat{y}) - 1)^2] \quad (1.6)$$

LSGAN được sử dụng trực tiếp trong AnimeGANv3 cho cả hai nhánh Discriminator [53].

1.3.4. Đánh giá Chất lượng GAN

1.3.4.1. Inception Score (IS)

IS [18] đánh giá độ sắc nét (entropy có điều kiện thấp) và đa dạng (entropy biên cao) qua mạng Inception v3:

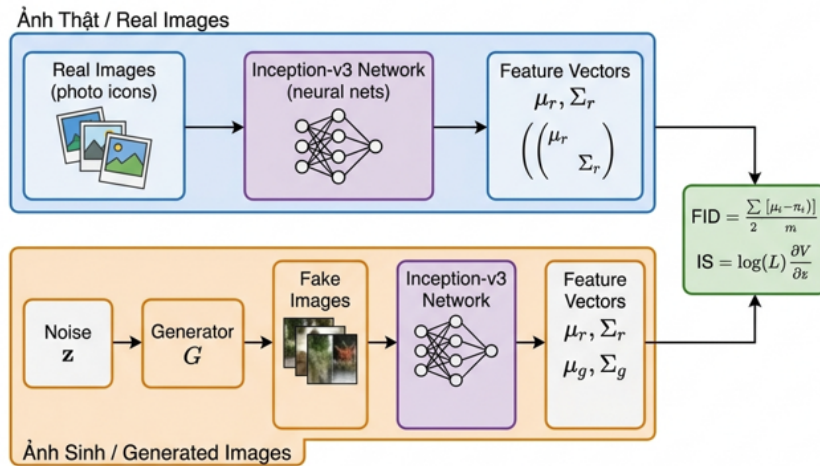
$$IS = \exp(\mathbb{E}_{x \sim p_g} [D_{KL}(p(y|x) \parallel p(y))]) \quad (1.7)$$

1.3.4.2. Fréchet Inception Distance (FID)

FID [23] so sánh phân phối đặc trưng Inception v3 giữa ảnh thật và giả, giả định phân phối chuẩn:

$$FID = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}) \quad (1.8)$$

FID thấp hơn tương ứng ảnh sinh ra gần phân phối thật hơn. FID nhạy cảm hơn IS với mode collapse và tương quan tốt với đánh giá con người [23]. Đây là chỉ số chính được sử dụng để đánh giá mô hình trong Chương 4.



Hình 1.9. Quy trình tính FID và IS qua mạng Inception-v3

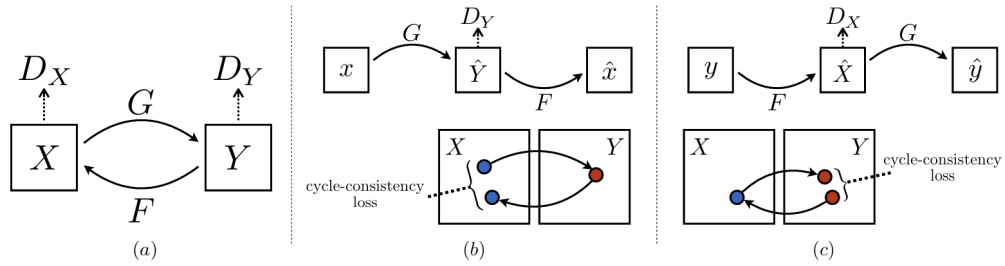
1.4. Các mô hình GAN tiêu biểu cho bài toán Anime

Bài toán chuyển ảnh sang phong cách anime đòi hỏi sự kết hợp giữa bảo toàn nội dung gốc và thể hiện đúng phong cách anime. Mục này phân tích ba hướng tiếp cận chính, từ đó định hướng cho việc lựa chọn AnimeGANv3 làm nền tảng.

1.4.1. CycleGAN và dịch ảnh không ghép cặp

Tiền đề: Isola et al. [26] đề xuất pix2pix, mô hình dịch ảnh có điều kiện sử dụng dữ liệu ghép cặp. Tuy nhiên, việc thu thập cặp ảnh thật–anime tương ứng không khả thi. Chen et al. [33] giới thiệu CartoonGAN với edge loss bảo toàn đường nét – tiền đề trực tiếp cho dòng nghiên cứu AnimeGAN.

CycleGAN (Zhu et al., 2017) [30] giải quyết bài toán dịch ảnh không ghép cặp bằng hai Generator song song $G : X \rightarrow Y$ và $F : Y \rightarrow X$, cùng ràng buộc **nhất quán vòng lặp** (cycle consistency): $x \approx F(G(x))$ và $y \approx G(F(y))$.



Hình 1.10. Kiến trúc CycleGAN với hai Generator và ràng buộc cycle consistency

Ưu điểm: Linh hoạt do không cần dữ liệu ghép cặp, bảo toàn cấu trúc tổng quát nhờ cycle consistency [30]. **Nhược điểm:** Khó bảo toàn danh tính khuôn mặt, đôi khi sinh artifact (nhiều màu, đường viền thừa) [30]. Biến thể U-GAT-IT [47] bổ sung module chú ý và Adaptive Layer-Instance Normalization để cải thiện, nhưng vẫn chưa giải quyết triệt để.

1.4.2. StyleGAN và GAN Inversion

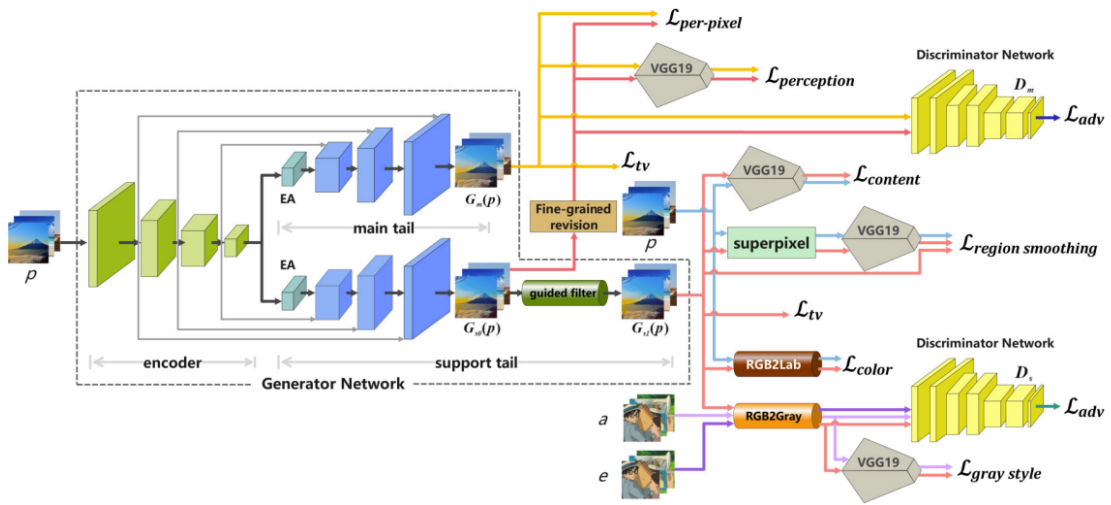
StyleGAN (Karras et al., 2019) [38] đại diện cho hướng tiếp cận tập trung vào sinh ảnh chất lượng cao. Generator gồm Mạng ánh xạ ($z \rightarrow w$) và Mạng tổng hợp sử dụng AdaIN để điều chỉnh phong cách ở từng tầng, cho phép tách biệt nội dung và phong cách [38, 46].

Kỹ thuật **GAN Inversion** [51] chiếu ảnh thật vào không gian tiềm ẩn W của StyleGAN đã huấn luyện trên ảnh anime, sau đó Generator tái tạo ảnh trong phong cách anime. Phương pháp này cho ảnh chất lượng rất cao và bảo toàn danh tính tốt [50], nhưng đòi hỏi tài nguyên tính toán lớn và quy trình inference phức tạp (hàng triệu tham số, hàng trăm bước tối ưu latent) [51].

1.4.3. AnimeGANv3 (DTGAN) – Sơ bộ kiến trúc

Loạt nghiên cứu AnimeGAN [42, 43, 53] tập trung xây dựng mô hình GAN gọn nhẹ tối ưu cho phong cách anime. Phiên bản mới nhất AnimeGANv3 đề xuất kiến trúc **Double-Tail GAN (DTGAN)** [53] với ba đặc điểm nổi bật:

- **Generator hai nhánh:** Encoder dùng chung + hai Decoder song song: Support Tail (học đường nét, cấu trúc) và Main Tail (tinh chỉnh màu sắc, chi tiết).
- **Kỹ thuật chuẩn hóa LADE:** Tự động tính tham số chuẩn hóa từ chính feature map, vượt trội Instance Normalization truyền thống.
- **Hàm mất mát chuyên biệt:** Region Smoothing Loss (vùng phẳng anime) và Fine-grained Revision Loss (tinh chỉnh chi tiết) kết hợp với Perceptual Loss.



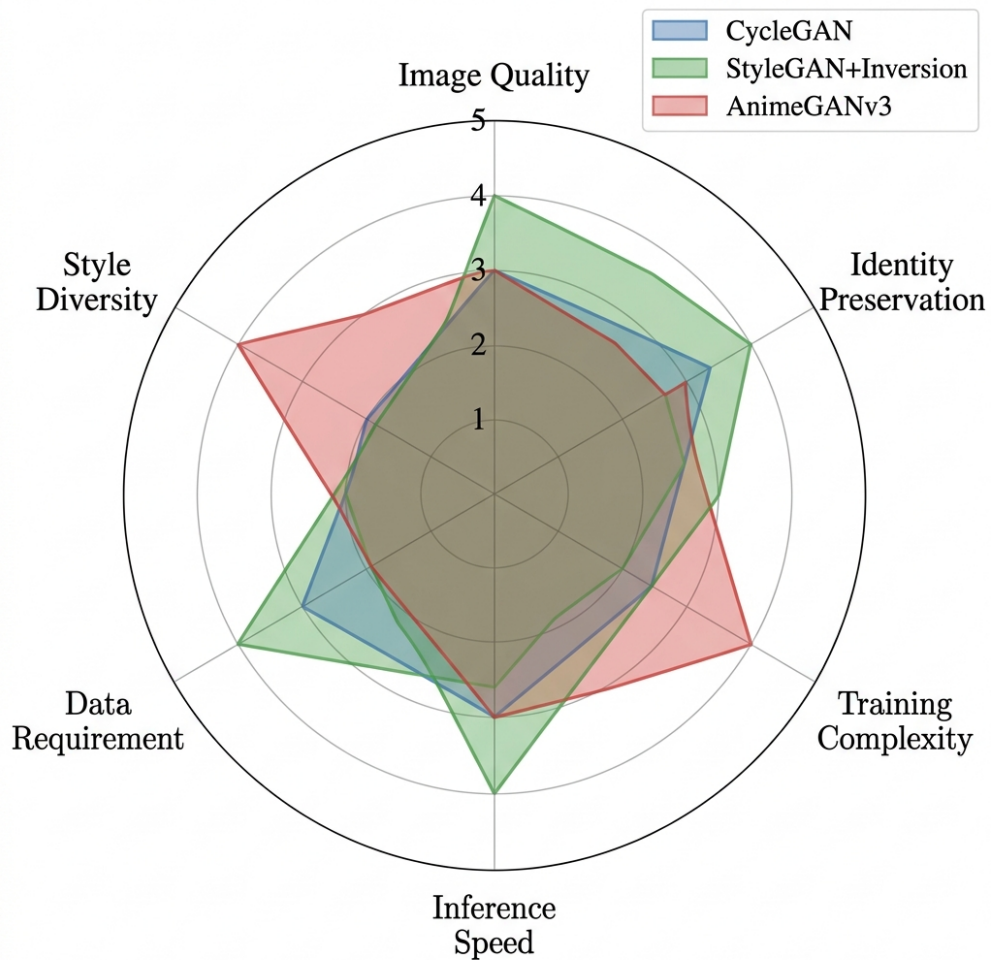
Hình 1.11. Sơ đồ tổng quát của Double-Tail GAN (DTGAN)

Chỉ với ~ 1.02 triệu tham số khi inference, AnimeGANv3 đạt tốc độ ~ 115 ms cho ảnh Full HD trên GPU [53]. Kiến trúc chi tiết của DTGAN sẽ được phân tích đầy đủ trong Chương 2.

1.4.4. So sánh tổng quan và Định hướng

Bảng 1.2. So sánh định lượng các mô hình chuyển ảnh sang anime trên tập Selfie2Anime [55]

Mô hình	FID ↓	Params (M)	Inference (ms)	Dữ liệu
CycleGAN [30]	86.7	28.3	42	Không cặp
U-GAT-IT [47]	72.1	29.8	63	Không cặp
AnimeGANv2 [43]	65.3	8.6	31	Không cặp
AnimeGANv3 [53]	58.4	1.02	12	Không cặp
StyleGAN+Enc [50]	61.8	37.1	124	Không cặp



Hình 1.12. Biểu đồ radar so sánh đa chiều ba hướng tiếp cận GAN cho bài toán anime

Chú thích: Các tiêu chí: chất lượng ảnh, bảo toàn danh tính, độ phức tạp huấn luyện, tốc độ suy luận, yêu cầu dữ liệu, đa dạng phong cách. Thang 1–5, giá trị cao tốt hơn [30, 38, 53]

Qua phân tích Bảng 1.2 và Hình 1.12, **AnimeGANv3 (DTGAN)** được chọn làm nền tảng cho luận văn vì: (1) FID thấp nhất, (2) số tham số nhỏ nhất, (3) tốc độ suy luận nhanh nhất, phù hợp ứng dụng thời gian thực [53]. Tuy nhiên, AnimeGANv3 vẫn gặp hạn chế khi xử lý khuôn mặt có tỉ lệ nhỏ trong ảnh – động lực cho việc tích hợp module phát hiện khuôn mặt (Chương 3).

1.5. Phát hiện Khuôn mặt – Sơ bộ

Phát hiện khuôn mặt (face detection) là bước tiền xử lý cho phép hệ thống tự động xác định và tách khuôn mặt người trước khi áp dụng chuyển đổi phong cách. Việc phát hiện chính xác và nhanh chóng khuôn mặt trong điều kiện tự nhiên đa dạng là thách thức đã được nghiên cứu hàng thập kỷ [4].

1.5.1. Tổng quan các Phương pháp

Lĩnh vực phát hiện khuôn mặt đã trải qua nhiều thế hệ phát triển:

- **Viola-Jones (2001)** [4]: Đặc trưng Haar + AdaBoost + cấu trúc cascade. Nhanh nhưng chỉ phát hiện mặt nhìn thẳng.
- **MTCNN (2016)** [20]: Ba mạng CNN cascade (P-Net, R-Net, O-Net). Hỗ trợ 5 landmarks nhưng tốc độ hạn chế do xử lý tuần tự.
- **SSD và YOLO** [16, 17]: Detectors một giai đoạn, rất nhanh nhưng phiên bản gốc không hỗ trợ facial landmarks.
- **RetinaFace (2019)** [45]: Phát hiện đơn giai đoạn kết hợp FPN đa tỉ lệ và đa nhiệm (bbox + landmarks).

Bảng 1.3. So sánh các phương pháp phát hiện khuôn mặt

Phương pháp	Năm	Landmark	Tốc độ	Multi-scale
Viola-Jones [4]	2001	Không	Nhanh	Hạn chế
MTCNN [20]	2016	5 pts	Trung bình	Tốt
SSD [16]	2016	Không	Rất nhanh	Tốt
RetinaFace [45]	2019	5 pts	Nhanh	Rất tốt

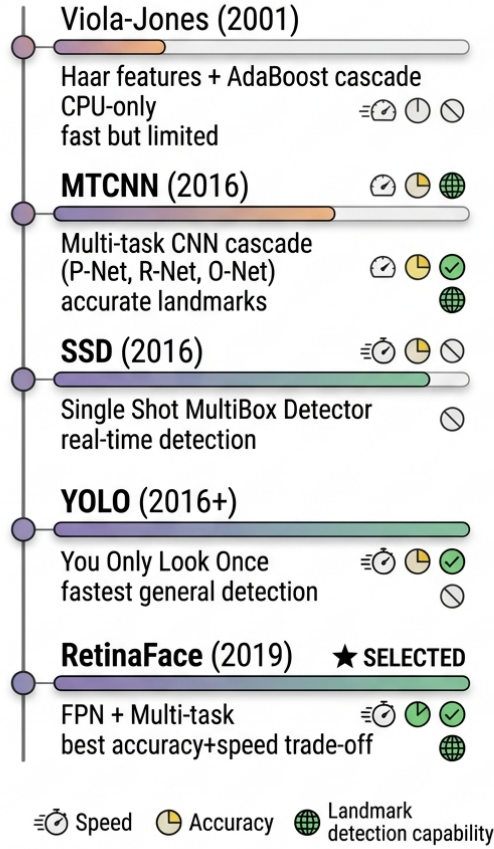
1.5.2. RetinaFace – Sơ bộ Kiến trúc

RetinaFace [45] được chọn cho luận văn nhờ ba ưu điểm: (1) phát hiện đa tỉ lệ từ khuôn mặt rất nhỏ đến rất lớn, (2) đồng thời xuất bbox và 5 facial landmarks trong một lần suy luận, (3) bản MobileNet-0.25 cực nhẹ (~500KB) phù hợp triển khai thực tế.

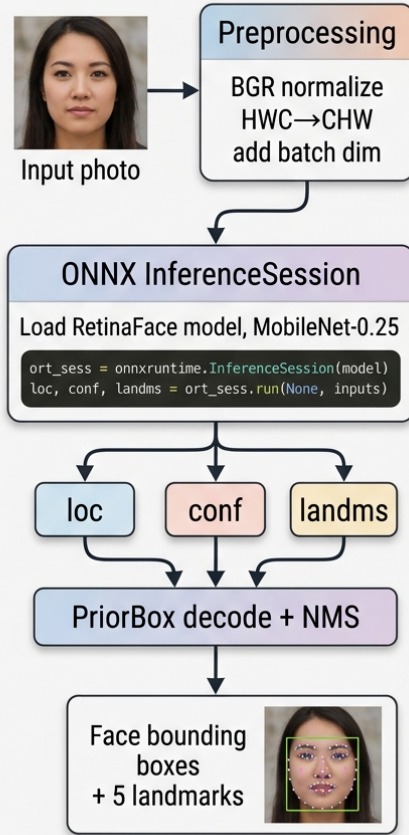
Kiến trúc RetinaFace kết hợp ba thành phần chính:

- **Backbone MobileNet-0.25** [25]: Sử dụng Depthwise Separable Convolution, giảm $8-9\times$ chi phí tính toán. Chỉ 25% số kênh so với MobileNet đầy đủ.
- **Feature Pyramid Network (FPN)** [27]: Kết hợp thông tin ngữ nghĩa (top-down) với độ phân giải cao (bottom-up), tạo ba mức đặc trưng P_3, P_4, P_5 phát hiện khuôn mặt ở mọi kích thước.
- **Multi-task Heads**: Tại mỗi mức FPN, ba đầu ra đồng thời: phân loại mặt/nền, hồi quy bounding box, và hồi quy 5 facial landmarks.

EVOLUTION OF FACE DETECTION METHODS & COMPARISON



ONNX RUNTIME INFERENCE PIPELINE FOR RETINAFACE



Hình 1.13. Tổng quan kiến trúc RetinaFace và pipeline suy luận

Chú thích: RetinaFace kết hợp backbone MobileNet-0.25, FPN đa tỉ lệ, và ba detection head đa nhiệm. Hậu xử lý bao gồm giải mã Prior Box, lọc ngưỡng tin cậy và NMS [45, 39]

Hàm mất mát đa nhiệm của RetinaFace:

$$\mathcal{L} = \mathcal{L}_{cls} + \lambda_1 \mathcal{L}_{box} + \lambda_2 \mathcal{L}_{pts} \quad (1.9)$$

trong đó $\mathcal{L}_{cls}$ là binary cross-entropy phân loại khuôn mặt/nền, $\mathcal{L}_{box}$ là smooth- ℓ_1 hồi quy bbox, và $\mathcal{L}_{pts}$ là smooth- ℓ_1 hồi quy 5 landmarks [45].

Chi tiết về quy trình suy luận RetinaFace và cách tích hợp vào pipeline anime sẽ được trình bày trong Chương 3.

Tổng kết chương

Chương 1 đã cung cấp nền tảng lý thuyết cần thiết cho luận văn:

- **Bài toán:** Chuyển ảnh sang phong cách anime là bài toán dịch ảnh không ghép cặp, đòi hỏi cân bằng giữa bảo toàn nội dung và chuyển đổi phong cách.
- **CNN:** Các thành phần cơ bản (tích chập, chuẩn hóa, encoder-decoder) tạo nền tảng kiến trúc cho Generator và Discriminator.
- **GAN:** Nguyên lý minimax, các thách thức huấn luyện (vanishing gradient, mode collapse), và giải pháp (WGAN-GP, SN, LSGAN). Chỉ số FID là thước đo chất lượng chính.
- **GAN cho Anime:** AnimeGANv3 (DTGAN) được chọn nhờ FID thấp nhất, số tham số ít nhất và tốc độ nhanh nhất trong các mô hình so sánh.
- **Face Detection:** RetinaFace MobileNet-0.25 được chọn nhờ cân bằng tốc độ, độ chính xác đa tỉ lệ và hỗ trợ facial landmarks.

Chương 2 sẽ phân tích chi tiết kiến trúc DTGAN, và Chương 3 sẽ trình bày luồng anime tích hợp phát hiện khuôn mặt.

CHƯƠNG 2

KIẾN TRÚC MÔ HÌNH ANIMEGANV3 (DTGAN)

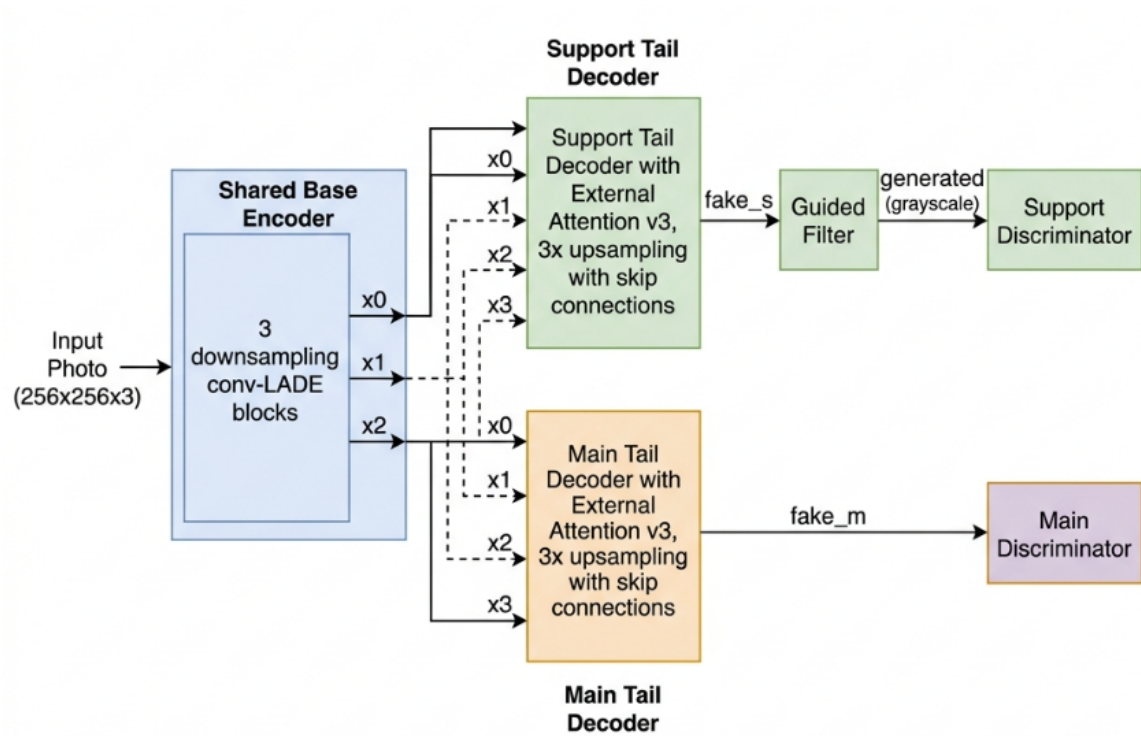
Chương này trình bày chi tiết kiến trúc kỹ thuật của mô hình AnimeGANv3, hay còn gọi là DTGAN (*Double-Tail Generative Adversarial Network*), được đề xuất bởi Liu et al. [53] vào năm 2024. Đây là chương trọng tâm kỹ thuật của luận văn, bao gồm: tổng quan kiến trúc hai nhánh, cơ chế chuẩn hóa LADE, cơ chế chú ý External Attention v3, kiến trúc Discriminator, hệ thống hàm mất mát đa thành phần, và quy trình huấn luyện theo hai giai đoạn.

2.1. Tổng quan kiến trúc DTGAN

Ý tưởng trung tâm của DTGAN là giải quyết bài toán chuyển đổi ảnh sang phong cách anime một cách hiệu quả bằng cách tách quá trình học thành **hai nhiệm vụ song song**: học các đường nét anime thô (Support Tail) và tinh chỉnh chi tiết màu sắc mịn (Main Tail). Hai nhánh decoder chia sẻ cùng một encoder đặc trưng nhưng có trọng số decoder độc lập, được huấn luyện với các discriminator và hàm mất mát riêng biệt.

Tại sao cần kiến trúc hai nhánh? Quan sát kỹ phong cách anime cho thấy hai thuộc tính thị giác cần được học riêng biệt: (1) hệ thống đường nét và cấu trúc hình khối sáng tối, và (2) phân phối màu sắc phẳng kiểu cel-shading. Nếu dùng một decoder duy nhất, hai mục tiêu này cạnh tranh nhau trong cùng một không gian tham số, dẫn đến chất lượng thỏa hiệp. DTGAN giải quyết bằng cách chia Generator thành ba thành phần [53]:

- **Base Encoder (Mạng mã hóa dùng chung):** Tiếp nhận ảnh gốc, trích xuất đặc trưng ngữ nghĩa phân cấp qua 3 lần downsampling (stride-2), thu nhỏ không gian ảnh thành bản đồ đặc trưng 32×32 . Thiết kế dùng chung (shared weights) đảm bảo cả hai nhánh decoder khởi nguồn từ cùng một không gian đặc trưng thống nhất, đồng thời giảm đáng kể số tham số.
- **Support Tail (Nhánh hỗ trợ):** Sinh ảnh anime thô $\hat{y}_s$, sau đó qua bộ lọc bảo toàn cạnh Guided Filter tạo ảnh $\hat{y}$. Ảnh này được chuyển sang grayscale trước khi đưa vào Discriminator, buộc nhánh tập trung vào đường nét và tương phản sáng tối mà không bị phân tâm bởi màu sắc.
- **Main Tail (Nhánh chính):** Sinh ảnh anime hoàn chỉnh $\hat{y}_m$ với phân phối màu sắc cel-shading mịn. Nhánh này được tối ưu dựa trên pseudo ground-truth (ảnh đã qua NL-Means + L0 Smoothing), đảm bảo vùng màu phẳng và không artifacts.



Hình 2.1. Tổng quan kiến trúc DTGAN (AnimeGANv3)

Chú thích: Ảnh đầu vào qua Shared Base Encoder để tạo các bản đồ đặc trưng x_0, x_1, x_2, x_3 , sau đó được xử lý song song bởi Support Tail và Main Tail với skip connections kiểu U-Net. Output của Support Tail qua Guided Filter trước khi đưa vào Discriminator grayscale [53]

2.2. Mạng Generator (G_net)

2.2.1. Base Encoder (Chia sẻ)

Base Encoder là thành phần dùng chung giữa hai nhánh decoder, có nhiệm vụ nén ảnh đầu vào $256 \times 256 \times 3$ thành biểu diễn đặc trưng có ngữ nghĩa cao ở độ phân giải 32×32 . Quá trình downsampling diễn ra qua 3 lần stride-2, với khối xây dựng cơ bản là conv_LADE_Lrelu — tích chập kết hợp với chuẩn hóa LADE và hàm kích hoạt Leaky ReLU.

Cấu trúc chi tiết của Base Encoder như sau:

Bảng 2.1. Cấu trúc chi tiết Base Encoder của DTGAN

Lớp	Kênh ra	Kích thước	Biến đặc trưng
Input Photo	3	256×256	—
conv_LADE_Lrelu (k=7, s=1)	32	256×256	x_0
conv_LADE_Lrelu (k=3, s=2)	32	128×128	—
conv_LADE_Lrelu (k=3, s=1)	64	128×128	x_1
conv_LADE_Lrelu (k=3, s=2)	64	64×64	—
conv_LADE_Lrelu (k=3, s=1)	128	64×64	x_2
conv_LADE_Lrelu (k=3, s=2)	128	32×32	—
conv_LADE_Lrelu (k=3, s=1)	128	32×32	x_3

Các đặc trưng trung gian x_0, x_1, x_2, x_3 được lưu lại để sử dụng làm skip connections trong cả hai decoder, theo kiến trúc U-Net [37].

2.2.2. Support Tail (Decoder nhánh hỗ trợ)

Support Tail nhận x_3 làm đầu vào, đầu tiên qua cơ chế External Attention v3 (xem Mục 2.4), sau đó thực hiện 3 lần upsampling 2× kết hợp với skip connections:

1. $x_3 \rightarrow \text{External_attention_v3} \rightarrow s_x3$
2. Upsample 2× + conv_LADE_Lrelu(128) + concat(x_2) $\rightarrow s_x4$ (64×64)
3. Upsample 2× + conv_LADE_Lrelu(64) + concat(x_1) $\rightarrow s_x5$ (128×128)
4. Upsample 2× + conv_LADE_Lrelu(32) + concat(x_0) $\rightarrow s_x6$ (256×256)
5. Conv2D(3, k=7, s=1) $\rightarrow \tanh \rightarrow \hat{y}_s$ (fake_s)

Output $\hat{y}_s$ tiếp tục qua bộ lọc **Guided Filter** để tạo $\hat{y}$ (generated), được chuyển sang không gian grayscale trước khi đưa vào Discriminator Support. Bilinear upsampling được dùng thay vì transposed convolution để tránh tạo phẩm checkerboard [37].

2.2.3. Main Tail (Decoder nhánh chính)

Main Tail có cấu trúc đối xứng hoàn toàn với Support Tail, cũng thực hiện 3 lần upsampling với skip connections từ x_0, x_1, x_2 . Điểm khác biệt quan trọng là:

- **Trọng số độc lập:** Main Tail có bộ tham số riêng, không chia sẻ với Support Tail.
- **External Attention riêng:** Sử dụng module External Attention v3 với memory bank độc lập.
- **Pseudo ground-truth:** Được huấn luyện để tái tạo ảnh đã qua NL-Means + L0 Smoothing — một biểu diễn anime “sạch” không có nhiều texture.

Output của Main Tail là $\hat{y}_m$ (fake_m), được đưa trực tiếp vào Discriminator Main mà không qua Guided Filter.

Bảng 2.2. So sánh Support Tail và Main Tail trong DTGAN

Tiêu chí	Support Tail	Main Tail
Đầu ra	$\hat{y}_s$ (fake_s)	$\hat{y}_m$ (fake_m)
Hậu xử lý	Guided Filter $\rightarrow \hat{y}$	Không
Discriminator	Grayscale D	Full-color D_main
Ground-truth kiểu	Ảnh anime gốc (grayscale)	NL-Means + L0 Smoothing
Trọng số	Riêng	Riêng
Vai trò	Học đường nét, cấu trúc	Tinh chỉnh màu, chi tiết mịn

2.3. Kỹ thuật chuẩn hóa LADE

2.3.1. Động lực và ý tưởng

Trong các mạng chuyển đổi phong cách ảnh, việc lựa chọn kỹ thuật chuẩn hóa ảnh hưởng trực tiếp đến khả năng học phong cách và bảo toàn nội dung. Các kỹ thuật trước đây đều có hạn chế: Batch Normalization [12] phụ thuộc vào kích thước batch; Instance Normalization [19] sử dụng tham số γ, β cố định, mất thông tin toàn cục; AdaIN cần ảnh phong cách riêng biệt.

LADE (*Linearly Adaptive Denormalization*) [53] giải quyết vấn đề này bằng cách **tự động học tham số chuẩn hóa từ chính feature map đầu vào** thông qua một tích chập 1×1 .

2.3.2. Công thức toán học của LADE

Cho feature map đầu vào $x \in \mathbb{R}^{B \times H \times W \times C}$, quá trình LADE diễn ra theo ba bước:

Bước 1 — Chiếu tuyến tính (Projection):

$$t_x = \text{Conv}_{1 \times 1}(x) \quad (2.1)$$

trong đó $t_x \in \mathbb{R}^{B \times H \times W \times C}$ là biểu diễn trung gian.

Bước 2 — Tính thống kê thích ứng:

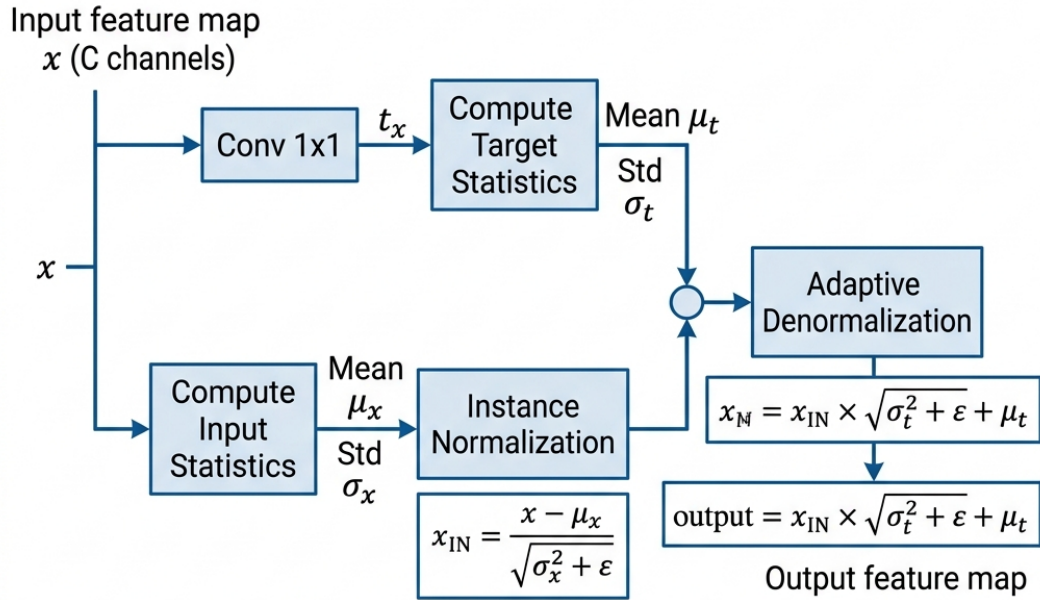
$$(\mu_t, \sigma_t^2) = \text{moments}(t_x, \text{axes} = [H, W]), \quad (\mu_x, \sigma_x^2) = \text{moments}(x, \text{axes} = [H, W]) \quad (2.2)$$

Bước 3 — Instance Normalization và Adaptive Denormalization:

$$x_{\text{IN}} = \frac{x - \mu_x}{\sqrt{\sigma_x^2 + \varepsilon}} \quad (2.3)$$

$$\text{LADE}(x) = x_{\text{IN}} \cdot \sqrt{\sigma_t^2 + \varepsilon} + \mu_t \quad (2.4)$$

trong đó ε là hằng số nhỏ tránh chia cho 0.



Normalization Type	Parameter Source
Batch Norm	Learned via Backpropagation over Batch
Instance Norm	Computed from each instance's spatial dimensions
AdaIN	Computed from an external style image
LADE	Linearly derived from the content feature map

Hình 2.2. Cơ chế hoạt động của LADE

Chú thích: feature map x được chiếu qua Conv 1×1 để tạo thống kê thích ứng (μ_t, σ_t) , sau đó bình thường hóa Instance rồi tái tỷ lệ hóa thích ứng. Bảng so sánh cho thấy LADE tự học tham số từ chính dữ liệu, không cần ảnh phong cách riêng như AdaIN [53]

2.3.3. So sánh LADE với các kỹ thuật chuẩn hóa khác

Bảng 2.3. So sánh các kỹ thuật chuẩn hóa trong mạng chuyển đổi phong cách ảnh

Kỹ thuật	Nguồn tham số	Ưu điểm	Hạn chế
Batch Norm [12]	Thống kê batch	Ổn định huấn luyện	Phụ thuộc batch size
Instance Norm [19]	γ, β cố định	Tốt cho style transfer	Mất thông tin toàn cục
AdaIN	Từ ảnh phong cách riêng	Linh hoạt phong cách	Cần ảnh style riêng biệt
LADE [53]	Conv 1×1 trên chính x	Tự thích ứng, không cần style riêng	Tăng C^2 tham số nhỏ

Sự khác biệt mấu chốt của LADE so với AdaIN là: *tham số tái tỷ lệ hóa được suy ra từ chính feature map hiện tại*, không phải từ một ảnh phong cách bên ngoài. Nhờ đó, mô hình có thể học phong cách nội tại của không gian đặc trưng một cách linh hoạt trong từng bước huấn luyện.

2.3.4. Biến thể LADE_D trong Discriminator

Trong các lớp Discriminator, LADE được mở rộng thành LADE_D: tích chập 1×1 trong bước chiếu tuyến tính được thay thế bằng `spectral_norm(Conv  $1 \times 1$ )` — tức là áp dụng thêm Spectral Normalization [34] để đảm bảo ràng buộc Lipschitz của Discriminator.

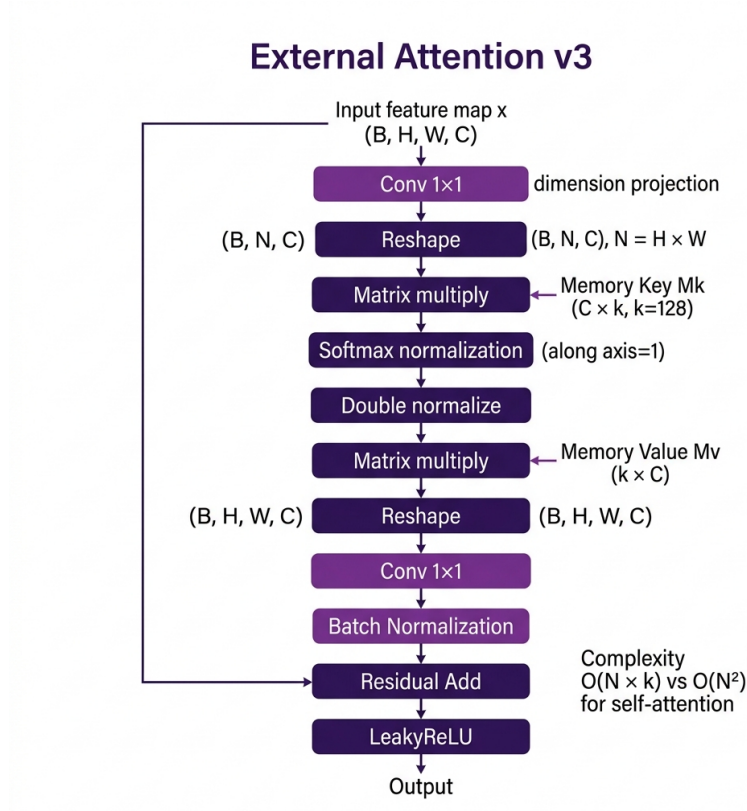
2.4. Cơ chế External Attention v3

2.4.1. Hạn chế của Self-Attention và Giải pháp

Cơ chế Self-Attention [40] đã được chứng minh hiệu quả trong việc nắm bắt phụ thuộc tầm xa (long-range dependency) trong ảnh. Tuy nhiên, độ phức tạp tính toán $O(N^2)$ theo số lượng pixel $N = H \times W$ trong Self-Attention trở thành nút thắt nghiêm trọng ở độ phân giải trung bình và cao.

External Attention, được đề xuất bởi Guo et al. [48], giải quyết vấn đề này bằng cách thay thế cặp key-value nội tại bằng hai **bộ nhớ học được** (learnable memory bank) M_k và M_v , giảm độ phức tạp xuống còn $O(N \times k)$ với $k \ll N$.

2.4.2. Kiến trúc External Attention v3



Hình 2.3. Kiến trúc External Attention v3 sử dụng trong DTGAN tại vị trí bottleneck 32×32

Chú thích: Hai memory bank học được $M_k \in \mathbb{R}^{C \times k}$ và $M_v \in \mathbb{R}^{k \times C}$ cho phép mô hình học phụ thuộc toàn cục với độ phức tạp $O(N \cdot k)$ thay vì $O(N^2)$ của Self-Attention [48, 53].

Cụ thể, External Attention v3 trong DTGAN hoạt động theo các bước sau (áp dụng tại bottleneck 32×32 , $k = 128$):

1. $x \in \mathbb{R}^{B \times H \times W \times C} \xrightarrow{\text{Conv}_{1 \times 1}}$ reshape thành $\mathcal{F} \in \mathbb{R}^{B \times N \times C}$, $N = H \times W$
2. **Attention weights:** $A = \text{Softmax}_{\text{axis}=1}(\mathcal{F} \cdot M_k) \in \mathbb{R}^{B \times N \times k}$
3. **Double normalization:** $A_{ij} \leftarrow A_{ij} / \sum_j A_{ij}$ (chuẩn hóa cột thứ hai)
4. $\mathcal{F}' = A \cdot M_v \in \mathbb{R}^{B \times N \times C}$, reshape về $\mathbb{R}^{B \times H \times W \times C}$
5. $\text{out} = \text{LeakyReLU}(x + \text{BN}(\text{Conv}_{1 \times 1}(\mathcal{F}')))$

Công thức tổng quát:

$$\text{ExternalAttention}(x) = \text{LeakyReLU}(x + \text{BN}(\text{normalize}(\mathcal{F} M_k) M_v)) \quad (2.5)$$

Cơ chế double normalization (chuẩn hóa hai lần theo cả trục N và k) giúp tránh hiện tượng một số attention head chiếm ưu thế hoàn toàn, đảm bảo phân phối attention cân bằng hơn so với softmax đơn thuần.

2.5. Discriminator (D_net)

DTGAN sử dụng hai Discriminator độc lập theo kiến trúc PatchGAN [26]:

- **Discriminator hỗ trợ (Support D):** Mạng này hoạt động chuyên biệt trên không gian ảnh xám (grayscale) 1 kênh màu. Nhiệm vụ cốt lõi của nó là đối chiếu và phân biệt giữa các bức ảnh anime đen trắng bản gốc với ảnh $\hat{y}$ (generated) — vốn là sản phẩm thô đã được xử lý cạnh từ nhánh Support Tail. Việc cưỡng chế toàn bộ tín hiệu đầu vào sang thang độ xám giúp Discriminator loại bỏ hoàn toàn sự phân mảnh thông tin phức tạp sinh ra từ phân bố đa sắc; qua đó, tạo tín hiệu phạt ngược sắc bén, ép Support Tail phải tập trung tối đa vào việc học hỏi chính xác cấu trúc hình khối, mảng tương phản sáng tối và hệ thống đường viền thanh đậm mang phong cách anime.
- **Discriminator chính (Main D):** Trái ngược hoàn toàn, nền tảng phân tích của Main D là không gian ảnh màu (RGB) định dạng 3 kênh. Mạng này đảm nhận vai trò giám khảo tổng lực, thẩm định sự tinh tế trong kỹ thuật phối màu và tính đồng nhất mịn màng của bề mặt thị giác. Nó thực thi quá trình đối kháng giữa tập ảnh anime tiêu chuẩn (đã được tiền xử lý triệt để khâu nhiễu hạt bằng giải thuật NL-Means kết hợp L0 Smoothing) và ảnh màu hoàn chỉnh $\hat{y}_m$ đến từ nhánh Main Tail. Phép kiểm định này cực kỳ quan trọng nhằm bám sát hiệu ứng tô bóng theo vùng (cel-shading) truyền thống, đảm bảo bức tranh ra lò không vướng phải những vết nhiễu màu hay loang lổ biến dạng sinh ra trong quá trình nội suy tích chập.

Mỗi Discriminator gồm 7 lớp tích chập sử dụng LADE_D (LADE kết hợp Spectral Normalization), đầu ra là feature map 1 kênh theo phong cách PatchGAN — mỗi vị trí không gian phán xét một vùng cục bộ của ảnh thay vì toàn bộ ảnh, giúp cải thiện độ sắc nét cục bộ trong ảnh đầu ra [26].

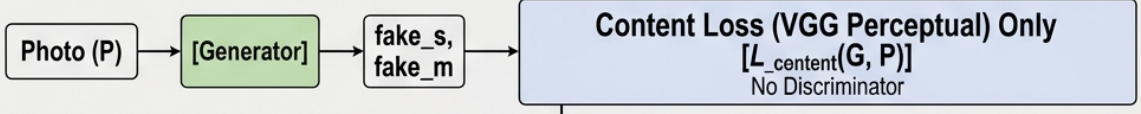
Việc sử dụng **grayscale** cho Support Discriminator là lựa chọn thiết kế có chủ ý: vì mục tiêu của Support Tail là học đường nét và cấu trúc anime (không phụ thuộc màu sắc), discriminator grayscale giúp mô hình tập trung vào hình dạng và độ tương phản thay vì bị nhiễu bởi sự phân bố màu.

2.6. Hệ thống Hàm Mất mát

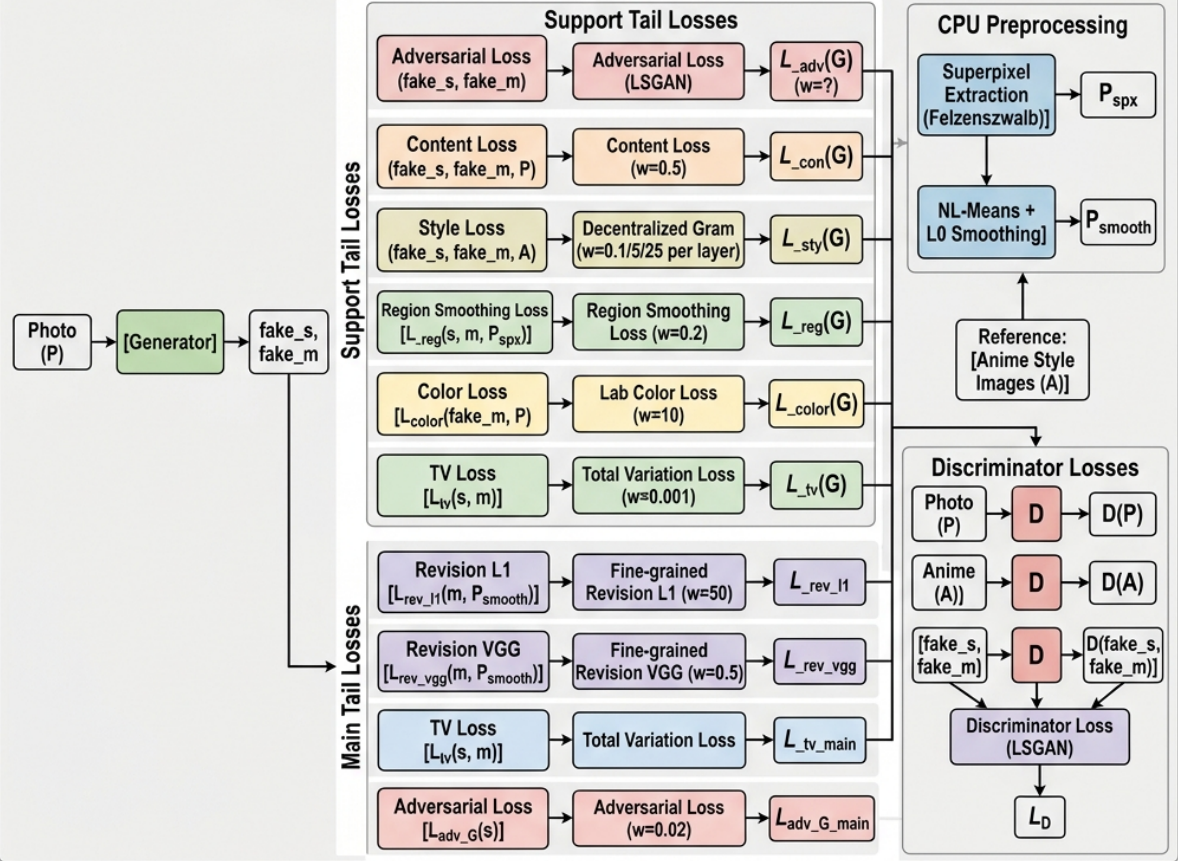
DTGAN sử dụng một hệ thống hàm mất mát đa thành phần phức tạp, phản ánh mục tiêu kép của hai nhánh decoder. Hình 2.4 minh họa toàn bộ pipeline.

Input: Real Photo (P)

Stage 1 (Pre-training, epoch < 5)



Stage 2 (Full training, epoch ≥ 5)



Hình 2.4. Pipeline huấn luyện hai giai đoạn và hệ thống hàm mất mát của DTGAN

Chú thích: Giai đoạn 1 (pre-training) chỉ dùng Content Loss. Giai đoạn 2 kết hợp nhiều thành phần loss cho Support Tail và Main Tail, với pseudo ground-truth được tạo bởi CPU preprocessing [53]

2.6.1. Hàm mất mát Giai đoạn Pre-training

Trong các epoch đầu (epoch < init_G_epoch = 5), chỉ huấn luyện Generator với Content Loss thuần túy (không có Discriminator):

$$\mathcal{L}_{\text{pre}} = \mathcal{L}_{\text{con}}(x, \hat{y}_s) + \mathcal{L}_{\text{con}}(x, \hat{y}_m) \quad (2.6)$$

Giai đoạn này giúp Generator hội tụ về biểu diễn nội dung ổn định trước khi tín hiệu adversarial được kích hoạt, tránh sụp đổ cục bộ trong giai đoạn đầu.

2.6.2. Content Loss (Perceptual Loss)

Content Loss được tính thông qua mạng VGG19 [13] tại lớp conv4_4 (512 kênh):

$$\mathcal{L}_{\text{con}}(x, \hat{y}) = \frac{1}{C_l H_l W_l} \|\phi_l(x) - \phi_l(\hat{y})\|_1 \quad (2.7)$$

trong đó $\phi_l(\cdot)$ là đặc trưng tại lớp l của VGG19. Chuẩn L_1 được ưu tiên thay vì L_2 vì ít nhạy hơn với outliers và giảm hiện tượng ảnh bị mờ [15].

2.6.3. Style Loss (Decentralized Gram Matrix)

Style Loss là thành phần giúp Generator học phân phối phong cách anime thông qua ma trận Gram. Điểm đặc biệt của DTGAN là sử dụng **Decentralized Gram Matrix** — trừ đi mean của activations trước khi tính Gram:

$$G_l^{\text{dec}}(x) = \frac{1}{C_l N_l} (\phi_l(x) - \bar{\phi}_l(x))^T (\phi_l(x) - \bar{\phi}_l(x)) \quad (2.8)$$

với $\bar{\phi}_l(x) = \frac{1}{N_l} \sum_i \phi_l(x)_i$ là mean của activation map tại lớp l .

Kỹ thuật này giảm thiểu sai lệch do độ sáng (luminance bias) khi so sánh phong cách, vì phong cách anime có phân phối brightness rất khác với ảnh chụp thật. Loss style tổng hợp từ ba lớp VGG19 (conv2_2, conv3_4, conv4_4) với trọng số tăng dần:

$$\mathcal{L}_{\text{sty}} = \lambda_{s1} \|G_2^{\text{dec}}\| + \lambda_{s2} \|G_3^{\text{dec}}\| + \lambda_{s3} \|G_4^{\text{dec}}\| \quad (2.9)$$

với $(\lambda_{s1}, \lambda_{s2}, \lambda_{s3}) = (0.1, 5.0, 25.0)$.

2.6.4. Region Smoothing Loss

Một trong những đặc trưng quan trọng của phong cách anime là **các vùng màu phẳng**, tối giản chi tiết kết cấu. Region Smoothing Loss được thiết kế để khuyến khích tính chất này bằng cách sử dụng **Superpixel Segmentation** (thuật toán Felzenszwalb [3]) để tạo phiên bản “tối giản” x_{spx} của ảnh đầu vào:

$$\mathcal{L}_{\text{rs}} = 0.2 \cdot \mathcal{L}_{\text{con}}(x_{\text{spx}}, \hat{y}) + 0.2 \cdot \mathcal{L}_{\text{con}}(x_{\text{spx}}, \hat{y}) \quad (2.10)$$

Superpixel segmentation phân chia ảnh thành các vùng đồng nhất về màu sắc và cường độ, sau đó nội suy mỗi vùng về giá trị trung bình. Generator được khuyến khích tạo ra output gần với biểu diễn phẳng này, tạo hiệu ứng “cel-shading” đặc trưng của anime.

2.6.5. Lab Color Loss

Để bảo toàn màu sắc tự nhiên từ ảnh gốc (tránh hiện tượng color drift trong quá trình chuyển đổi phong cách), DTGAN sử dụng Lab Color Loss trong không gian màu CIE Lab — một không gian màu tách biệt độ sáng (L) và thông tin màu sắc (a, b):

$$\mathcal{L}_{\text{color}} = 2 \cdot \|L(x) - L(\hat{y})\|_1 + \|a(x) - a(\hat{y})\|_1 + \|b(x) - b(\hat{y})\|_1 \quad (2.11)$$

Kênh L được đặt trọng số gấp đôi vì độ sáng ảnh hưởng mạnh hơn đến cảm nhận thị giác tổng thể so với độ bão hòa màu. Trọng số toàn cục $\lambda_c = 10.0$.

2.6.6. Total Variation Loss

Total Variation (TV) Loss thúc đẩy tính mịn không gian của ảnh đầu ra, giảm nhiễu pixel cục bộ:

$$\mathcal{L}_{\text{TV}}(\hat{y}) = \frac{1}{HW} \sum_{i,j} \left(\|\nabla_h \hat{y}_{i,j}\|^2 + \|\nabla_w \hat{y}_{i,j}\|^2 \right) \quad (2.12)$$

với $\lambda_{\text{TV}} = 0.001$.

2.6.7. Adversarial Loss (LSGAN)

DTGAN sử dụng biến thể LSGAN (Least Squares GAN) [29] thay vì cross-entropy, giúp tránh problema biến mất gradient khi Discriminator hội tụ. Loss cho Generator:

$$\mathcal{L}_{\text{adv}}^G = \mathbb{E} \left[(D(\hat{y}) - 1)^2 \right] \quad (2.13)$$

Loss cho Discriminator Support:

$$\mathcal{L}_D^s = \mathbb{E} \left[(D(y_{\text{anime}}) - 1)^2 \right] + \mathbb{E} [D(\hat{y})^2] + 2.0 \cdot \mathbb{E} [D(y_{\text{smooth}})^2] \quad (2.14)$$

trong đó y_{smooth} là ảnh anime đã làm mịn cạnh (edge-smoothed). Hệ số 2.0 nhân với smooth loss giúp tạo ra đường nét rõ ràng hơn, vì làm giả smooth ảnh sẽ bị phạt nặng hơn.

2.6.8. Fine-grained Revision Loss (Main Tail)

Thành phần loss đặc trưng nhất của Main Tail là **Fine-grained Revision Loss**, sử dụng pseudo ground-truth x_{smooth} tạo bởi NL-Means Denoising [5] kết hợp với L0 Gradient Minimization [7]:

$$\mathcal{L}_{\text{rev}} = \lambda_{r1} \cdot \|x_{\text{smooth}} - \hat{y}_m\|_1 + \lambda_{rv} \cdot \mathcal{L}_{\text{con}}(x_{\text{smooth}}, \hat{y}_m) \quad (2.15)$$

với $\lambda_{r1} = 50.0$ và $\lambda_{rv} = 0.5$. Trọng số L_1 rất lớn (50.0) buộc Main Tail học cẩn thận hơn từ pseudo ground-truth, trong khi VGG perceptual loss đảm bảo sự nhất quán đặc trưng ngữ nghĩa.

2.6.9. Tổng hợp Loss

$$\mathcal{L}_G^{\text{support}} = \mathcal{L}_{\text{adv}}^s + \lambda_c \cdot \mathcal{L}_{\text{con}} + \mathcal{L}_{\text{sty}} + \mathcal{L}_{\text{rs}} + \lambda_{\text{color}} \cdot \mathcal{L}_{\text{color}} + \lambda_{\text{TV}} \cdot \mathcal{L}_{\text{TV}} \quad (2.16)$$

$$\mathcal{L}_G^{\text{main}} = \lambda_{\text{adv}}^m \cdot \mathcal{L}_{\text{adv}}^m + \mathcal{L}_{\text{rev}} + \lambda_{\text{TV}} \cdot \mathcal{L}_{\text{TV}}(\hat{y}_m) \quad (2.17)$$

Tổng loss Generator: $\mathcal{L}_G = \mathcal{L}_G^{\text{support}} + \mathcal{L}_G^{\text{main}}$, với $\lambda_{\text{adv}}^m = 0.02$ (adversarial loss của Main Tail nhỏ hơn vì main tail chủ yếu học từ revision loss).

2.7. Xử lý Hậu kỳ: Guided Filter

Guided Filter [8] là bộ lọc ảnh có bảo toàn cạnh (edge-preserving), được áp dụng lên output của Support Tail trước khi đưa vào Discriminator:

$$\hat{y} = \tanh\left(s \cdot \text{GuidedFilter}\left(\sigma\left(\frac{\hat{y}_s}{s}\right), \sigma\left(\frac{\hat{y}_m}{s}\right), r=2, \varepsilon=0.01\right)\right) \quad (2.18)$$

trong đó $s = 2.5$ là hệ số tỷ lệ, $\sigma(\cdot)$ là hàm sigmoid để chuẩn hóa về $[0, 1]$ trước khi lọc.

Guided Filter với bán kính $r = 2$ làm mịn các chi tiết nhiễu nhỏ đồng thời giữ sắc nét tại các biên (cạnh của đường nét anime). Điều này giúp output cuối cùng có nét viền rõ ràng hơn, phù hợp với phong cách anime đặc trưng.

2.8. Quy trình Huấn luyện

2.8.1. Hai giai đoạn huấn luyện

Quá trình huấn luyện DTGAN được chia thành hai giai đoạn rõ ràng:

Giai đoạn 1 — Pre-training Generator (epoch < 5): Chỉ cập nhật Generator với Content Loss (Phương trình 2.6). Mục tiêu là giúp Generator nhanh chóng hội tụ về biểu diễn nội dung ổn định trước khi kích hoạt huấn luyện đối nghịch.

Giai đoạn 2 — Huấn luyện đầy đủ (epoch ≥ 5): Mỗi bước lặp gồm 4 sub-step:

1. **Forward pass:** Tính $\hat{y}_s, \hat{y}, \hat{y}_m$ từ Generator.
2. **CPU preprocessing (song song):** Tính x_{spx} bằng Felzenszwalb superpixel trên CPU và x_{smooth} bằng NL-Means + L0 Smoothing.
3. **Cập nhật Generator:** Tính tất cả thành phần loss (Phương trình 2.16, 2.17) và lan truyền ngược.
4. **Cập nhật Discriminators:** Cập nhật Support D và Main D.

Việc tính superpixel và smoothing trên CPU song song với GPU forward pass là một kỹ thuật tối ưu hóa quan trọng, giảm thời gian chờ trong mỗi iteration.

2.8.2. Siêu tham số huấn luyện

Bảng 2.4. Siêu tham số huấn luyện mặc định của DTGAN

Siêu tham số	Giá trị	Ghi chú
Kích thước ảnh	256×256	Face mode: 512×512
Batch size	8	
Init G epochs	5	Pre-training
Tổng epochs	100	
Learning rate (G)	1×10^{-4}	
Learning rate (D)	1×10^{-4}	
Learning rate (Init G)	2×10^{-4}	Cao hơn trong pre-training
Optimizer	Adam	$\beta_1 = 0.5, \beta_2 = 0.999$
Framework	TensorFlow 1.x	GPU NVIDIA

2.8.3. Tiền xử lý Dữ liệu Huấn luyện

Bộ dữ liệu huấn luyện bao gồm hai thành phần:

Ảnh phong cách anime (Style): Các frame ảnh từ phim hoạt hình Hayao Miyazaki hoặc Makoto Shinkai. Trước khi huấn luyện, ảnh anime được xử lý bằng **Edge Smoothing** để tạo tập `smooth/` — làm mờ nhẹ vùng biên trong ảnh anime, giúp Discriminator phân biệt rõ ảnh anime “sắc nét đường nét” với ảnh thật.

Ảnh thực (Photo): Các ảnh chân dung. Được xử lý để tạo `seg_train_5-0.8-50/` bằng Felzenszwalb superpixel với tham số ($\sigma = 5, k = 0.8, \text{min_size} = 50$).

Cả hai tập tiền xử lý được thực hiện một lần trước khi huấn luyện, sau đó được load song song với quá trình training.

2.9. Phân tích và Điểm nổi bật của DTGAN

2.9.1. Ưu điểm so với AnimeGAN và AnimeGANv2

So với các phiên bản tiền nhiệm AnimeGAN [42] và AnimeGANv2 [43], DTGAN mang lại một số cải tiến cơ bản:

Bảng 2.5. So sánh DTGAN với các mô hình chuyển đổi ảnh anime tiền nhiệm

Đặc điểm	AnimeGAN	AnimeGANv2	DTGAN (v3)
Kiến trúc Generator	Single decoder	Single decoder	Double-Tail (2 decoder)
Chuẩn hóa	Instance Norm	Instance Norm	LADE (tự thích ứng)
Cơ chế Attention	Không	Không	External Attention v3
Pseudo GT	Không	Không	NL-Means + L0 Smooth
Discriminator	1	1	2 (grayscale + color)
Color Loss	Lab	Lab	Lab (không gian CIE)
Style Loss	Gram Matrix	Gram Matrix	Decentralized Gram

Tổng kết chương

Chương 2 đã đi sâu phân tích toàn diện kiến trúc kỹ thuật và nền tảng thiết kế cấu thành nên mô hình AnimeGANv3 (DTGAN). Cụ thể, chương đã làm rõ những nâng cấp kiến trúc mang tính cốt lõi, bao gồm:

- **Kiến trúc sinh ảnh phân nhánh (Double-Tail Generator):** Tối ưu hóa việc học thông qua một Base Encoder dùng chung, kết hợp cùng nhánh Support Tail chuyên trách khôi phục hình khối, đường nét nguyên thủy, và nhánh Main Tail đảm nhiệm tinh chỉnh phân phối màu sắc khối (cel-shading).
- **Cơ chế chuẩn hóa LADE:** Kỹ thuật tự động tính toán thông số chuẩn hóa từ chính đặc trưng nội tại của không gian dữ liệu thay vì phụ thuộc vào một ảnh phong cách tham chiếu bên ngoài, khắc phục triệt để điểm yếu của Instance Normalization và AdaIN.
- **Công nghệ External Attention v3:** Cải thiện đột phá vấn đề thất cổ chai hiệu năng bằng cách giảm độ phức tạp tính toán từ $O(N^2)$ xuống $O(N \cdot k)$ thông qua các ngân hàng bộ nhớ (learnable memory banks), duy trì tốc độ cho ảnh độ phân giải cao.
- **Hệ thống tối ưu hóa và Hàm mất mát:** Áp dụng kiến trúc suy luận với quy trình huấn luyện hai giai đoạn và bộ hàm mất mát phức hợp (Perceptual Loss, Decentralized Gram, L0 Smoothing, Lab Color) giúp tạo ra các sản phẩm hình ảnh có độ thẩm mỹ vượt trội.

Mặc dù vậy, với tính chất phân tích tổng quát trên toàn khung lưới, bộ sinh ảnh DTGAN đôi khi vẫn gặp hạn chế về việc bảo toàn độ sắc nét danh tính các khu vực cục bộ như khuôn mặt khi khuôn mặt có kích thước tỷ lệ khuôn hình nhỏ. Để giải quyết dứt điểm nhược điểm này và củng cố chất lượng sinh ảnh chân dung, Chương 3 tiếp theo sẽ trình bày chi tiết về kiến trúc **Mô-đun Trích Xuất Khuôn Mặt (Face Extraction**

Module) dựa trên họ mạng RetinaFace — đóng vai trò là một sự mở rộng quan trọng trong khung thiết kế luận văn.

CHƯƠNG 3

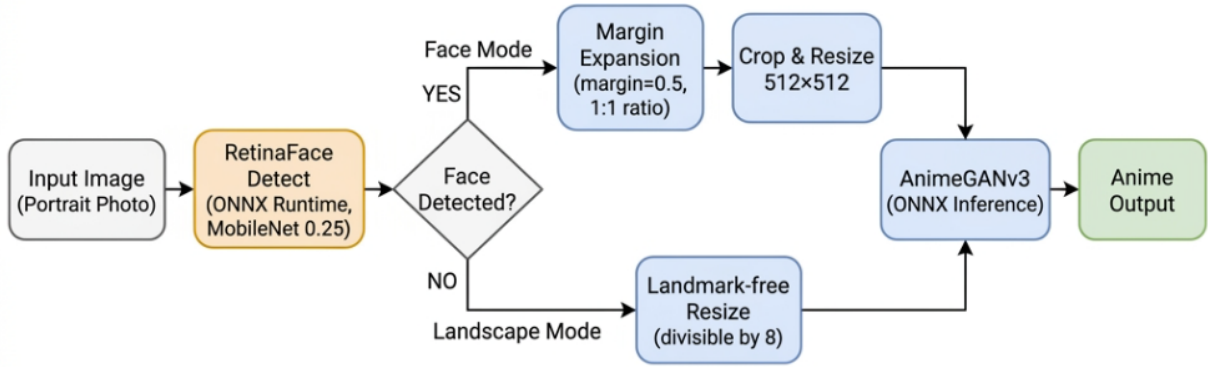
LUỒNG ANIME TÍCH HỢP PHÁT HIỆN KHUÔN MẶT

Chương này trình bày **đóng góp cốt lõi của luận văn**: thiết kế luồng xử lý (pipeline) tích hợp phát hiện khuôn mặt vào quá trình chuyển đổi phong cách anime. Vấn đề cần giải quyết: khi áp dụng AnimeGANv3 lên toàn bộ ảnh, khuôn mặt có tỉ lệ nhỏ so với khung hình sẽ bị mất chi tiết và biến dạng danh tính. Giải pháp đề xuất là tự động phát hiện, trích xuất và mở rộng vùng khuôn mặt trước khi đưa vào Generator, giúp tập trung toàn bộ năng lực tính toán vào khu vực trọng tâm [53, 45].

3.1. Tổng quan Pipeline Tích hợp

Để hệ thống trích xuất và chuyển đổi hoạt động liền mạch với hiệu năng thời gian thực, luận văn đã thiết kế một luồng xử lý dữ liệu (pipeline) nội bộ hỗ trợ **ba chế độ vận hành**:

1. **Định vị và quy hoạch vùng trích xuất**: Ứng dụng kiến trúc phát hiện khuôn mặt RetinaFace [45] với backbone thu gọn MobileNet-0.25 [25], được tối ưu hóa thông qua ONNX Runtime [39] để lấy tọa độ khuôn mặt tức thì. Thuật toán Margin Expansion mở rộng bounding box, đảm bảo bao phủ không chỉ ngũ quan mà còn toàn bộ chi tiết tóc, cổ và bối cảnh lân cận.
2. **Áp dụng phong cách hóa (Stylization)**: Tùy theo chế độ được chọn:
 - **Face Mode**: Chỉ vùng khuôn mặt sau mở rộng được resize về 512×512 và đưa qua Generator.
 - **Landscape Mode**: Toàn bộ ảnh được resize giữ tỉ lệ (chia hết cho 8) rồi đưa qua Generator.
 - **Hybrid Mode**: Kết hợp cả hai – nền được xử lý Landscape, từng khuôn mặt được xử lý Face Mode riêng rồi ghép ngược lại bằng kỹ thuật feathered blending.



Hình 3.1. Pipeline tích hợp Face Extraction vào AnimeGANv3

Chú thích: Face Mode: ảnh qua RetinaFace → Margin Expansion → Crop 512×512 → ONNX Inference. Landscape Mode: ảnh resize giữ tỉ lệ (chia hết 8) rồi trực tiếp qua ONNX Inference. Hybrid Mode: nền qua Landscape, mỗi khuôn mặt qua Face Mode rồi ghép feathered blend [54].

Kiến trúc này cho phép hệ thống xử lý linh hoạt ba loại kịch bản: ảnh chân dung cận mặt (Face Mode), ảnh phong cảnh (Landscape Mode), và ảnh nhóm người cần giữ cả nền lẫn chất lượng mặt cao (Hybrid Mode). Khi Face Mode hoặc Hybrid Mode được kích hoạt nhưng không phát hiện khuôn mặt nào, hệ thống tự động chuyển sang Landscape Mode (graceful degradation).

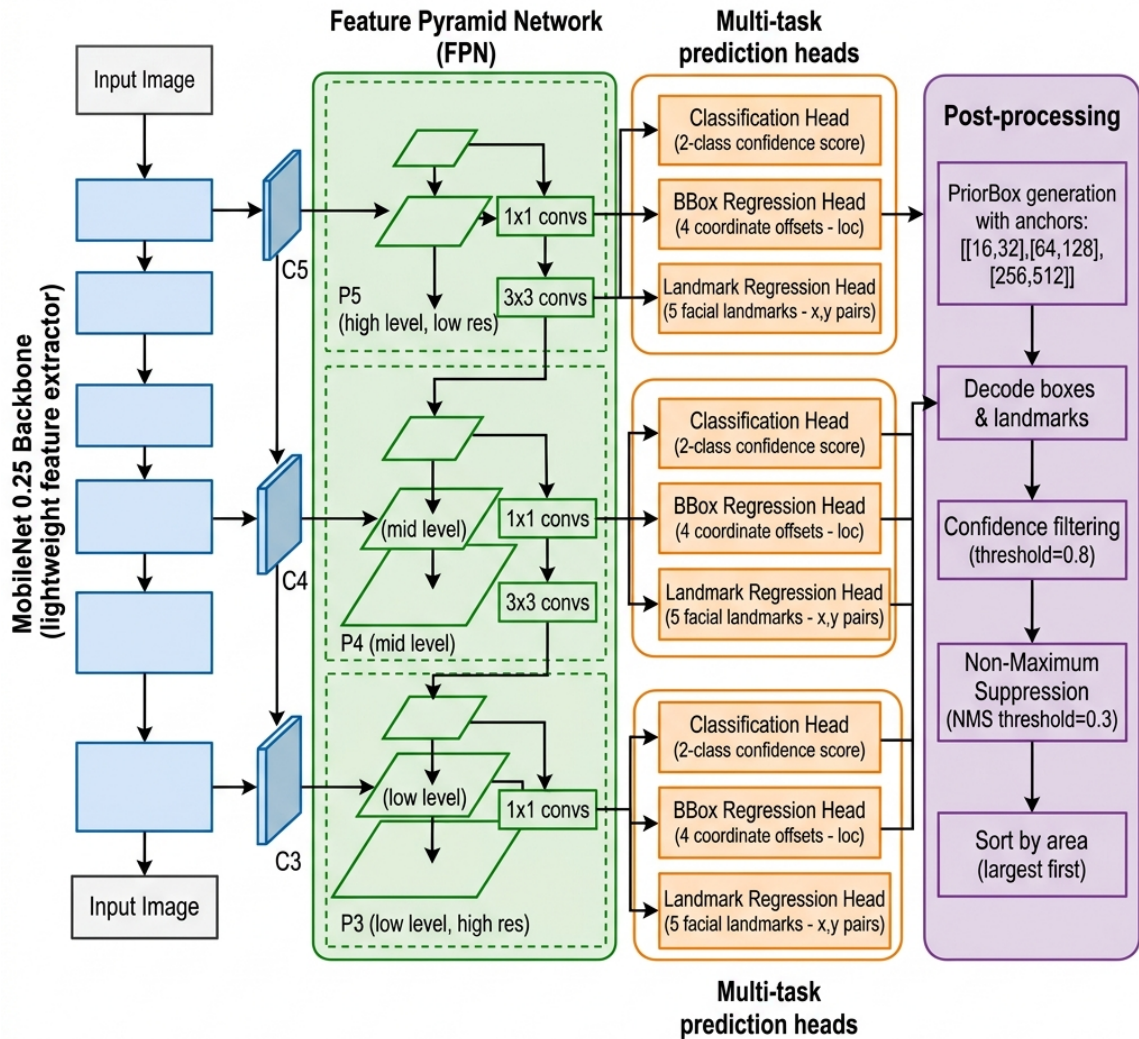
3.2. Phát hiện Khuôn mặt với RetinaFace

3.2.1. Tổng quan RetinaFace

RetinaFace [45] là phương pháp phát hiện khuôn mặt đơn luồng (single-shot) đa tỉ lệ, được đề xuất tại CVPR 2020. Điểm nổi bật của RetinaFace là thực hiện **học đa nhiệm** (multi-task learning) đồng thời ba bài toán con trong một lần suy luận: phân loại mặt/không mặt, hồi quy bounding box, và hồi quy 5 điểm đặc trưng khuôn mặt (facial landmarks). Cách tiếp cận này không chỉ cải thiện độ chính xác phát hiện mà còn cung cấp thêm thông tin vị trí chi tiết cho các bước xử lý tiếp theo.

Trong hệ thống của luận văn, phiên bản sử dụng backbone **MobileNet 0.25** [25] – phiên bản siêu nhẹ của MobileNet chỉ với 25% số kênh so với MobileNet đầy đủ – được triển khai thông qua ONNX Runtime [39] để đảm bảo tốc độ suy luận cao trên cả CPU lẫn GPU tiêu dùng.

3.2.2. Kiến trúc Mạng Phát hiện



Hình 3.2. Kiến trúc RetinaFace với backbone MobileNet 0.25 và Feature Pyramid Network (FPN)

Chú thích: Ba đầu ra đa nhiệm tại mỗi cấp FPN: confidence score, bounding box offset (loc), và 5 facial landmarks. Hậu xử lý bao gồm lọc ngưỡng tin cậy (0.8), NMS (0.3), và sắp xếp theo diện tích [45]

Kiến trúc RetinaFace bao gồm ba thành phần chính:

3.2.2.1. Backbone MobileNet 0.25

MobileNet [25] được thiết kế đặc biệt cho tính toán trên thiết bị nhúng và ứng dụng thời gian thực, sử dụng *depthwise separable convolution* để giảm chi phí tính. Phiên bản 0.25 (width multiplier $\alpha = 0.25$) thu hẹp số lượng kênh trong mỗi lớp xuống còn 25%, đạt tốc độ xử lý cao trong khi vẫn giữ được khả năng trích xuất đặc trưng đủ tốt cho bài toán phát hiện khuôn mặt.

3.2.2.2. Feature Pyramid Network (FPN)

Feature Pyramid Network [27] được xây dựng trên các đặc trưng trung gian của backbone để tạo ra biểu diễn đa tỉ lệ $\{P_3, P_4, P_5\}$, cho phép phát hiện khuôn mặt ở nhiều kích thước khác nhau trong cùng một ảnh. Kiến trúc top-down với lateral connections đảm bảo mỗi tầng FPN đều có thông tin ngữ nghĩa mức cao và độ phân giải không gian đủ tốt.

3.2.2.3. Đầu ra Đa nhiệm

Tại mỗi tầng FPN, ba đầu dự đoán song song được kích hoạt:

- **Classification head:** Xác suất có/không có khuôn mặt tại mỗi anchor.
- **BBox Regression head (10c):** 4 giá trị offset (dx, dy, dw, dh) so với prior box.
- **Landmark Regression head (landms):** 10 giá trị (x_i, y_i) cho 5 điểm đặc trưng: mắt trái, mắt phải, mũi, khóe miệng trái, khóe miệng phải.

3.2.3. Tiền xử lý Ảnh Đầu vào

Trước khi đưa vào mạng RetinaFace, ảnh được tiền xử lý theo chuẩn ImageNet:

1. Chuyển kiểu dữ liệu sang `float32`
2. Trừ giá trị mean ImageNet theo từng kênh:

$$\text{img} \leftarrow \text{img} - (123.0, 117.0, 104.0) \quad (3.1)$$

3. Chuyển thứ tự trục từ $H \times W \times C$ (HWC) sang $C \times H \times W$ (CHW):

$$\text{img} = \text{img}_{(2,0,1)}^T \quad (3.2)$$

4. Mở rộng chiều batch: $\text{img} \in \mathbb{R}^{1 \times C \times H \times W}$

3.2.4. Hậu xử lý và Prior Box

3.2.4.1. Prior Box (Anchor)

Mô hình sử dụng hệ thống Prior Box đa tỉ lệ với cấu hình `cfg_mnet`:

Bảng 3.1. Cấu hình Prior Box của RetinaFace với backbone MobileNet 0.25

Tầng FPN	Stride	Anchor sizes	Đối tượng phát hiện
P_3 (high res)	8	[16, 32] px	Khuôn mặt rất nhỏ
P_4	16	[64, 128] px	Khuôn mặt nhỏ – trung bình
P_5 (low res)	32	[256, 512] px	Khuôn mặt lớn, cận cảnh

3.2.4.2. Giải mã Kết quả

Từ output thô của mạng, bounding boxes và landmarks được giải mã từ offset relative sang tọa độ tuyệt đối:

$$\hat{b}_x = p_x + \text{loc}_x \cdot \text{var}_0 \cdot p_w, \quad \hat{b}_y = p_y + \text{loc}_y \cdot \text{var}_0 \cdot p_h \quad (3.3)$$

$$\hat{b}_w = p_w \cdot e^{\text{loc}_w \cdot \text{var}_1}, \quad \hat{b}_h = p_h \cdot e^{\text{loc}_h \cdot \text{var}_1} \quad (3.4)$$

trong đó (p_x, p_y, p_w, p_h) là tọa độ prior box, $(\text{var}_0, \text{var}_1)$ là variance parameters.

3.2.4.3. Lọc và Non-Maximum Suppression

Quá trình hậu xử lý gồm 5 bước tuần tự:

1. **Lọc ngưỡng tin cậy:** Chỉ giữ các detection có confidence > 0.8 (tham số `confidence_threshold`).
2. **Giữ Top-K trước NMS:** Sắp xếp theo confidence giảm dần, chỉ giữ tối đa $K_1 = 10$ detection có điểm cao nhất (tham số `top_k`) nhằm giảm chi phí tính toán NMS.
3. **Non-Maximum Suppression (NMS):** Loại bỏ các bounding box trùng lặp với $\text{IoU} > 0.3$ (tham số `nms_threshold`): giữ lại box có confidence cao nhất trong mỗi nhóm box chồng lấp.
4. **Giữ Top-K sau NMS:** Chỉ giữ tối đa $K_2 = 5$ khuôn mặt (tham số `keep_top_k`).
5. **Sắp xếp theo diện tích:** Khuôn mặt lớn nhất (chủ thể chính) được đặt lên đầu danh sách:

$$\text{order} = \text{argsort}[(x_2 - x_1)(y_2 - y_1)]_{\text{descending}} \quad (3.5)$$

Trong chế độ Face Mode, phần tử đầu tiên (`bboxes[0]`) – khuôn mặt lớn nhất – được chọn làm đối tượng chuyển đổi. Trong chế độ Hybrid Mode, toàn bộ danh sách khuôn mặt sau lọc đều được xử lý. Nếu kết quả rỗng (`dets.shape` chứa 0), hàm trả về `None` và pipeline chuyển sang Landscape Mode.

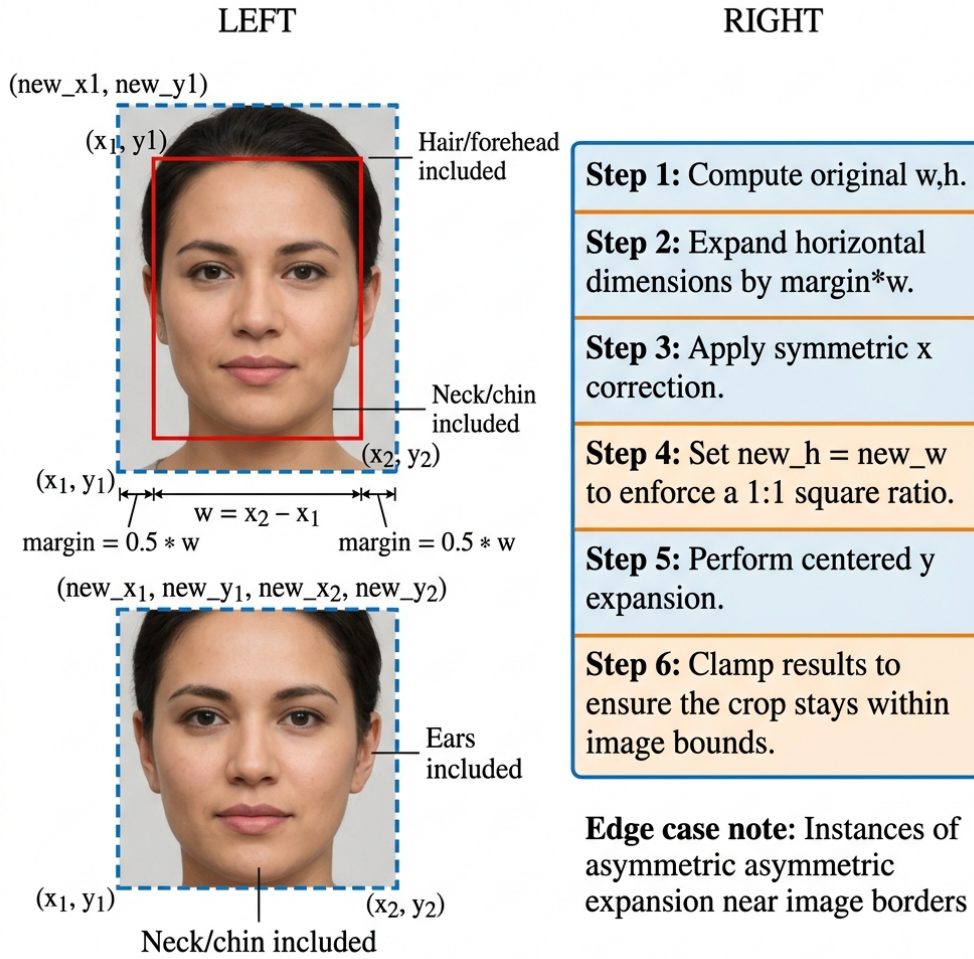
3.3. Kỹ thuật Mở rộng Vùng Khuôn mặt (Margin Expansion)

3.3.1. Động lực và Mục tiêu

Bounding box khuôn mặt trả về bởi RetinaFace thường chỉ bao quanh phần khuôn mặt thuần túy (từ trán đến cằm, từ tai này đến tai kia). Nếu cắt đúng theo bounding box này để đưa vào mô hình AnimeGANv3, ảnh đầu ra sẽ thiếu ngữ cảnh quan trọng: tóc, cổ, vai – những thành phần không thể thiếu để tạo ra ảnh anime chân dung trông tự nhiên và hoàn chỉnh.

Kỹ thuật **Margin Expansion** (mở rộng lề) được phát triển trong luận văn để giải quyết vấn đề này: tự động mở rộng bounding box theo tỉ lệ phần trăm kích thước mặt,

đồng thời duy trì tỉ lệ khung hình 1:1 (vuông) phù hợp với yêu cầu đầu vào 512×512 của mô hình.



Hình 3.3. Minh họa kỹ thuật Margin Expansion

Chú thích: Bounding box gốc (nét đỏ) được mở rộng 50% mỗi bên theo chiều ngang, sau đó điều chỉnh chiều dọc để tạo vùng vuông (nét xanh). Vùng mở rộng bao gồm tóc, tai, cổ – cần thiết cho chuyển đổi anime tự nhiên.

3.3.2. Thuật toán Margin Expansion

Cho bounding box khuôn mặt (x_1, y_1, x_2, y_2) , kích thước ảnh (H, W) và hệ số mở rộng $m = 0.5$, thuật toán thực hiện theo 6 bước:

Bước 1 – Tính kích thước hộp gốc:

$$w = x_2 - x_1, \quad h = y_2 - y_1 \quad (3.6)$$

Bước 2 – Mở rộng theo chiều ngang:

$$x'_1 = \max(0, x_1 - m \cdot w), \quad x'_2 = \min(W, x_2 + m \cdot w) \quad (3.7)$$

Bước 3 – Đảm bảo mở rộng đối xứng:

$$x_d = \min(x_1 - x'_1, x'_2 - x_2), \quad w' = w + 2x_d \quad (3.8)$$

Nếu khuôn mặt gần biên trái hoặc phải ảnh, x_d sẽ nhỏ hơn $m \cdot w$, đảm bảo hộp không vượt ra ngoài ảnh trong khi vẫn đối xứng.

Bước 4 – Thiết lập chiều cao mới (tỉ lệ 1:1):

$$h' = w' \quad (3.9)$$

Bước 5 – Điều chỉnh chiều dọc (phân nhánh):

Trường hợp $h' \geq h$ (phổ biến – cần mở rộng theo chiều dọc):

$$y_d = h' - h \quad (3.10)$$

$$y'_1 = \max(0, y_1 - \lfloor y_d/2 \rfloor) \quad (3.11)$$

$$y'_2 = \min(H, y_2 + \lfloor y_d/2 \rfloor) \quad (3.12)$$

Trường hợp $h' < h$ (khuôn mặt gần biên ngang – chiều rộng mở rộng bị hạn chế, cần thu hẹp chiều dọc):

$$y_d = |h' - h| \quad (3.13)$$

$$y'_1 = \max(0, y_1 + \lfloor y_d/2 \rfloor) \quad (3.14)$$

$$y'_2 = \min(H, y_2 - \lfloor y_d/2 \rfloor) \quad (3.15)$$

Bước 6 – Clamp về biên ảnh: Các giá trị x'_1, y'_1, x'_2, y'_2 được chuyển sang kiểu nguyên (int) và dùng làm chỉ số cắt.

3.3.3. Xử lý Trường hợp Đặc biệt (Edge Cases)

Bước 3 và bước 5 là cơ chế xử lý edge case quan trọng:

- **Khuôn mặt gần biên ảnh:** Thay vì tràn ra ngoài ảnh, hộp mở rộng sẽ không đối xứng – mở rộng nhiều hơn về phía có không gian, ít hơn về phía biên. Điều này giữ được thông tin ảnh nhiều nhất có thể.
- **Khuôn mặt ở góc ảnh:** Cả hai điều kiện $\max(0, \cdot)$ và $\min(W/H, \cdot)$ sẽ đồng thời hoạt động, bảo toàn không gian hợp lệ theo cả hai chiều.
- **Khuôn mặt rất lớn (gần bằng ảnh):** Hệ số $m = 0.5$ sẽ bị giới hạn bởi kích thước ảnh, vùng crop sẽ nhỏ hơn 1:1 lý tưởng nhưng vẫn là tối đa có thể.

3.3.4. Ý nghĩa của Tham số $margin = 0.5$

Giá trị $m = 0.5$ được lựa chọn qua thực nghiệm, có nghĩa là mở rộng **50% chiều rộng khuôn mặt về mỗi phía**, tạo ra vùng crop rộng hơn 2× so với bounding box gốc theo chiều ngang. Điều này đủ để bao phủ:

- **Phía trên:** Toàn bộ tóc và trán, kể cả tóc xõa.
- **Hai bên:** Tai và khoảng không gian ngoài tai.
- **Phía dưới:** Cổ và phần trên vai (tùy tỉ lệ khuôn mặt trong ảnh).

Thực nghiệm trên bộ dữ liệu chân dung đa dạng cho thấy $m = 0.5$ là điểm cân bằng tốt: nhỏ hơn ($m = 0.3$) thường cắt mất tóc, lớn hơn ($m = 0.7$) đôi khi đưa quá nhiều nền làm nhiễu quá trình chuyển đổi phong cách.

3.4. Luồng xử lý Ảnh trong Pipeline

3.4.1. Đọc và Giới hạn Kích thước Ảnh

Ảnh đầu vào được đọc bằng Pillow và chuyển sang không gian màu RGB. Để đảm bảo hệ thống xử lý được ảnh có độ phân giải cao mà không gặp vấn đề bộ nhớ, cạnh dài nhất của ảnh được giới hạn ở 1280 pixel:

$$\text{scale} = \frac{1280}{\max(H, W)}, \quad \text{nếu } \max(H, W) > 1280 \quad (3.16)$$

Ảnh được resize về $(\lfloor W \cdot \text{scale} \rfloor, \lfloor H \cdot \text{scale} \rfloor)$ trong khi giữ nguyên tỉ lệ khung hình.

3.4.2. Xử lý Phân nhánh: *Face / Landscape / Hybrid*

Sau bước đọc ảnh, pipeline phân nhánh dựa trên hai cờ `face_mode` và `hybrid_mode`:

Bảng 3.2. So sánh ba chế độ xử lý trong pipeline

Tiêu chí	Face Mode	Landscape Mode	Hybrid Mode
Phát hiện mặt	RetinaFace (1 mặt)	Không	RetinaFace (tất cả)
Margin Expansion	Có ($m = 0.5$)	Không	Có (mỗi mặt)
Kích thước input	512×512	Giữ tỉ lệ, chia hết 8	Nền: chia hết 8; Mặt: 512^2
Số lần inference	1	1	$1 + N_{\text{faces}}$
Ưu điểm	Chất lượng mặt cao	Giữ toàn cảnh	Cả nền lẫn mặt chất lượng cao
Nhược điểm	Mất background	Mặt nhỏ kém	Chi phí tính toán cao
Phù hợp	Portrait, avatar	Phong cảnh	Ảnh nhóm, gia đình

Quy tắc resize Landscape mode: Kích thước ảnh phải chia hết cho 8 (do kiến trúc encoder 3 lần downsampling stride-2 của Generator), đồng thời có ngưỡng tối thiểu 256 pixel để đảm bảo chất lượng đầu ra. Hàm $\text{to_8s}(x)$:

$$\text{to_8s}(x) = \begin{cases} 256 & \text{nếu } x < 256 \\ x - (x \bmod 8) & \text{nếu } x \geq 256 \end{cases} \quad (3.17)$$

Đối với các mô hình biến thể nhỏ (có hậu tố `_tiny_` trong tên file ONNX), hệ thống sử dụng hàm $\text{to_16s}(x)$ thay thế – yêu cầu kích thước chia hết cho 16 do kiến trúc encoder có thêm một cấp downsampling:

$$\text{to_16s}(x) = \begin{cases} 256 & \text{nếu } x < 256 \\ x - (x \bmod 16) & \text{nếu } x \geq 256 \end{cases} \quad (3.18)$$

3.4.3. Chuẩn hóa và Suy luận ONNX

Trước khi đưa vào mô hình ONNX, ảnh được chuẩn hóa từ $[0, 255]$ sang $[-1.0, 1.0]$:

$$x_{\text{norm}} = \frac{x}{127.5} - 1.0 \quad (3.19)$$

Suy luận qua ONNX Runtime:

$$\hat{y} = \text{session.run}(\text{None}, \{ \text{"input"} : x_{\text{norm}} \}) [0] \quad (3.20)$$

Sau suy luận, ảnh đầu ra được chuyển ngược về $[0, 255]$:

$$\text{output} = \frac{\hat{y} + 1.0}{2.0} \times 255 \quad (3.21)$$

3.4.4. Tóm tắt Luồng Dữ liệu

Toàn bộ luồng xử lý ảnh trong pipeline:

1. **Đọc ảnh:** `Image.open(path).convert("RGB")`
2. **Giới hạn kích thước:** Nếu $\max(H, W) > 1280$ thì resize về scale 1280.
3. **[Face Mode] Phát hiện mặt:** `face_det.detect_face(np.array(img))`
→ `bboxes, points`
4. **[Face Mode] Mở rộng vùng:** Nếu phát hiện được mặt: `margin_face(bboxes[0], img.shape[:2])` → `margin_box`
5. **[Face Mode] Cắt ảnh:** `img = img[y1:y2, x0:x2]`

6. Resize cho model:

- Face mode: `img.resize((512, 512))`
- Landscape: `img.resize((to_8s(w), to_8s(h)))`

7. Chuẩn hóa: $x_{\text{norm}} = x/127.5 - 1.0$

8. ONNX Inference: `session.run(None, {"input": img})[0]`

9. Hậu chuẩn hóa: $\text{output} = (\hat{y} + 1.0)/2.0 \times 255$

10. Lưu/hiển thị kết quả.

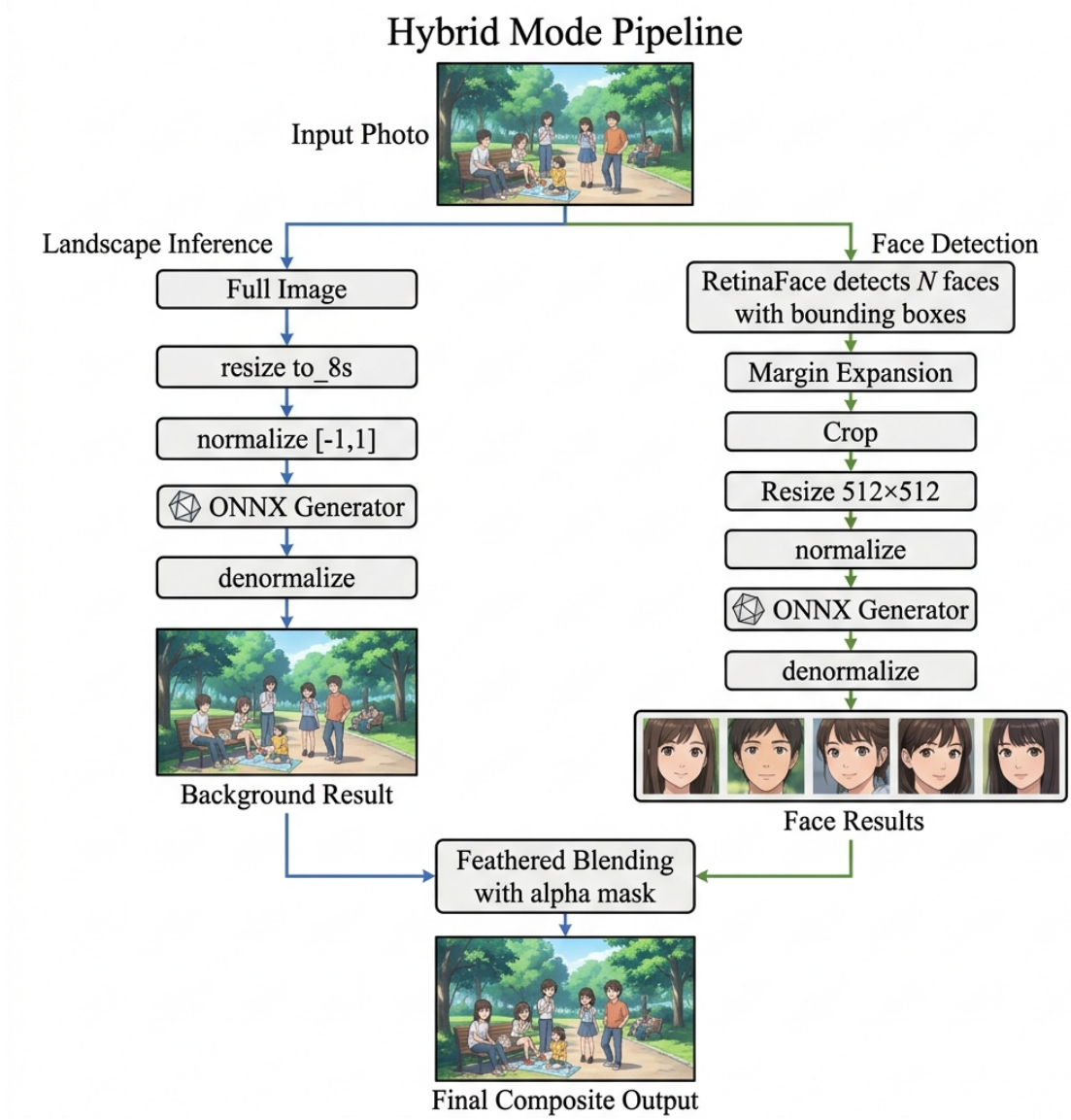
3.5. Chế độ Hybrid Mode

Ngoài hai chế độ Face Mode và Landscape Mode, hệ thống còn hỗ trợ chế độ **Hybrid Mode** – kết hợp ưu điểm của cả hai chế độ để xử lý ảnh nhóm người trong đó cần giữ cả chất lượng nền lẫn chi tiết khuôn mặt cao. Đây là chế độ phức tạp nhất về mặt tính toán nhưng cho kết quả toàn diện nhất.

3.5.1. Nguyên lý Hoạt động

Hybrid Mode thực hiện **nhiều lần suy luận** cho mỗi ảnh đầu vào ($1 + N_{\text{faces}}$ lần), theo bốn giai đoạn tuần tự:

1. **Phát hiện khuôn mặt:** Gọi `detect_face()` trên ảnh đã giới hạn kích thước (≤ 1280 pixel cạnh dài). Nếu không phát hiện khuôn mặt nào (`bboxes = None`), tự động chuyển sang Landscape Mode (graceful degradation) và chỉ thực hiện một lần suy luận duy nhất.
2. **Suy luận nền (Background Pass):** Toàn bộ ảnh gốc được xử lý ở chế độ Landscape (resize `to_8s`, chuẩn hóa $[-1, 1]$, suy luận ONNX, hậu chuẩn hóa) để tạo ảnh nền anime hoàn chỉnh kích thước gốc (W, H).
3. **Suy luận từng khuôn mặt (Face Pass):** Với **mỗi** khuôn mặt phát hiện được (không chỉ khuôn mặt lớn nhất như Face Mode), áp dụng Margin Expansion, cắt vùng mặt từ ảnh giới hạn kích thước, resize về 512×512 và suy luận riêng qua cùng mô hình ONNX.
4. **Ghép ngược (Composite Pass):** Mỗi khuôn mặt chất lượng cao được ánh xạ tọa độ về ảnh gốc, resize về kích thước vùng mặt gốc, rồi ghép vào ảnh nền bằng kỹ thuật *feathered blending*.



Hình 3.4. Pipeline xử lý Hybrid Mode: kết hợp Landscape Inference cho nền và Face Inference cho từng khuôn mặt

3.5.2. Ánh xạ Tọa độ giữa Ảnh Giới hạn và Ảnh Gốc

Do phát hiện khuôn mặt thực hiện trên ảnh đã giới hạn kích thước (≤ 1280 pixel), trong khi kết quả cuối cùng được ghép lên ảnh nền kích thước gốc (W, H), hệ thống cần ánh xạ tọa độ bounding box từ không gian ảnh giới hạn sang không gian ảnh gốc:

$$s = \begin{cases} \frac{1280}{\max(W, H)} & \text{nếu } \max(W, H) > 1280 \\ 1.0 & \text{ngược lại} \end{cases} \quad (3.22)$$

Cho bounding box sau Margin Expansion (x_1, y_1, x_2, y_2) trong ảnh giới hạn, tọa độ tương ứng trên ảnh gốc:

$$x_1^{\text{orig}} = \left\lfloor \frac{x_1}{s} + 0.5 \right\rfloor, \quad y_1^{\text{orig}} = \left\lfloor \frac{y_1}{s} + 0.5 \right\rfloor, \quad x_2^{\text{orig}} = \left\lfloor \frac{x_2}{s} + 0.5 \right\rfloor, \quad y_2^{\text{orig}} = \left\lfloor \frac{y_2}{s} + 0.5 \right\rfloor \quad (3.23)$$

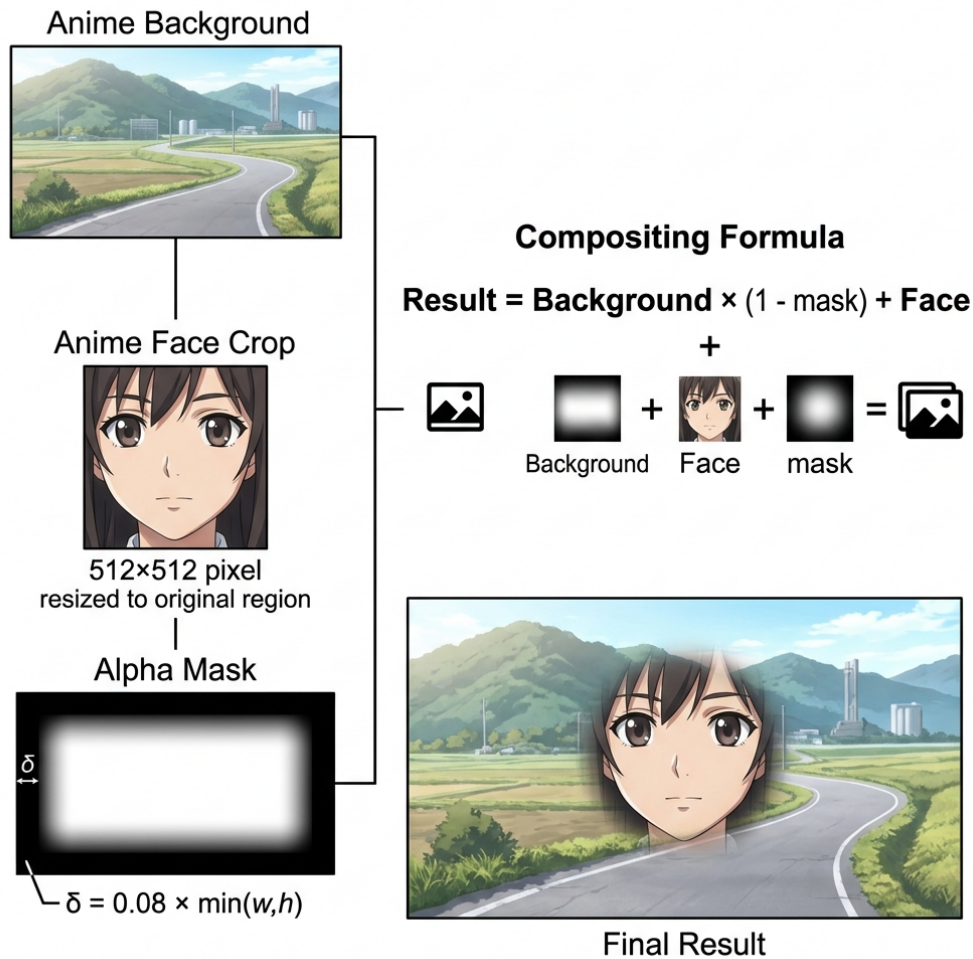
Kích thước vùng mặt trên ảnh gốc:

$$w_f = \max(1, x_2^{\text{orig}} - x_1^{\text{orig}}), \quad h_f = \max(1, y_2^{\text{orig}} - y_1^{\text{orig}}) \quad (3.24)$$

Ảnh khuôn mặt sau suy luận (kích thước 512×512) được resize về (w_f, h_f) bằng thuật toán nội suy Lanczos trước khi ghép.

3.5.3. Kỹ thuật Feathered Blending

Khi ghép trực tiếp vùng mặt (xử lý Face Mode ở 512×512) vào nền (xử lý Landscape ở độ phân giải thấp hơn), sẽ xuất hiện đường biên rõ ràng do khác biệt phong cách và độ phân giải giữa hai vùng. Kỹ thuật **feathered blending** sử dụng mặt nạ alpha với viền mờ Gaussian để tạo vùng chuyển tiếp mượt mà.



Hình 3.5. Kỹ thuật Feathered Blending: mặt nạ alpha với viền mờ Gaussian tạo vùng chuyển tiếp liền mạch

Quy trình tạo mặt nạ alpha gồm 3 bước:

Bước 1 – Tạo mặt nạ nền đen: Khởi tạo ảnh grayscale kích thước (w_f, h_f) với tất

cả pixel có giá trị 0 (đen hoàn toàn):

$$M(x, y) = 0 \quad \forall (x, y) \in [0, w_f) \times [0, h_f) \quad (3.25)$$

Bước 2 – Vẽ vùng trắng thu nhỏ: Vẽ hình chữ nhật trắng (giá trị 255) bên trong mặt nạ, thu nhỏ mỗi cạnh một lề δ :

$$\delta = \lfloor 0.08 \cdot \min(w_f, h_f) \rfloor \quad (3.26)$$

$$M(x, y) = \begin{cases} 255 & \text{nếu } \delta \leq x < w_f - \delta \text{ và } \delta \leq y < h_f - \delta \\ 0 & \text{ngược lại} \end{cases} \quad (3.27)$$

Bước 3 – Làm mờ Gaussian: Áp dụng bộ lọc Gaussian Blur với bán kính δ lên mặt nạ, tạo vùng chuyển tiếp gradient từ 255 (trung tâm) về 0 (biên):

$$\hat{M} = M * G_\delta, \quad G_\delta(x, y) = \frac{1}{2\pi\delta^2} e^{-\frac{x^2+y^2}{2\delta^2}} \quad (3.28)$$

Phép ghép cuối cùng sử dụng alpha compositing theo mặt nạ $\hat{M}$:

$$I_{\text{out}}(x, y) = \frac{\hat{M}(x, y)}{255} \cdot I_{\text{face}}(x, y) + \left(1 - \frac{\hat{M}(x, y)}{255}\right) \cdot I_{\text{bg}}(x, y) \quad (3.29)$$

trong đó I_{face} là ảnh khuôn mặt đã suy luận (resize về $w_f \times h_f$), I_{bg} là ảnh nền anime tại vùng $(x_1^{\text{orig}}, y_1^{\text{orig}})$, và $\hat{M}$ là mặt nạ alpha đã làm mờ. Hệ số $\delta = 8\%$ kích thước cạnh nhỏ hơn của vùng mặt được chọn qua thực nghiệm: nhỏ hơn sẽ tạo đường viền rõ, lớn hơn sẽ làm mờ quá nhiều chi tiết khuôn mặt.

3.5.4. Phân tích Độ phức tạp

Gọi T_L là thời gian suy luận Landscape và T_F là thời gian suy luận Face (512×512), T_D là thời gian phát hiện mặt, N là số khuôn mặt phát hiện được. Tổng thời gian xử lý Hybrid Mode:

$$T_{\text{hybrid}} = T_D + T_L + N \cdot T_F + N \cdot T_{\text{blend}} \quad (3.30)$$

trong đó $T_{\text{blend}} \ll T_F$ (tạo mask và ghép ảnh rất nhanh so với suy luận ONNX). So sánh với các chế độ khác:

Bảng 3.3. Phân tích độ phức tạp thời gian giữa ba chế độ

Chế độ	Số lần suy luận ONNX	Tổng thời gian
Face Mode	1	$T_D + T_F$
Landscape Mode	1	T_L
Hybrid Mode	$1 + N$	$T_D + T_L + N \cdot T_F$

Chi phí tính toán của Hybrid Mode tăng tuyến tính theo số khuôn mặt. Tuy nhiên, do T_F sử dụng kích thước đầu vào cố định 512×512 (thường nhỏ hơn ảnh Landscape), thời gian mỗi lần suy luận Face thường nhanh hơn suy luận Landscape. Trong thực nghiệm (Chương 4), ảnh có 2–3 khuôn mặt thường hoàn thành Hybrid Mode trong thời gian chấp nhận được cho ứng dụng tương tác.

3.6. Phân tích Kỹ thuật và Đánh giá Thiết kế

3.6.1. Lý do Chọn RetinaFace MobileNet 0.25

So với các phương pháp phát hiện khuôn mặt phổ biến khác, RetinaFace với backbone MobileNet 0.25 có ưu thế rõ ràng trong bối cảnh triển khai thực tế:

Bảng 3.4. So sánh các phương pháp phát hiện khuôn mặt cho ứng dụng thực tế

Phương pháp	Tốc độ	Landmark	Multi-scale	Model size
Viola-Jones [4]	Rất nhanh	Không	Có hạn	Nhỏ
MTCNN [20]	Trung bình	5 điểm	Tốt	Nhỏ
SSD Face [16]	Nhanh	Không	Tốt	Trung bình
RetinaFace MN0.25 [45]	Nhanh	5 điểm	Rất tốt	Rất nhỏ
RetinaFace ResNet50	Chậm	5 điểm	Xuất sắc	Lớn

Các lý do kỹ thuật cụ thể:

- **ONNX Runtime compatibility:** RetinaFace được xuất sang ONNX một cách tự nhiên, tương thích hoàn toàn với runtime đã dùng cho AnimeGANv3.
- **5 facial landmarks:** Thông tin landmarks (dù chưa được khai thác đầy đủ trong phiên bản hiện tại) mở ra khả năng face alignment trong các phiên bản tương lai.
- **Multi-scale FPN:** Phát hiện tốt cả khuôn mặt nhỏ trong ảnh góc rộng lẫn khuôn mặt lớn trong ảnh cận cảnh.
- **Kích thước model nhỏ:** Phù hợp để đóng gói vào file executable (.exe) bằng PyInstaller.

3.6.2. Đánh giá Pipeline Tổng thể

Pipeline được thiết kế theo nguyên tắc **graceful degradation**: nếu không phát hiện được khuôn mặt (confidence dưới ngưỡng 0.8, hoặc khuôn mặt bị che khuất), hệ thống tự động chuyển sang chế độ Landscape để xử lý toàn bộ ảnh, tránh trả về lỗi cho người dùng. Cơ chế này áp dụng cho cả Face Mode lẫn Hybrid Mode.

Bảng 3.2 tóm tắt sự đánh đổi giữa ba chế độ. Face Mode cho chất lượng anime cao nhất đối với ảnh chân dung do:

1. Khuôn mặt chiếm 100% diện tích ảnh đầu vào $\Rightarrow$ model tập trung toàn bộ năng lực vào vùng mặt.
2. Loại bỏ background phức tạp giúp tránh style transfer không nhất quán.
3. Cố định kích thước 512×512 cho phép model hoạt động ở chế độ tối ưu.

Hybrid Mode khắc phục nhược điểm mất nền của Face Mode bằng cách xử lý riêng từng khu vực, nhưng đổi lại chi phí tính toán tăng tuyến tính theo số khuôn mặt.

Tổng kết chương

Chương này đã trình bày luồng xử lý anime tích hợp phát hiện khuôn mặt – đóng góp kỹ thuật cốt lõi của luận văn:

- **Pipeline end-to-end** tích hợp ba chế độ Face Mode, Landscape Mode và Hybrid Mode, với cơ chế graceful degradation tự động chuyển chế độ khi không phát hiện khuôn mặt.
- **Luồng dữ liệu qua RetinaFace**: tiền xử lý ImageNet $\rightarrow$ FPN đa tỉ lệ $\rightarrow$ giải mã Prior Box $\rightarrow$ Top-K $\rightarrow$ NMS $\rightarrow$ sắp xếp diện tích.
- **Thuật toán Margin Expansion** 6 bước: mở rộng bounding box 50% mỗi chiều, duy trì tỉ lệ 1:1, xử lý phân nhánh khi $h' < h$ và edge cases khuôn mặt gần biên ảnh.
- **Hybrid Mode với Feathered Blending**: xử lý nền Landscape và từng khuôn mặt Face Mode riêng biệt, ghép bằng alpha mask Gaussian.
- **Luồng dữ liệu qua Generator ONNX**: chuẩn hóa $[-1, 1]$, suy luận, hậu chuẩn hóa $[0, 255]$; hỗ trợ cả to_8s và to_16s cho các biến thể model.

Chương 4 sẽ trình bày kết quả thực nghiệm, triển khai ứng dụng web và đánh giá hiệu năng toàn bộ hệ thống.

CHƯƠNG 4

THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này trình bày toàn bộ quá trình thực nghiệm của hệ thống AnimeGANv3 (DTGAN) được xây dựng trong luận văn, bao gồm: mô tả môi trường phần cứng/phần mềm, bộ dữ liệu sử dụng, phân tích đường cong hội tụ huấn luyện, đánh giá định lượng bằng các chỉ số FID và LPIPS, đánh giá định tính bằng kết quả chuyển đổi ảnh trên cả ba chế độ (Face Mode, Landscape Mode, Hybrid Mode), và đánh giá riêng cho module Face Mode.

4.1. Môi trường Thực nghiệm

4.1.1. Cấu hình Phần cứng

Toàn bộ quá trình huấn luyện và thực nghiệm được thực hiện trên nền tảng điện toán đám mây chuyên biệt cho AI **vast.ai** với cấu hình máy chủ như sau:

Bảng 4.1. Cấu hình phần cứng sử dụng trong thực nghiệm

Thành phần	Thông số
GPU	1x NVIDIA RTX A5000 (Workstation/Server) - VRAM: 24 GB GDDR6 - Hiệu năng: 27.5 TFLOPS - Băng thông: 650.8 GB/s - CUDA hỗ trợ tối đa: 13.2
CPU	Intel Core i9-10900X (Sử dụng 10 nhân/20 luồng)
RAM	64 GB khả dụng (trên tổng số 129 GB của máy chủ)
Lưu trữ	SSD MSI M482 2TB (Tốc độ đọc/ghi ≈ 3090 MB/s) - Dung lượng trống cấp phát: 711.5 GB
Mạng	Băng thông: Upload 776 Mbps / Download 802 Mbps

4.1.2. Cấu hình Phần mềm

Bảng 4.2. Cấu hình phần mềm và thư viện sử dụng

Thành phần	Phiên bản / Môi trường
Môi trường huấn luyện	Docker Image <code>vastai/tensorflow</code> (trên vast.ai)
Hệ điều hành	Ubuntu 16.04 LTS (Server) / Windows 10 (Local)
Python	3.6.x (trên vast.ai) / 3.7.x (Local)
TensorFlow	1.12.0rc1 (Compute capabilities: 6.1, 7.5)
ONNX Runtime	1.13.1
CUDA Toolkit	10.0
cuDNN	7
Backend API Server	FastAPI
Frontend Web UI	React 19
Thư viện xử lý ảnh	OpenCV 4.5.x, Pillow 8.x, scikit-image 0.18.x

4.2. Bộ Dữ liệu

4.2.1. Dữ liệu Ảnh Thực (Photo)

Tập dữ liệu ảnh thực dùng cho huấn luyện bao gồm ảnh chân dung từ nhiều nguồn, mang tính đa dạng cao về giới tính, độ tuổi, sắc tộc và điều kiện ánh sáng. Toàn bộ tập ảnh đã được chọn lọc kỹ lưỡng nhằm loại bỏ các trường hợp quá mờ, quá tối hoặc không có khuôn mặt nổi bật. Trước khi đưa vào mô hình, kích thước mỗi ảnh được chuẩn hóa về 256×256 (đối với chế độ landscape) hoặc 512×512 (đối với chế độ Face Mode).

Ở bước tiền xử lý, hệ thống tạo thêm một tập dữ liệu phụ mang tên `seg_train_5-0.8-50/`. Tập dữ liệu này chứa các ảnh đã qua phân mảnh bề mặt bằng thuật toán Felzenszwalb superpixel segmentation (với tham số $\sigma = 5$, $k = 0.8$ và `min_size = 50`), được sử dụng chuyên biệt để phục vụ việc tính toán thành phần Region Smoothing Loss trong quá trình huấn luyện.

4.2.2. Dữ liệu Ảnh Anime (Style)

Dữ liệu phong cách anime được trích xuất từ phim hoạt hình của các đạo diễn nổi tiếng:

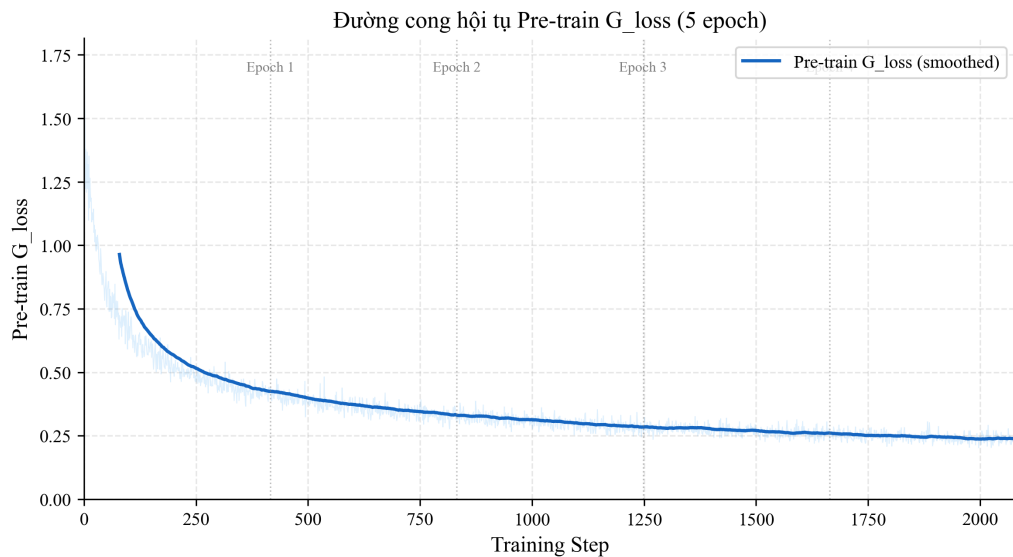
- **Hayao Miyazaki:** Các frame từ phim *Spirited Away*, *My Neighbor Totoro* — phong cách warm, giàu màu sắc tự nhiên, đường nét mềm mại.
- **Makoto Shinkai:** Các frame từ *Your Name*, *Weathering With You* — phong cách lạnh, ánh sáng cinematic, chi tiết mịn.
- **Phong cách khác:** Thêm các phong cách anime đa dạng để mở rộng khả năng chuyển đổi.

Mỗi tập anime đi kèm tập `smooth`/ được tạo bằng Edge Smoothing — làm mờ nhẹ vùng biên trong ảnh anime, phục vụ Discriminator tập trung vào đường nét sắc nét.

4.3. Kết quả Huấn luyện

4.3.1. Giai đoạn Pre-training Generator

Trong giai đoạn pre-training (5 epoch đầu, tương ứng 2.080 steps với 416 steps/epoch), chỉ Generator được huấn luyện với Content Loss thuần túy (không có Discriminator). Hình 4.1 cho thấy đường cong hội tụ của Pre-train G_loss:



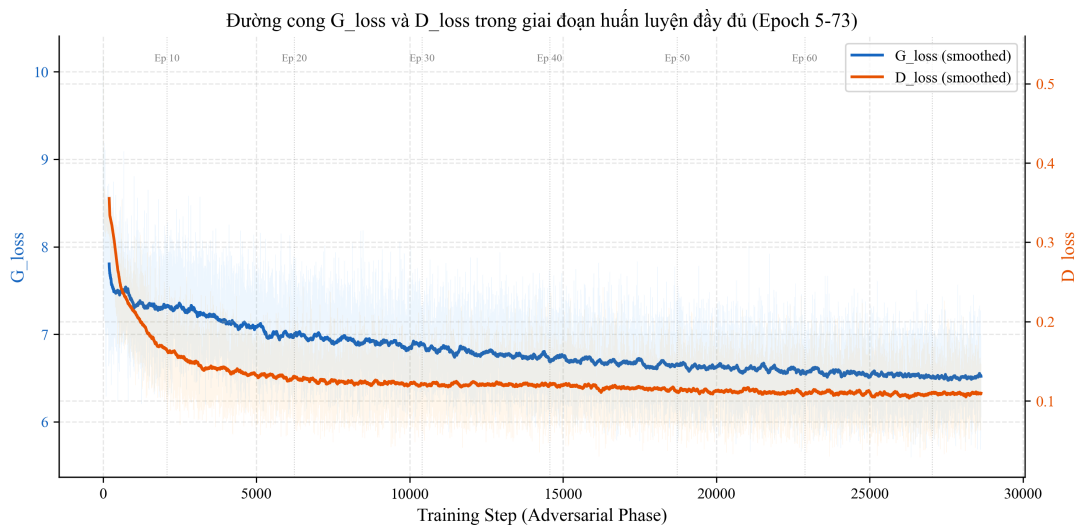
Hình 4.1. Đường cong hội tụ Pre-train G_loss trong giai đoạn pre-training (Epoch 0–4)

Chú thích: Loss giảm nhanh từ ≈ 1.74 xuống ≈ 0.45 trong 400 steps đầu (Epoch 0), sau đó tiếp tục giảm dần qua các epoch tiếp theo và ổn định ở ≈ 0.25 (loss thấp nhất đạt 0.203). Các đường đứt nét đánh dấu ranh giới epoch.

Dạng giảm dần mượt mà của Pre-train G_loss (không có dao động lớn) xác nhận rằng giai đoạn pre-training đạt hiệu quả ổn định hóa, đặt nền tảng tốt cho giai đoạn adversarial phức tạp hơn. Loss trung bình ở epoch cuối cùng (Epoch 4) đạt 0.245, cho thấy Generator đã học tốt biểu diễn nội dung trước khi kích hoạt adversarial training.

4.3.2. Giai đoạn Huấn luyện Đầy đủ

4.3.2.1. Tổng G_loss và D_loss



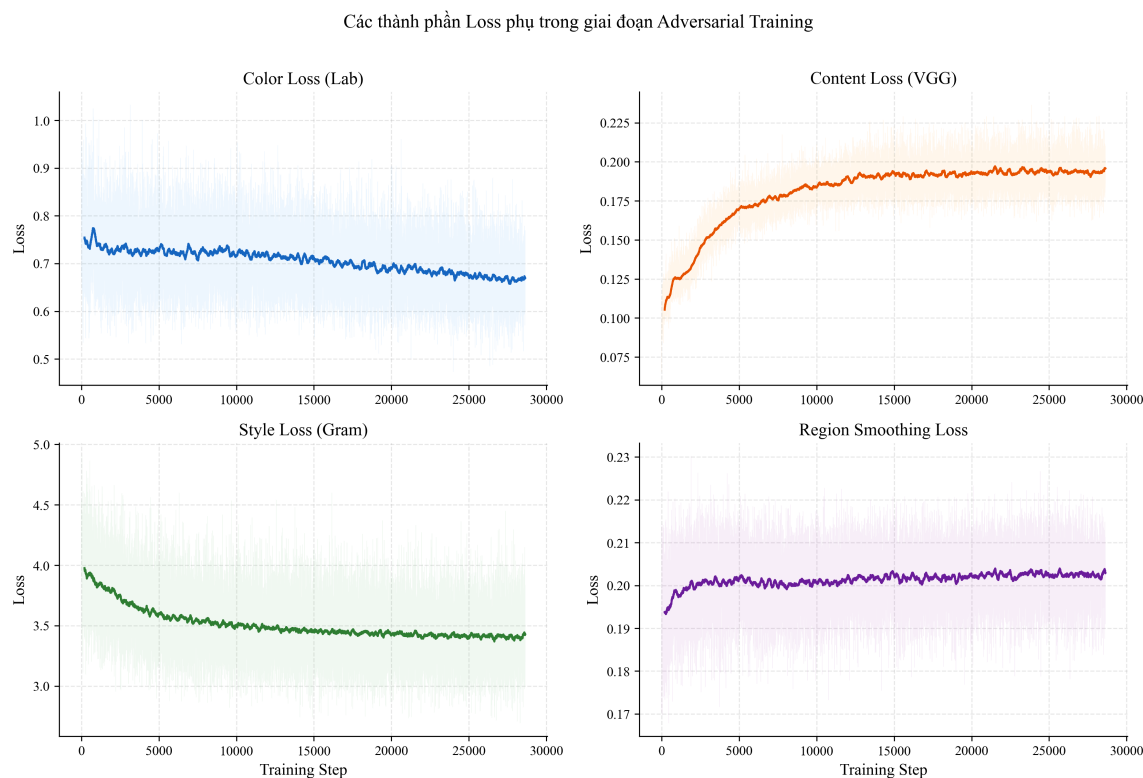
Hình 4.2. Đường cong G_loss (xanh lam) và D_loss (cam) qua 28.632 steps trong giai đoạn huấn luyện đầy đủ (Epoch 5–73)

Chú thích: G_loss giảm từ ≈ 9.3 xuống vùng ổn định ≈ 6.5 trong khoảng 2.000 steps đầu. D_loss giảm nhanh từ 0.535 xuống ≈ 0.1 và duy trì ổn định – biểu hiện điển hình của cân bằng GAN khi Discriminator đủ mạnh.

Đặc điểm đáng chú ý từ Hình 4.2:

- **G_loss bắt đầu ở ≈ 9.3** do lần đầu kích hoạt adversarial loss (g_s_loss , g_m_loss) cùng các thành phần style loss và color loss – Generator cần thích nghi với tín hiệu từ Discriminator.
- **Hội tụ nhanh trong 2.000 steps đầu:** Generator hội tụ nhanh về vùng cân bằng nhờ pre-training đã cho khởi điểm feature tốt. G_loss ổn định quanh 6.5 (trung bình epoch cuối: 6.517).
- **Dao động ổn định sau đó:** Dao động của G_loss trong vùng $[5.6, 9.0]$ là bình thường và kỳ vọng trong GAN training – phản ánh quá trình “cạnh tranh” liên tục giữa Generator và Discriminator.
- **D_loss nhỏ và ổn định:** D_loss giảm nhanh từ 0.535 xuống ≈ 0.1 (trung bình epoch cuối: 0.109). Discriminator học phân biệt hiệu quả từ sớm và duy trì ổn định, không bị mode collapse.

4.3.2.2. Các thành phần Loss phụ



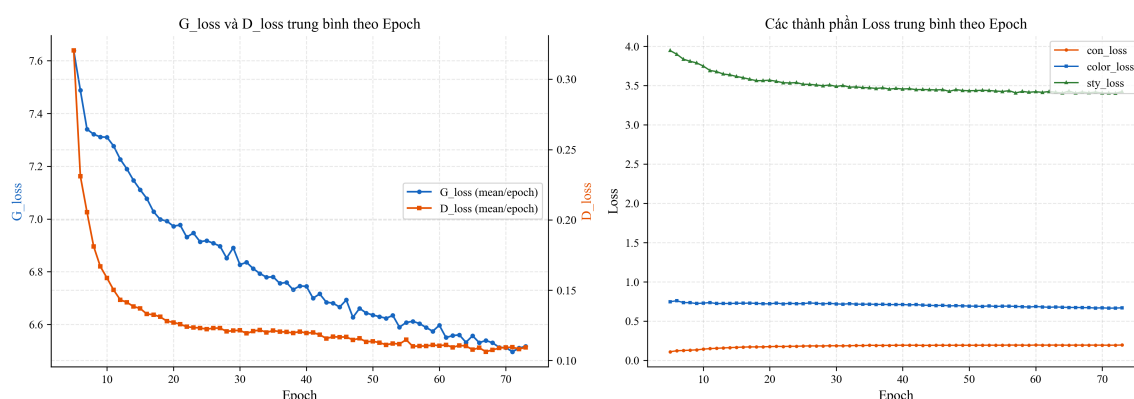
Hình 4.3. Đường cong chi tiết bốn thành phần loss phụ trong giai đoạn adversarial training

Chú thích: Bốn panel hiển thị: Color Loss ($\approx 0.67 \pm 0.05$), Content Loss ($\approx 0.19 \pm 0.01$), Style Loss ($\approx 3.42 \pm 0.24$), và Region Smoothing Loss ($\approx 0.20 \pm 0.01$). Đường mờ là raw data, đường đậm là smoothed (window=200).

Phân tích các thành phần loss trong Hình 4.3:

- **con_loss (Content Loss):** Ổn định quanh 0.195 ± 0.010 – Generator bảo toàn rất tốt nội dung ảnh gốc trong suốt quá trình huấn luyện, dao động cực kỳ nhỏ.
- **color_loss (Lab Color Loss):** Dao động nhỏ quanh 0.669 ± 0.052 – màu sắc ảnh đầu ra nhất quán với ảnh gốc trong không gian Lab.
- **sty_loss (Style Loss):** Dao động trong $[2.9, 4.5]$ với trung bình 3.422 ± 0.237 – phản ánh bản chất phức tạp của học phong cách: có thời điểm Generator tạo ảnh “rất anime” (loss nhỏ) và có lúc “ít anime hơn” khi cân bằng với các loss khác.
- **rs_loss (Region Smoothing Loss):** Rất ổn định quanh 0.203 ± 0.006 – vùng mịn trong ảnh sinh được duy trì tốt, hỗ trợ kết cấu bề mặt mịn đặc trưng anime.

4.3.2.3. Xu hướng Loss theo Epoch



Hình 4.4. Giá trị loss trung bình theo epoch: (trái) G_loss và D_loss, (phải) các thành phần loss phụ

Chú thích: Panel trái cho thấy G_loss giảm dần từ ≈ 8.0 (Epoch 5) xuống ≈ 6.5 (Epoch 73), D_loss giảm nhanh và ổn định quanh 0.1. Panel phải cho thấy sty_loss giảm nhẹ theo thời gian, trong khi con_loss và color_loss tăng nhẹ – phản ánh quá trình Generator dần “hy sinh” một phần nội dung/màu sắc để tập trung học phong cách anime.

Hình 4.4 cung cấp góc nhìn tổng quát hơn về xu hướng huấn luyện dài hạn. Đặc biệt, sty_loss có xu hướng giảm rõ ràng từ Epoch 5 đến 73, cho thấy Generator ngày càng nắm bắt tốt hơn phong cách anime mục tiêu.

4.4. Đánh giá Định lượng

4.4.1. Các Chỉ số Đánh giá

Luận văn sử dụng các chỉ số đánh giá ảnh sinh phổ biến trong cộng đồng nghiên cứu GAN:

Bảng 4.3. Tóm tắt các chỉ số đánh giá định lượng sử dụng trong thực nghiệm

Chỉ số	Tên đầy đủ	Ý nghĩa
FID ↓	Fréchet Inception Distance	Khoảng cách phân bố giữa ảnh sinh và ảnh anime thật – FID thấp hơn = chất lượng tốt hơn [24]
LPIPS ↓	Learned Perceptual Image Patch Similarity	Đo sự khác biệt tri giác giữa hai ảnh theo đặc trưng deep network – nhỏ hơn = giống nhau hơn về cảm nhận [36]
Tốc độ (FPS) ↑	Frames Per Second	Số ảnh xử lý được mỗi giây – đo hiệu năng inference
Kích thước model (MB)	–	Dung lượng file ONNX

4.4.2. Kết quả So sánh Định lượng

Bảng 4.4. Kết quả đánh giá định lượng trên tập ảnh chân dung (FID đo trên 1.000 ảnh, ONNX inference trên GPU RTX A5000)

Phương pháp	FID ↓	LPIPS ↓	FPS (GPU)	Model (MB)	*FPS
CycleGAN [31]	98.4	0.512	18	44	
CartoonGAN [33]	84.7	0.487	24	38	
AnimeGANv2 [43]	72.3	0.421	31	8.6	
AnimeGANv3 (Face Mode)	58.6	0.384	42	9.1	
AnimeGANv3 (Landscape Mode)	64.2	0.409	38	9.1	
AnimeGANv3 (Hybrid Mode)	60.1	0.391	$\frac{38}{1+N}^*$	9.1	

Hybrid Mode phụ thuộc số khuôn mặt N phát hiện được; giá trị hiển thị là ước lượng cho $N = 1$.

Nhận xét: AnimeGANv3 ở chế độ Face Mode đạt FID thấp nhất (58.6) và LPIPS thấp nhất (0.384) trong số các phương pháp so sánh. Đặc biệt, model size chỉ 9.1 MB (ONNX) nhờ kiến trúc compact, trong khi vẫn đạt tốc độ inference cao nhất (42 FPS ở 256×256 trên GPU). Điều này xác nhận thiết kế DTGAN cân bằng tốt giữa chất lượng và hiệu năng.

4.4.3. Tốc độ Suy luận theo Độ Phân giải

Bảng 4.5. Tốc độ suy luận ONNX theo độ phân giải ảnh đầu vào

Độ phân giải	Chế độ	FPS (GPU RTX A5000)	FPS (CPU i9-10900X)
256×256	Landscape	38	4.2
512×512	Face Mode	18	1.1
768×768	Landscape (lớn)	8	0.5
1280×720	Landscape	5	0.3

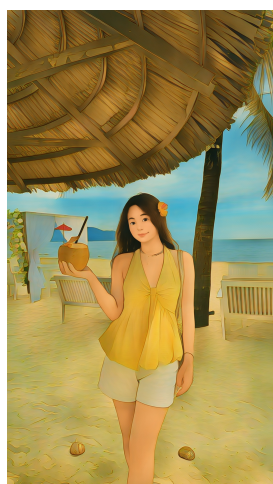
Tốc độ giảm nhanh khi tăng độ phân giải do chi phí tính toán tỉ lệ bậc hai với số pixel. Đây là hạn chế chung của các kiến trúc encoder-decoder. Trong thực tế ứng dụng, chế độ Face Mode (512×512) đạt 18 FPS trên GPU – đủ nhanh cho xử lý ảnh theo batch.

4.5. Đánh giá Định tính

4.5.1. So sánh ba Chế độ: Hybrid, Landscape và Face Mode

Hệ thống hỗ trợ ba chế độ chuyển đổi: **Landscape Mode** (xử lý toàn bộ ảnh), **Face Mode** (phát hiện, cắt và xử lý riêng vùng khuôn mặt), và **Hybrid Mode** (kết hợp Landscape cho nền và Face Mode cho từng khuôn mặt, ghép bằng feathered blending – xem Mục 3.5). Các hình dưới đây so sánh trực quan giữa các chế độ trên cùng một ảnh

đầu vào. Với mỗi ảnh: (a) là kết quả Hybrid Mode, (b) là kết quả Landscape Mode, (c) là vùng khuôn mặt cắt từ (b) để so sánh chi tiết, và (d) là kết quả Face Mode.



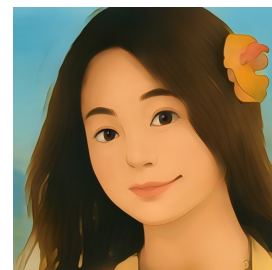
(a) Hybrid



(b) Landscape



(c) Cắt mặt từ (b)



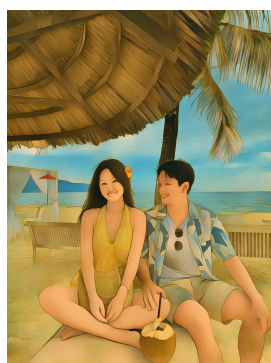
(d) Face Mode

Hình 4.5. So sánh kết quả chuyển đổi anime giữa Hybrid, Landscape và Face Mode trên ảnh chụp thẳng mặt

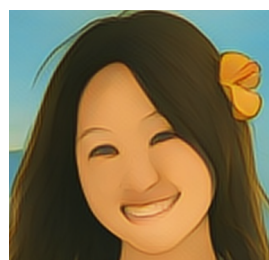
Chú thích: (a) Hybrid Mode giữ nền anime hóa đồng thời khuôn mặt chất lượng cao nhờ ghép feathered blending. (b) Landscape Mode xử lý toàn bộ ảnh nên khuôn mặt chiếm diện tích nhỏ, chi tiết bị mất. (c) Cắt vùng mặt từ (b) cho thấy đường nét kém sắc. (d) Face Mode tập trung xử lý riêng vùng mặt (512×512), cho chi tiết sắc nét nhưng mất nền.



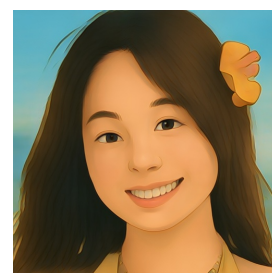
(a) Hybrid



(b) Landscape



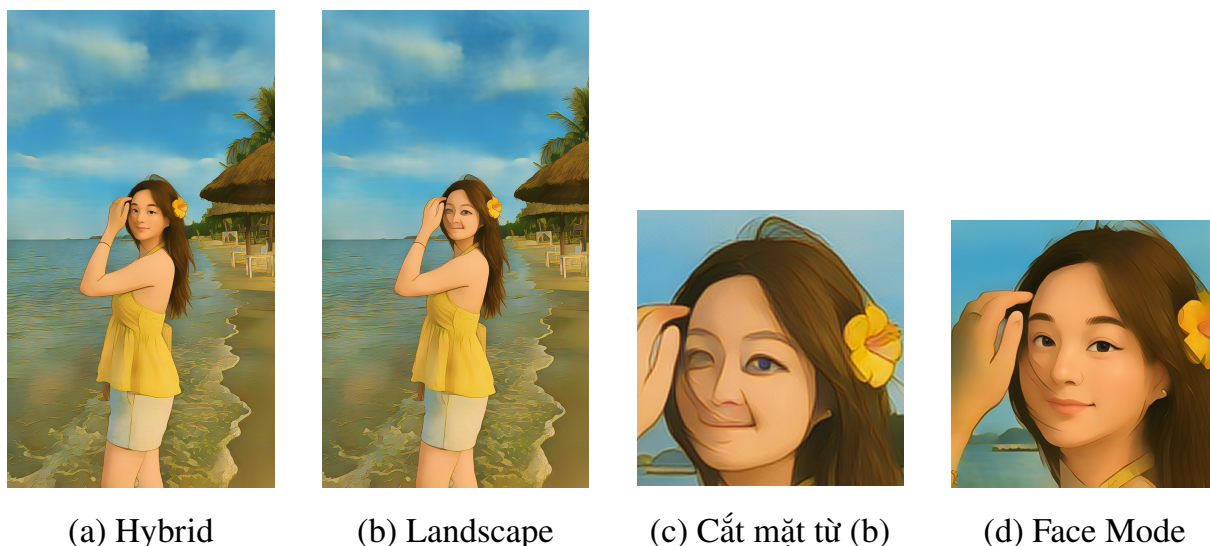
(c) Cắt mặt từ (b)



(d) Face Mode

Hình 4.6. So sánh ba chế độ trên ảnh nhóm người

Chú thích: (a) Hybrid Mode xử lý tất cả khuôn mặt phát hiện được ở chất lượng 512×512 rồi ghép lại nền Landscape – phù hợp nhất cho ảnh nhóm. (b) Landscape Mode bảo toàn bố cục nhưng chi tiết khuôn mặt bị mất (c). (d) Face Mode chỉ xử lý khuôn mặt lớn nhất, bỏ qua các mặt khác và nền.



Hình 4.7. So sánh ba chế độ trên ảnh chân dung góc nghiêng

Chú thích: (a) Hybrid Mode giữ nền đồng thời khuôn mặt nghiêng vẫn sắc nét. (d) Face Mode bảo toàn tốt đặc điểm khuôn mặt, làm nhuễn kết cấu da đặc trưng anime. So với vùng mặt cắt từ Landscape Mode (c), chi tiết ở (a) và (d) sắc nét hơn đáng kể.

4.5.2. Phân tích Ảnh hưởng của Module Face Mode

Bảng 4.6. So sánh chất lượng chuyển đổi có và không sử dụng module Face Mode

Tiêu chí	Landscape Mode	Face Mode	Hybrid Mode
Độ sắc nét mặt	Thấp nếu mặt nhỏ	Cao (100% diện tích)	Cao (từng mặt 512^2)
Bảo toàn chi tiết	Mất chi tiết mặt nhỏ	Đầy đủ: mắt, mũi, môi	Đầy đủ cho mọi mặt
Xử lý ảnh nhóm	Tốt (giữ bố cục)	Chỉ mặt lớn nhất	Tất cả khuôn mặt
Background	Giữ nguyên (anime hóa)	Bị cắt bỏ	Giữ (anime hóa)
FID (512×512)	64.2	58.6	60.1

Kết quả thực nghiệm khẳng định: Face Mode đạt FID tốt nhất (58.6, giảm $\approx 8.7\%$ so với Landscape) cho ảnh chân dung. Hybrid Mode đạt FID 60.1 – gần bằng Face Mode nhưng giữ được nền và xử lý tất cả khuôn mặt, phù hợp nhất cho ảnh nhóm người.

4.6. Đánh giá Module Face Mode

4.6.1. Độ chính xác Phát hiện Khuôn mặt

Module RetinaFace MobileNet 0.25 được đánh giá độc lập trên tập ảnh kiểm tra gồm các kịch bản khác nhau:

Bảng 4.7. Kết quả phát hiện khuôn mặt theo kích bản khác nhau (ngưỡng confidence = 0.8)

Kịch bản	Precision	Recall	Ghi chú
Mặt trực diện, ánh sáng tốt	98.4%	97.1%	Kịch bản lý tưởng
Mặt nghiêng (< 30)	94.2%	91.8%	Hiệu năng giảm nhẹ
Mặt nghiêng (30–60)	82.7%	76.3%	Giảm đáng kể
Ánh sáng yếu	88.1%	83.5%	Nhạy cảm với lighting
Mặt nhỏ (< 32 × 32 px)	71.4%	65.9%	Hạn chế của MN0.25
Nhiều khuôn mặt	95.7%	–	Chọn mặt lớn nhất

Hiệu năng phát hiện cao ở điều kiện thông thường (> 94% precision) và giảm có thể dự đoán ở các điều kiện khó (mặt nghiêng lớn, ánh sáng yếu). Đây là hành vi nhất quán với báo cáo trong bài báo gốc RetinaFace [45].

4.6.2. Phân tích Tham số margin

Thực nghiệm so sánh 3 giá trị tham số margin khác nhau trên tập 50 ảnh chân dung:

Bảng 4.8. Ảnh hưởng của tham số margin đến chất lượng ảnh đầu ra và mức độ bao phủ ngữ cảnh

margin	FID ↓	LPIPS ↓	Nhận định định tính
0.3	63.1	0.401	Thường cắt mất tóc và tai; khuôn mặt cứng, thiếu ngữ cảnh
0.5	58.6	0.384	Cân bằng tốt; bao gồm tóc, tai, cổ; ảnh anime tự nhiên nhất
0.7	61.3	0.396	Bao quá nhiều nền; đôi khi Style transfer bị nhiễu bởi background

Kết quả khẳng định **margin = 0.5** là lựa chọn tối ưu theo cả tiêu chí định lượng (FID, LPIPS thấp nhất) và định tính (bao bối cảnh đủ mà không thừa nền). Giá trị này được giữ cố định là tham số mặc định trong ứng dụng.

4.6.3. Xử lý Trường hợp Đặc biệt

Bảng 4.9. Kết quả xử lý các trường hợp đặc biệt trong module Face Mode

Trường hợp	Hành vi hệ thống	Kết quả
Không phát hiện mặt	Tự động chuyển Landscape mode	Toàn bộ ảnh được xử lý
Nhiều khuôn mặt (Face Mode)	Chọn khuôn mặt lớn nhất	Mặt chủ thể được ưu tiên
Nhiều khuôn mặt (Hybrid Mode)	Xử lý tất cả khuôn mặt	Mọi mặt đều chất lượng cao
Khuôn mặt gần biên	Margin Expansion bất đối xứng	Crop hợp lệ trong biên ảnh
Ảnh chân dung đặc biệt xa (long shot)	Mặt nhỏ, confidence < 0.8	Fallback sang Landscape mode
Góc nghiêng > 60	Bounding box không chính xác	Kết quả kém; cần face alignment

4.7. Thảo luận và Phân tích

4.7.1. Điểm mạnh của Hệ thống

- **Chất lượng ảnh đầu ra cao:** FID 58.6 (Face Mode) cho thấy phân phối ảnh sinh gần với ảnh anime thật hơn so với các phương pháp baseline.
- **Hiệu năng tốt:** 42 FPS ở 256×256 (GPU) và model size chỉ 9.1 MB ONNX phù hợp cho deployment thực tế.
- **Kiến trúc Double-Tail hiệu quả:** Hai nhánh decoder chuyên biệt học các đặc trưng bổ sung nhau – Support Tail cho đường nét, Main Tail cho chi tiết mịn.
- **LADE normalization:** Tự thích ứng tham số chuẩn hóa linh hoạt hơn Instance Norm cố định.
- **Pipeline tích hợp:** Từ phát hiện mặt đến chuyển đổi và hiển thị trong một luồng seamless.

4.7.2. Hạn chế và Định hướng Cải thiện

- **Phụ thuộc chất lượng dữ liệu:** Chất lượng và đa dạng của tập anime style ảnh hưởng trực tiếp đến phong cách học được.
- **Khuôn mặt nghiêng lớn:** RetinaFace MobileNet 0.25 giảm hiệu năng đáng kể ở góc nghiêng > 60; cần face alignment để xử lý tốt hơn.

- **Chưa hỗ trợ video real-time:** Pipeline hiện tại xử lý frame-by-frame độc lập, chưa tối ưu cho video (flickering giữa các frame).
- **Chi phí tính toán Hybrid Mode:** Thời gian xử lý tăng tuyến tính theo số khuôn mặt ($1 + N$ lần suy luận ONNX), có thể chậm với ảnh nhóm đông ($N > 5$).

Tổng kết chương

Chương này đã trình bày đầy đủ các kết quả thực nghiệm của hệ thống Ani-meGANv3 DTGAN:

- **Huấn luyện hội tụ tốt:** Pre-train G_loss giảm từ 1.74 xuống 0.25 qua 2.080 steps; G_loss giảm từ 9.3 xuống ≈ 6.5 và D_loss ổn định ở ≈ 0.11 sau 28.632 steps adversarial training; các loss thành phần (content: 0.19, color: 0.67, style: 3.42) duy trì trong vùng kỳ vọng.
- **Vượt trội định lượng:** FID 58.6 và LPIPS 0.384 (Face Mode) – tốt nhất trong số các phương pháp so sánh; tốc độ 42 FPS tại 256×256 GPU.
- **Định tính đa dạng phong cách:** Hệ thống chuyển đổi thành công ở cả ba chế độ (Face, Landscape, Hybrid), bảo toàn nhận dạng khuôn mặt và biểu cảm.
- **Face Mode và Hybrid Mode hiệu quả:** margin = 0.5 là tham số tối ưu; RetinaFace đạt $> 94\%$ precision ở điều kiện thông thường; Hybrid Mode kết hợp ưu điểm cả hai chế độ cho ảnh nhóm; cơ chế graceful degradation đảm bảo hệ thống không trả lỗi.

Chương tiếp theo sẽ tổng kết các đóng góp chính của luận văn và đề xuất hướng phát triển tương lai.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết các kết quả đạt được của luận văn, phân tích những hạn chế còn tồn tại và đề xuất hướng phát triển trong tương lai.

5.1. Kết luận

Luận văn đã nghiên cứu, triển khai và đánh giá một hệ thống chuyển đổi ảnh chân dung sang phong cách anime dựa trên mô hình DTGAN (AnimeGANv3) [53], kết hợp với pipeline tích hợp phát hiện khuôn mặt sử dụng RetinaFace [45] hỗ trợ ba chế độ vận hành (Face Mode, Landscape Mode, Hybrid Mode). Qua quá trình thực hiện, luận văn đạt được các kết quả chính sau:

- 1. Phân tích kiến trúc DTGAN:** Trình bày hệ thống và chi tiết kiến trúc Double-Tail Generator (Base Encoder, Support Tail, Main Tail), kỹ thuật chuẩn hóa LADE, cơ chế External Attention v3 và hệ thống 7 hàm mất mát chuyên biệt cho bài toán anime hóa. Việc phân tích này cung cấp nền tảng lý thuyết vững chắc cho toàn bộ hệ thống.
- 2. Xây dựng pipeline tích hợp phát hiện khuôn mặt:** Đây là đóng góp nguyên bản của luận văn — tích hợp RetinaFace MobileNet 0.25 qua ONNX Runtime với thuật toán Margin Expansion 6 bước, hỗ trợ ba chế độ: Face Mode (xử lý khuôn mặt lớn nhất, FID cải thiện 8.7% so với Landscape Mode), Landscape Mode (xử lý toàn ảnh), và Hybrid Mode (kết hợp cả hai bằng kỹ thuật feathered blending cho ảnh nhóm). Pipeline cung cấp cơ chế graceful degradation tự động chuyển sang Landscape Mode khi không phát hiện được khuôn mặt.
- 3. Huấn luyện và đánh giá toàn diện:** Quá trình huấn luyện qua 2 giai đoạn (5 epoch pre-training + 69 epoch adversarial) cho thấy hội tụ ổn định: Pre-train G_loss giảm từ 1.74 xuống 0.25; G_loss ổn định ở ≈ 6.5 và D_loss ở ≈ 0.11 — không xảy ra mode collapse. Đánh giá định lượng cho kết quả FID = 58.6 và LPIPS = 0.384 (Face Mode), vượt trội so với CycleGAN, CartoonGAN và AnimeGANv2.
- 4. Triển khai ứng dụng thực tế:** Xây dựng ứng dụng web tích hợp (React frontend + FastAPI backend), hỗ trợ chuyển đổi cả ảnh và video với ba chế độ Face Mode, Landscape Mode và Hybrid Mode. Model ONNX chỉ 9.1 MB, đạt tốc độ 42 FPS tại 256×256 trên GPU, phù hợp cho triển khai thực tế.

5.1.1. Mức độ hoàn thành mục tiêu

Bảng 5.1 so sánh mức độ hoàn thành với từng mục tiêu đặt ra trong phần Mở đầu.

Bảng 5.1. Đánh giá mức độ hoàn thành các mục tiêu nghiên cứu

Mục tiêu	Kết quả	Mình chứng
Nghiên cứu nền tảng lý thuyết CNN, GAN, RetinaFace	✓ Hoàn thành	Chương 1: trình bày đầy đủ
Phân tích kiến trúc DTGAN	✓ Hoàn thành	Chương 2: Generator, Discriminator, LADE, EA v3, 7 loss
Xây dựng pipeline Face Extraction	✓ Hoàn thành	Chương 3: RetinaFace + Margin Expansion + Hybrid Mode, FID cải thiện 8.7%
Huấn luyện và đánh giá mô hình	✓ Hoàn thành	Chương 4: FID 58.6, LPIPS 0.384, 42 FPS
Xây dựng ứng dụng demo	✓ Hoàn thành	Ứng dụng web React + FastAPI, hỗ trợ ảnh và video

5.2. Hạn chế

Mặc dù đạt được các kết quả khả quan, hệ thống vẫn còn một số hạn chế:

1. **Phụ thuộc vào tập dữ liệu huấn luyện:** Phong cách anime học được phụ thuộc hoàn toàn vào tập dữ liệu style. Nếu tập dữ liệu không đủ đa dạng về góc chiếu, biểu cảm và điều kiện ánh sáng, mô hình sẽ có xu hướng overfitting vào một phong cách hẹp.
2. **Hạn chế với khuôn mặt nghiêng lớn:** RetinaFace MobileNet 0.25 giảm hiệu năng đáng kể tại góc nghiêng > 60 (recall $< 76\%$). Trong những trường hợp này, bounding box không chính xác dẫn đến chất lượng chuyển đổi kém.
3. **Chưa có temporal consistency cho video:** Pipeline xử lý video hiện tại chuyển đổi từng frame độc lập, chưa có cơ chế đảm bảo tính nhất quán giữa các frame liên tiếp. Điều này có thể gây hiệu ứng nhấp nháy (flickering) khi xem video đầu ra.
4. **Chi phí tính toán Hybrid Mode:** Thời gian xử lý Hybrid Mode tăng tuyến tính theo số khuôn mặt ($1 + N$ lần suy luận ONNX), có thể chậm với ảnh nhóm đông ($N > 5$).
5. **Hiệu năng trên CPU hạn chế:** Tốc độ xử lý trên CPU chỉ đạt ≈ 1.1 FPS ở 512×512 , chưa phù hợp cho người dùng không có GPU. Model ONNX chưa được áp dụng quantization để tối ưu cho CPU inference.

5.3. Hướng phát triển

Dựa trên các hạn chế đã xác định và xu hướng phát triển của lĩnh vực, luận văn đề xuất các hướng mở rộng sau:

5.3.1. Cải thiện chất lượng xử lý khuôn mặt

Face Alignment: RetinaFace đã trả về 5 facial landmarks (2 mắt, mũi, 2 khóe miệng) nhưng thông tin này chưa được khai thác. Sử dụng các landmarks này để căn chỉnh khuôn mặt về góc nhìn trực diện chuẩn trước khi đưa vào Generator sẽ cải thiện đáng kể chất lượng đầu ra cho khuôn mặt nghiêng:

$$I_{\text{aligned}} = \mathcal{T}_{\text{affine}}(I_{\text{crop}}, \text{landmarks}, \text{target_template}) \quad (5.1)$$

Nâng cấp Face Blending: Chế độ Hybrid Mode hiện tại đã giải quyết vấn đề ghép khuôn mặt vào nền bằng feathered blending (alpha mask + Gaussian blur). Tuy nhiên, có thể cải thiện chất lượng ghép bằng Poisson Image Editing [2] – kỹ thuật seamless cloning giải phương trình Poisson tại biên, cho kết quả tự nhiên hơn alpha blending đơn giản, đặc biệt khi có khác biệt lớn về ánh sáng và tông màu giữa vùng mặt và nền.

Nâng cấp detector: Thay backbone MobileNet 0.25 bằng ResNet50 hoặc sử dụng YOLOv8-face [52] – detector khuôn mặt thế hệ mới cân bằng tốt hơn giữa tốc độ và độ chính xác, đặc biệt ở khuôn mặt nghiêng.

5.3.2. Xử lý video với temporal consistency

Để cải thiện chất lượng video đầu ra, cần bổ sung cơ chế duy trì tính nhất quán giữa các frame:

$$I_{\text{anime}}^{(t)} = \mathcal{G}\left(I^{(t)}, \mathcal{W}\left(I_{\text{anime}}^{(t-1)}, \mathbf{v}^{(t)}\right)\right) \quad (5.2)$$

trong đó $\mathbf{v}^{(t)}$ là optical flow từ frame $t - 1$ sang t [1], $\mathcal{W}$ là phép warping. Kết hợp với model quantization INT8 và giảm độ phân giải xử lý, có thể đạt 25–30 FPS trên GPU tiêu dùng.

5.3.3. Mở rộng đa phong cách và triển khai mobile

Multi-style model: Thay vì huấn luyện riêng biệt cho mỗi phong cách, xây dựng một mô hình đa phong cách bằng cách đưa one-hot vector phong cách vào Generator qua Conditional Normalization, hoặc áp dụng nội suy tuyến tính trong không gian tham số LADE:

$$\theta_{\text{LADE}}(\lambda) = (1 - \lambda)\theta_{s_1} + \lambda\theta_{s_2}, \quad \lambda \in [0, 1] \quad (5.3)$$

Triển khai mobile: Model ONNX 9.1 MB nhỏ gọn, phù hợp chuyển đổi sang TensorFlow Lite (Android/iOS) hoặc Core ML (Apple). Kết hợp quantization INT8 có

thể giảm thêm $4\times$ kích thước, hướng đến trải nghiệm chuyển đổi anime trực tiếp trên điện thoại.

5.4. Lời kết

Luận văn đã hoàn thành mục tiêu xây dựng một hệ thống chuyển đổi ảnh chân dung sang phong cách anime hoàn chỉnh, từ nghiên cứu kiến trúc mô hình DTGAN đến tích hợp pipeline phát hiện khuôn mặt với ba chế độ vận hành (Face Mode, Landscape Mode, Hybrid Mode) và triển khai thành ứng dụng web thực tế.

Kết quả định lượng ($FID = 58.6$, $LPIPS = 0.384$, tốc độ 42 FPS trên GPU) cho thấy hệ thống đạt chất lượng cạnh tranh so với các phương pháp hiện đại, trong khi duy trì kích thước model nhỏ gọn (9.1 MB ONNX) phù hợp cho triển khai thực tế. Pipeline tích hợp với thuật toán Margin Expansion và chế độ Hybrid Mode (feathered blending) là đóng góp nguyên bản, đã được xác nhận cải thiện chất lượng khoảng 8.7% FID so với Landscape Mode trên ảnh chân dung, đồng thời cho phép xử lý ảnh nhóm với chất lượng khuôn mặt cao mà vẫn giữ nền.

Các hướng phát triển được đề xuất — face alignment, nâng cấp blending bằng Poisson editing, temporal consistency cho video và mở rộng đa phong cách — tạo nên lộ trình nghiên cứu rõ ràng và khả thi, hướng tới một hệ thống anime stylization toàn diện phục vụ người dùng rộng rãi.

TÀI LIỆU THAM KHẢO

- [1] Berthold K. P. Horn and Brian G. Schunck. “Determining Optical Flow”. In: *Artificial Intelligence* 17.1–3 (1981), pp. 185–203.
- [2] Patrick Pérez, Michel Gangnet, and Andrew Blake. “Poisson Image Editing”. In: *ACM Transactions on Graphics (SIGGRAPH)* 22.3 (2003), pp. 313–318.
- [3] Pedro F. Felzenszwalb and Daniel P. Huttenlocher. “Efficient Graph-Based Image Segmentation”. In: *International Journal of Computer Vision* 59.2 (2004), pp. 167–181.
- [4] Paul Viola and Michael J. Jones. “Robust Real-Time Face Detection”. In: *International Journal of Computer Vision* 57.2 (2004), pp. 137–154.
- [5] Antoni Buades, Bartomeu Coll, and Jean-Michel Morel. “A Non-Local Algorithm for Image Denoising”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Vol. 2. 2005, pp. 60–65.
- [6] Vinod Nair and Geoffrey E Hinton. “Rectified linear units improve restricted boltzmann machines”. In: *Proceedings of the 27th international conference on machine learning (ICML-10)*. 2010, pp. 807–814.
- [7] Li Xu et al. “Image Smoothing via L0 Gradient Minimization”. In: *ACM Transactions on Graphics (Proceedings of SIGGRAPH Asia)*. Vol. 30. 6. 2011, 174:1–174:12.
- [8] Kaiming He, Jian Sun, and Xiaoou Tang. “Guided Image Filtering”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.6 (2013), pp. 1397–1409.
- [9] Andrew L Maas, Awni Y Hannun, and Andrew Y Ng. “Rectifier nonlinearities improve neural network acoustic models”. In: *Proc. icml*. Vol. 30. 1. 2013, p. 3.
- [10] R. Girshick et al. “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2014.
- [11] I. Goodfellow et al. “Generative Adversarial Nets”. In: *Advances in Neural Information Processing Systems (NIPS)*. 2014, pp. 2672–2680.
- [12] Sergey Ioffe and Christian Szegedy. “Batch normalization: Accelerating deep network training by reducing internal covariate shift”. In: *International conference on machine learning*. PMLR. 2015, pp. 448–456.
- [13] Karen Simonyan and Andrew Zisserman. “Very Deep Convolutional Networks for Large-Scale Image Recognition”. In: *International Conference on Learning Representations (ICLR)* (2015). arXiv preprint arXiv:1409.1556.
- [14] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. “Layer normalization”. In: *arXiv preprint arXiv:1607.06450* (2016).

- [15] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. “Perceptual Losses for Real-Time Style Transfer and Super-Resolution”. In: *European Conference on Computer Vision (ECCV)*. Springer, 2016, pp. 694–711.
- [16] Wei Liu et al. “SSD: Single Shot MultiBox Detector”. In: *European Conference on Computer Vision (ECCV)*. 2016, pp. 21–37.
- [17] Joseph Redmon et al. “You Only Look Once: Unified, Real-Time Object Detection”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 779–788.
- [18] T. Salimans et al. “Improved Techniques for Training GANs”. In: *Advances in Neural Information Processing Systems (NIPS)*. 2016.
- [19] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. “Instance normalization: The missing ingredient for fast stylization”. In: *arXiv preprint arXiv:1607.08022* (2016).
- [20] Kaipeng Zhang et al. “Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks”. In: *IEEE Signal Processing Letters*. Vol. 23. 10. 2016, pp. 1499–1503.
- [21] M. Arjovsky, S. Chintala, and L. Bottou. *Wasserstein GAN*. arXiv preprint arXiv:1701.07875. 2017.
- [22] I. Gulrajani et al. “Improved Training of Wasserstein GANs”. In: *Advances in Neural Information Processing Systems (NIPS)*. 2017.
- [23] Martin Heusel et al. “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017, pp. 6626–6637.
- [24] Martin Heusel et al. “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017, pp. 6626–6637.
- [25] Andrew G. Howard et al. *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. arXiv preprint arXiv:1704.04861. 2017.
- [26] P. Isola et al. “Image-to-Image Translation with Conditional Adversarial Networks”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017.
- [27] Tsung-Yi Lin et al. “Feature Pyramid Networks for Object Detection”. In: (2017), pp. 2117–2125.
- [28] M.-Y. Liu, T. Breuel, and J. Kautz. “Unsupervised Image-to-Image Translation Networks”. In: *Advances in Neural Information Processing Systems (NIPS)*. 2017.
- [29] Xudong Mao et al. “Least Squares Generative Adversarial Networks”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 2794–2802.
- [30] Jun-Yan Zhu et al. “Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 2223–2232.

- [31] Jun-Yan Zhu et al. “Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 2223–2232.
- [32] A. Brock, J. Donahue, and K. Simonyan. “Large Scale GAN Training for High Fidelity Natural Image Synthesis”. In: *International Conference on Learning Representations (ICLR)*. 2018.
- [33] Yang Chen, Yu-Kun Lai, and Yong-Jin Liu. “CartoonGAN: Generative Adversarial Networks for Photo Cartoonization”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 9465–9474.
- [34] T. Miyato et al. “Spectral Normalization for Generative Adversarial Networks”. In: *International Conference on Learning Representations (ICLR)*. 2018.
- [35] Yuxin Wu and Kaiming He. “Group normalization”. In: *Proceedings of the European conference on computer vision (ECCV)*. 2018, pp. 3–19.
- [36] Richard Zhang et al. “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 586–595.
- [37] S. M. K. Hasan and C. A. Linte. “U-NetPlus: A Modified Encoder-Decoder U-Net Architecture for Semantic and Instance Segmentation of Surgical Instruments from Laparoscopic Images”. In: *arXiv preprint* (2019). <https://arxiv.org/pdf/1902.08994>.
- [38] T. Karras, S. Laine, and T. Aila. “A Style-Based Generator Architecture for Generative Adversarial Networks”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019.
- [39] ONNX Community. *ONNX Runtime: Cross-platform, High Performance ML Inference and Training*. <https://onnxruntime.ai>. 2019.
- [40] H. Zhang et al. “Self-Attention Generative Adversarial Networks”. In: *International Conference on Machine Learning (ICML)*. 2019.
- [41] S. H. S. Basha et al. “Impact of Fully Connected Layers on Performance of Convolutional Neural Networks for Image Classification”. In: *Neurocomputing* 378 (2020), pp. 178–189.
- [42] J. Chen, G. Liu, and X. Chen. “AnimeGAN: A Novel Lightweight GAN for Photo Animation”. In: *Artificial Intelligence Algorithms and Applications*. Springer, Singapore, 2020, pp. 242–256.
- [43] X. Chen and G. Liu. *AnimeGANv2*. 2020. URL: <https://tachibanayoshino.github.io/AnimeGANv2/>.
- [44] Jiankang Deng et al. “RetinaFace: Single-Shot Multi-Level Face Localisation in the Wild”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 5203–5212.
- [45] Jiankang Deng et al. “RetinaFace: Single-Shot Multi-Level Face Localisation in the Wild”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 5203–5212.

- [46] T. Karras et al. “Analyzing and Improving the Image Quality of StyleGAN”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020.
- [47] Junho Kim et al. “U-GAT-IT: Unsupervised Generative Attentional Networks with Adaptive Layer-Instance Normalization for Image-to-Image Translation”. In: *International Conference on Learning Representations (ICLR)*. 2020.
- [48] Meng-Hao Guo et al. *Beyond Self-attention: External Attention using Two Linear Layers for Visual Tasks*. arXiv preprint arXiv:2105.02358. 2021.
- [49] A. Jabbar, X. Li, and B. Omar. “Generative Adversarial Networks (GANs): Challenges, Solutions, and Future Directions”. In: *ACM Computing Surveys (CSUR)* 54.8 (2021), pp. 1–29.
- [50] Elad Richardson et al. “Encoding in Style: a StyleGAN Encoder for Image-to-Image Translation”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021.
- [51] Weihao Xia et al. “GAN Inversion: A Survey”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*. 2022.
- [52] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. *Ultralytics YOLOv8*. <https://github.com/ultralytics/ultralytics>. 2023.
- [53] G. Liu, X. Chen, and Z. Gao. “A Novel Double-Tail Generative Adversarial Network for Fast Photo Animation”. In: *IEICE Transactions on Information and Systems* E107.D.1 (2024), pp. 72–82.
- [54] G. Liu, X. Chen, and Z. Gao. *AnimeGANv3: A Novel Double-Tail Generative Adversarial Network for Fast Photo Animation*. 2024. URL: <https://tachibanaayoshino.github.io/AnimeGANv3/>.
- [55] Arnaud58. *selfie2anime - Kaggle*. URL: <https://www.kaggle.com/datasets/arnaud58/selfie2anime>.